



数字逻辑与处理器基础

Fundamentals of Digital Logic and Processor



第五讲 时序逻辑 (2/2)

李学清

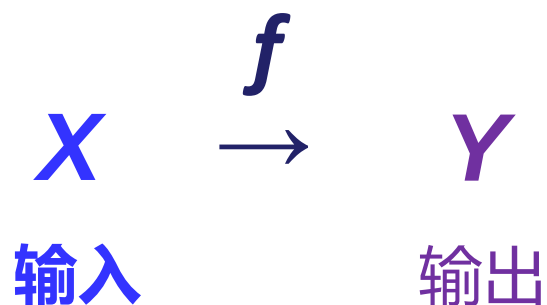
#腾讯会议：453-1488-6548

助教：曾令安*/唐文骏/陈仲豪

*本节内容负责答疑助教

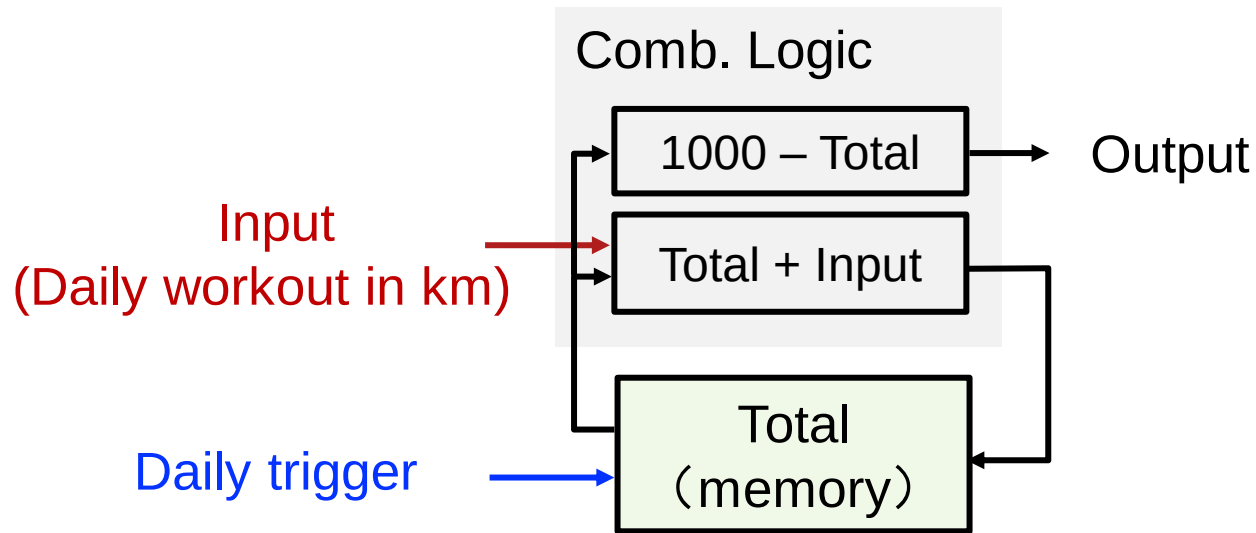
2024年10月14日

时序逻辑



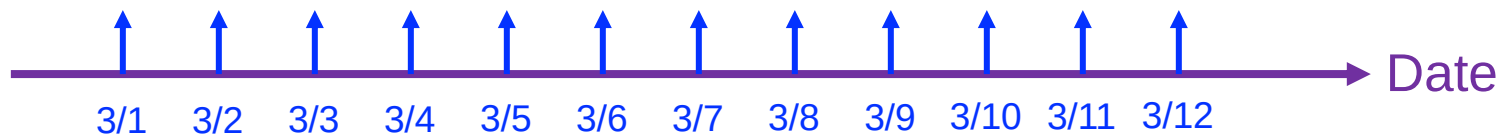
周	日期	暂定主要内容
1	9/9	绪论
2	9/21	布尔代数 (中秋节调休)
3	9/23	组合逻辑
4	9/30	时序逻辑
5	10/7	国庆节放假
6	10/14	时序逻辑
7	10/21	计算机指令系统
8	待定	期中考试 (TBD)
9	11/4	计算机指令系统
10	11/11	微处理器
11	11/18	微处理器
12	11/25	流水线
13	12/2	流水线
13	12/7	流水线/存储器 (调课)
15	12/16	存储器
16	12/23	存储器/IO与总线/总结

A Follow-up: Workout Example



Total States $\leq g(\text{Input}, \text{Previous States})$

Output = $f(\text{Input}, \text{Total States})$

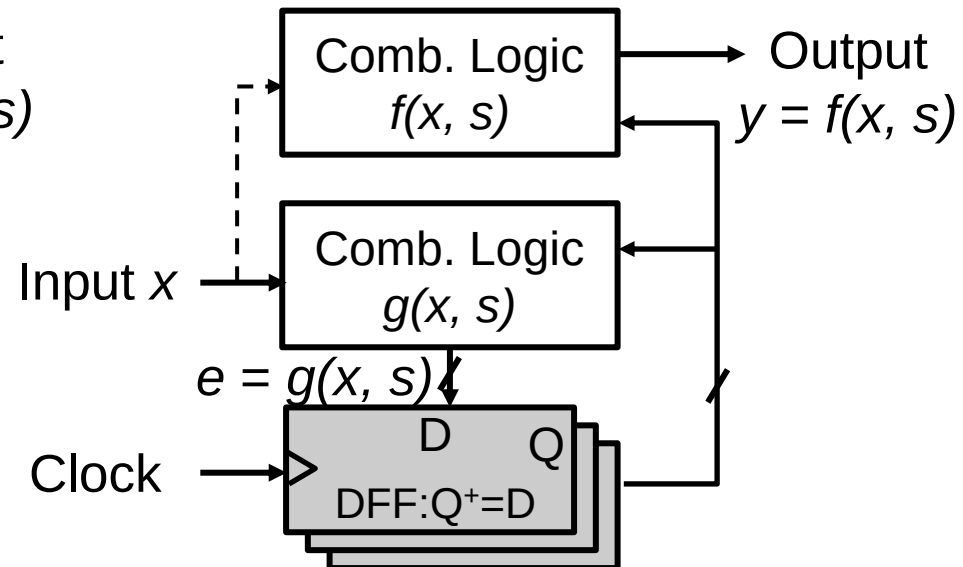
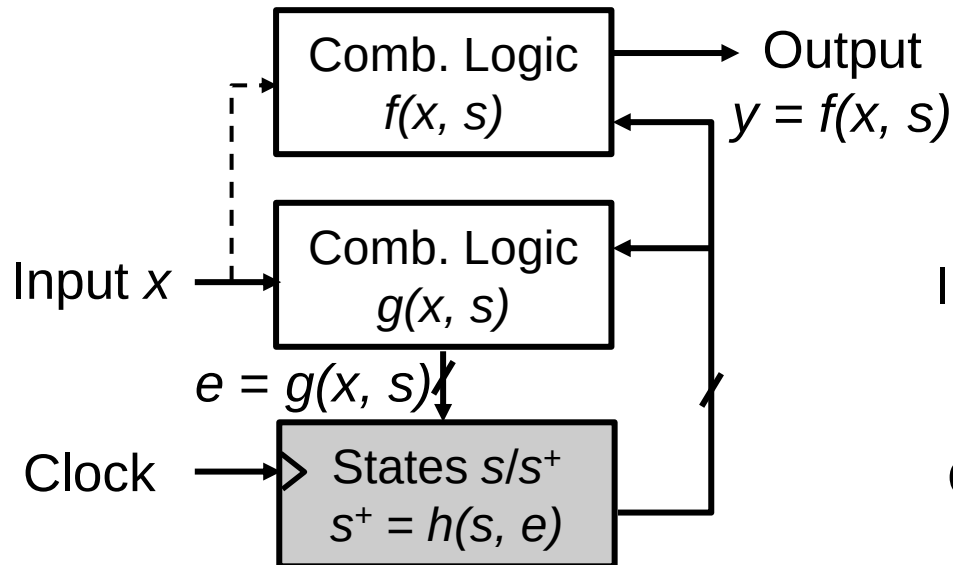


本节课学习目标

- 理解时序逻辑概念、理论、指标、非理想因素
 - 对比组合逻辑
- 分析和设计时序逻辑电路
 - 优化
 - 使用
- 树立过程同步化处理意识

Synchronized computing

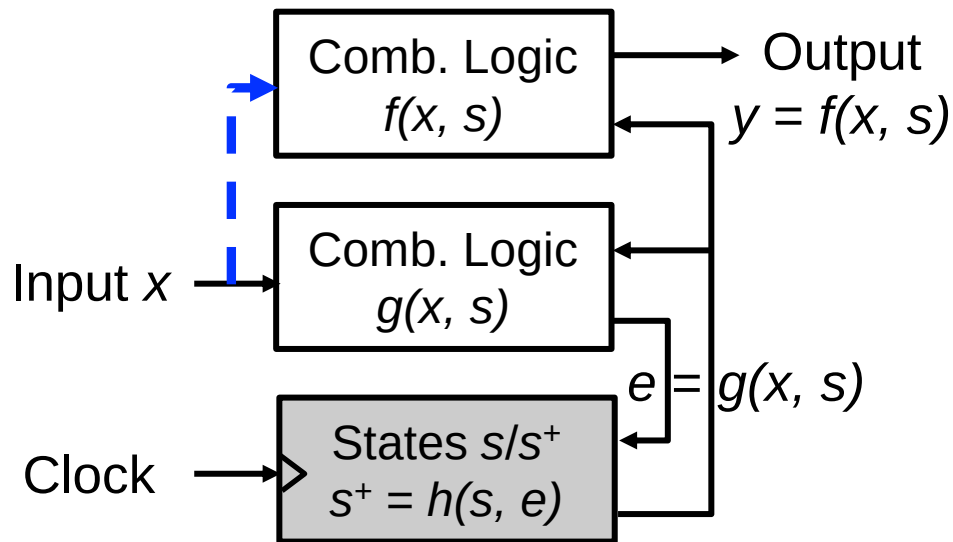
- Ideology
 - $E = g(x, s)$ accomplishes computing in each cycle.



本节课目录

- 概念：时钟，状态和FSM
- 时序逻辑电路：锁存器
- 时序逻辑电路：DFF
- 分析，设计，举例

同步时序电路分析



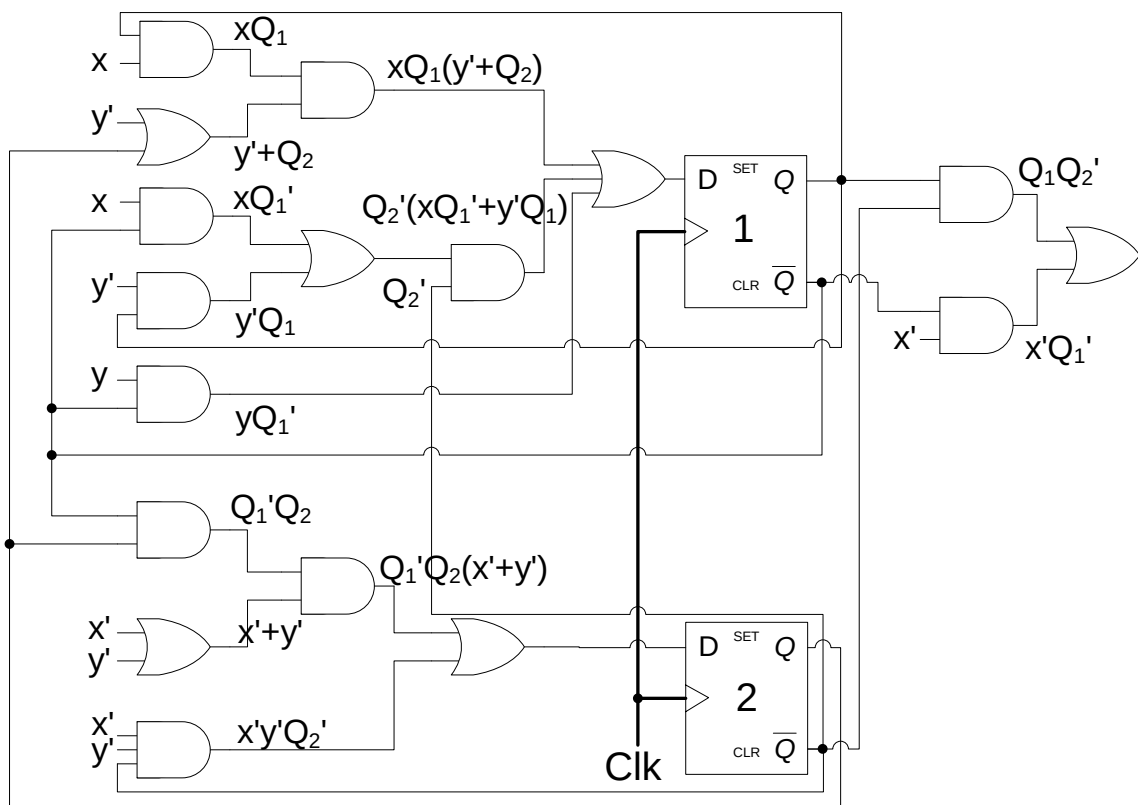
电路分析:

1. 输出方程: $y = f(x, s)$
2. 激励方程
 - 触发器的输入函数: $e = g(x, s)$
3. 状态转移方程 (次态方程)
 - 从 s 到 s^+ : $s^+ = h(s, e)$
 $= h(s, g(x, s))$
 - For DFF: $Q^+ = D = g(x, s)$

行为分析:

4. 状态转移表: 基于 y 和 h
5. 状态图
6. 时序图和电路行为

分析举例



1. 输出方程

$$z = \bar{x}\bar{Q}_1 + Q_1\bar{Q}_2$$

2. 激励方程(触发器输入)

$$D_1 = y\bar{Q}_1 + \bar{Q}_2(x\bar{Q}_1 + \bar{y}Q_1) + xQ_1(\bar{y} + Q_2)$$

$$D_2 = \bar{x}\bar{y}\bar{Q}_2 + \bar{Q}_1Q_2(\bar{x} + \bar{y})$$

3. 状态转移方程

$$Q_1^+ = y\bar{Q}_1 + x\bar{Q}_1\bar{Q}_2 + x\bar{y}Q_1 + xQ_1Q_2 + \bar{y}Q_1\bar{Q}_2$$

$$Q_2^+ = \bar{x}\bar{y}\bar{Q}_2 + \bar{x}\bar{Q}_1Q_2 + \bar{y}\bar{Q}_1Q_2$$

状态转移表

状态转移方程(重复)

$$Q_1^+ = y\bar{Q}_1 + x\bar{Q}_1\bar{Q}_2 + x\bar{y}Q_1 + xQ_1Q_2 + \bar{y}Q_1\bar{Q}_2$$

$$Q_2^+ = \bar{x} \cdot \bar{y}\bar{Q}_2 + \bar{x}\bar{Q}_1Q_2 + \bar{y}\bar{Q}_1Q_2$$

输出方程

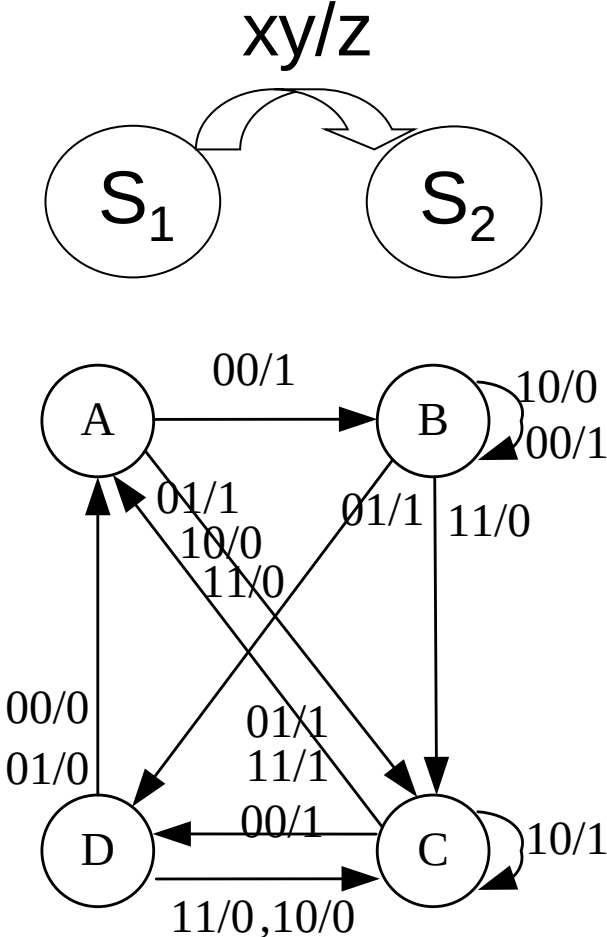
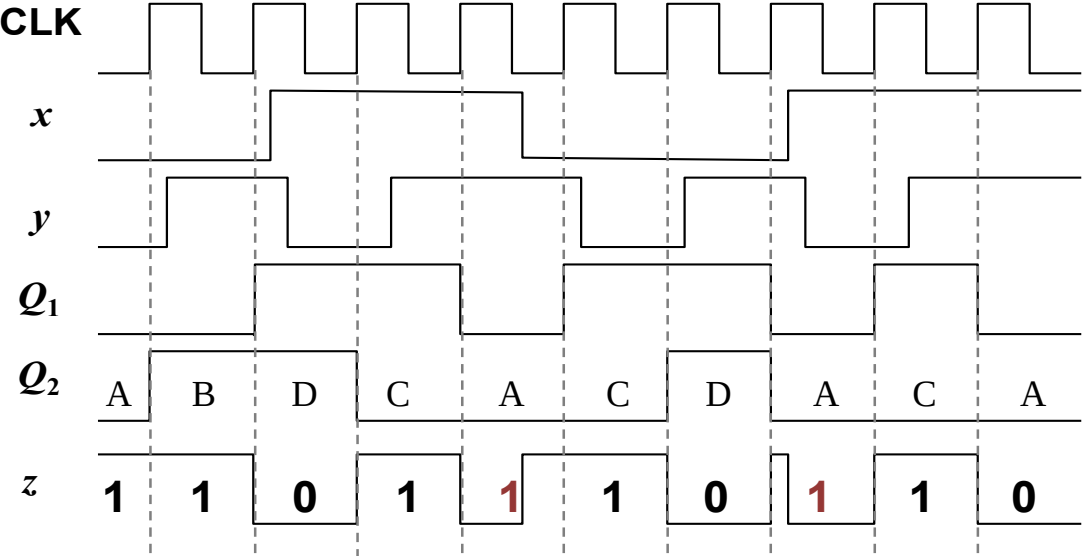
$$z = \bar{x}\bar{Q}_1 + Q_1\bar{Q}_2$$

假设 $00 \rightarrow A, 01 \rightarrow B, 10 \rightarrow C, 11 \rightarrow D$

现态 Q_1Q_2	次态 $Q_1^+Q_2^+$				输出 Z	
	xy=00	xy=01	xy=10	xy=11	x=0	x=1
$00 \rightarrow A$	$01 \rightarrow B$	$10 \rightarrow C$	$10 \rightarrow C$	$10 \rightarrow C$	1	0
$01 \rightarrow B$	$01 \rightarrow B$	$11 \rightarrow D$	$01 \rightarrow B$	$10 \rightarrow C$	1	0
$10 \rightarrow C$	$11 \rightarrow D$	$00 \rightarrow A$	$10 \rightarrow C$	$00 \rightarrow A$	1	1
$11 \rightarrow D$	$00 \rightarrow A$	$00 \rightarrow A$	$10 \rightarrow C$	$10 \rightarrow C$	0	0

状态表和时序图

现态	次态				输出 z	
	00	01	10	11	x=0	x=1
A	B	C	C	C	1	0
B	B	D	B	C	1	0
C	D	A	C	A	1	1
D	A	A	C	C	0	0



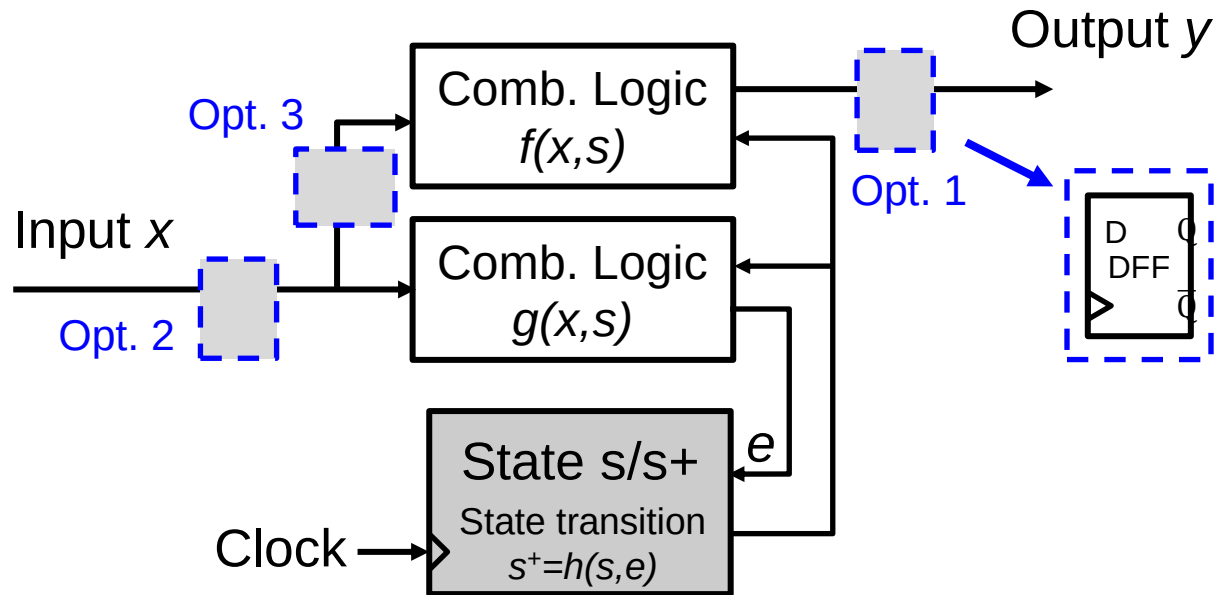
米利机

比较: Mealy vs Moore

- 摩尔机
 - 输出和状态变化同步
 - 通常有更多状态
- 米利机
 - 输出 和 状态变化 异步
 - 输出是输入和状态的组合逻辑
 - 通常有更少状态

比较: Mealy vs Moore

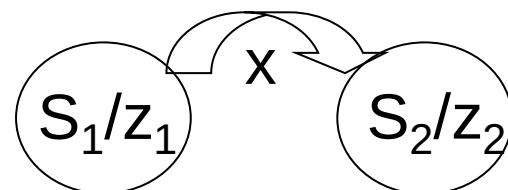
- Mealy机的输入直接驱动输出(未经时钟同步)
 - 如何转化为Moore机?



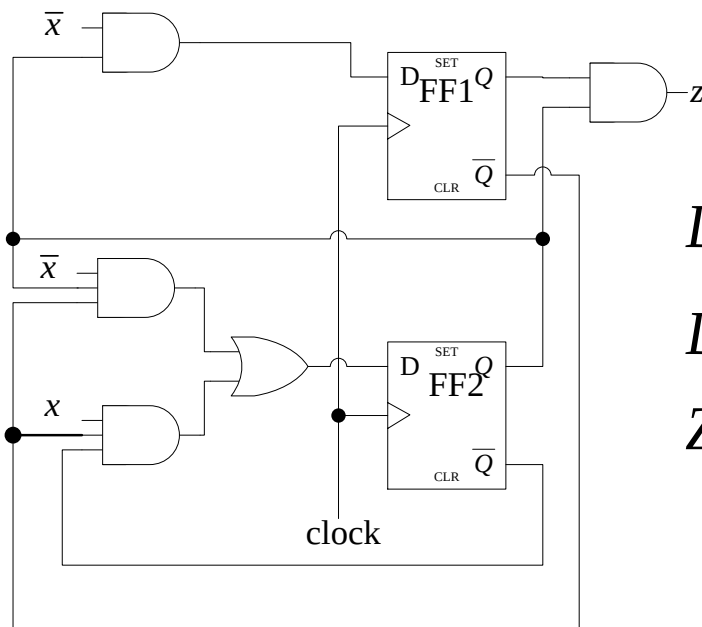
Moore机分析

- 状态转移表，状态图 (注意和Mealy型的不同)

当前状态	次态		输出 Z
	x=0	x=1	



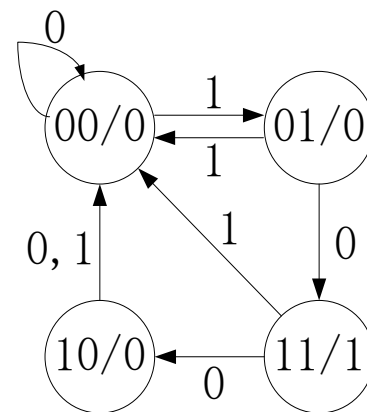
- 课后练习



$$D_1 = \bar{X}Q_2$$

$$D_2 = \bar{X}Q_2\bar{Q}_1 + X\bar{Q}_1\bar{Q}_2$$

$$Z = Q_1Q_2$$



问题

从
功能分析

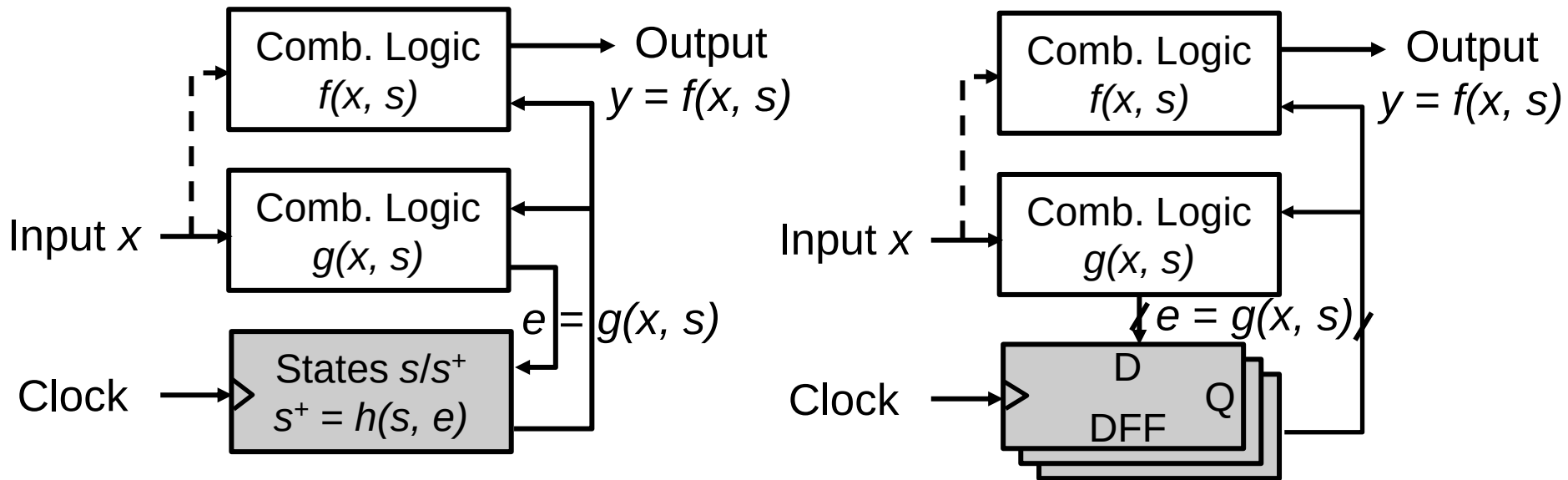
到
性能分析？

复习：建立时间和保持时间

- 建立时间 t_{su}
 - 锁存操作**开始之前**，输入信号应保持不变的最短时间
- 保持时间 t_h
 - 锁存操作**之后**，输入信号还应保持不变的最短时间
- 输出响应时间 t_{co} or t_{cq} (clk-to-output)
 - 从开始**采样到输出**信号的延时

为什么需要时序？

- 更好组织计算!

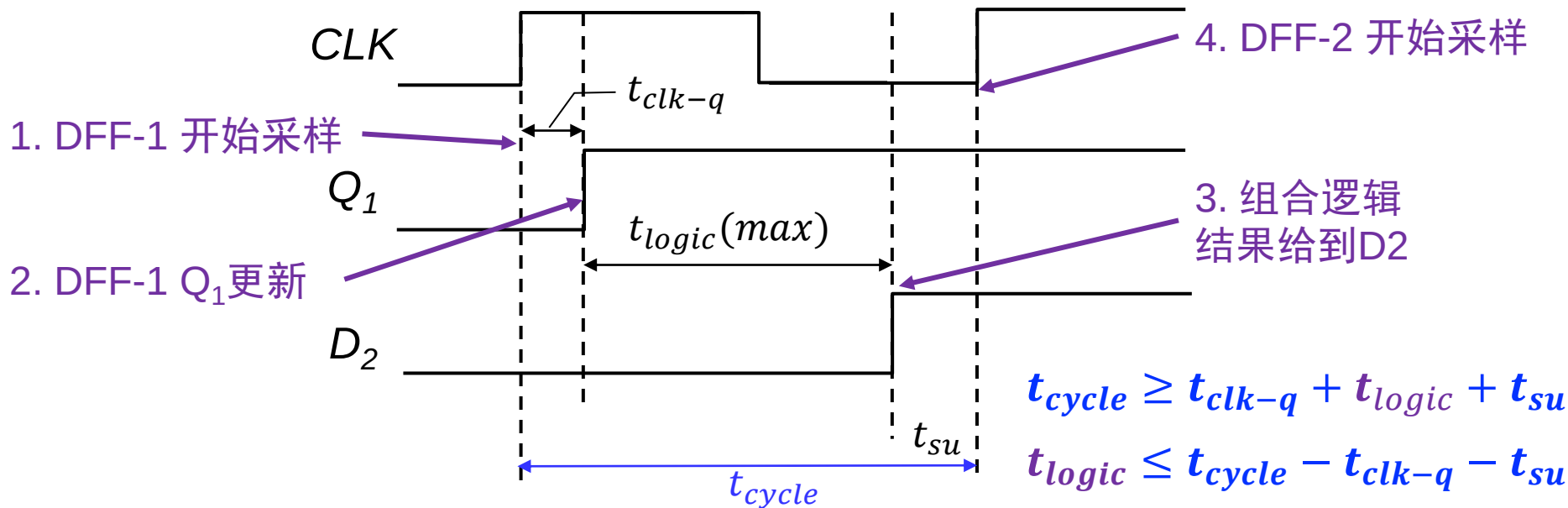
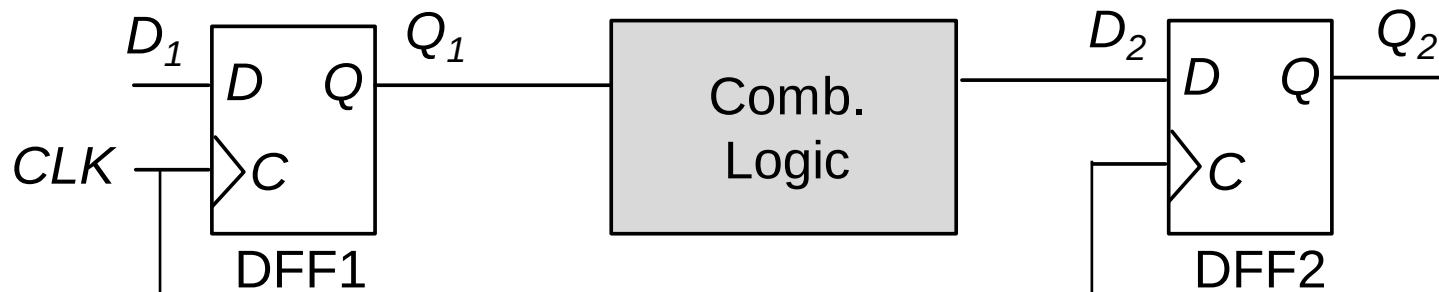


- ✓ D 应该在采样开始之前准备好: $e = f(x, s) \rightarrow D$ 可能因为一个慢的反馈环路花费太多时间!
- ✓ D 应该在采样开始后保持稳定: $e = f(x, s) \rightarrow D$ 可能因为一个快的反馈环路而被破坏!

建立时间约束

- 上升沿触发例子

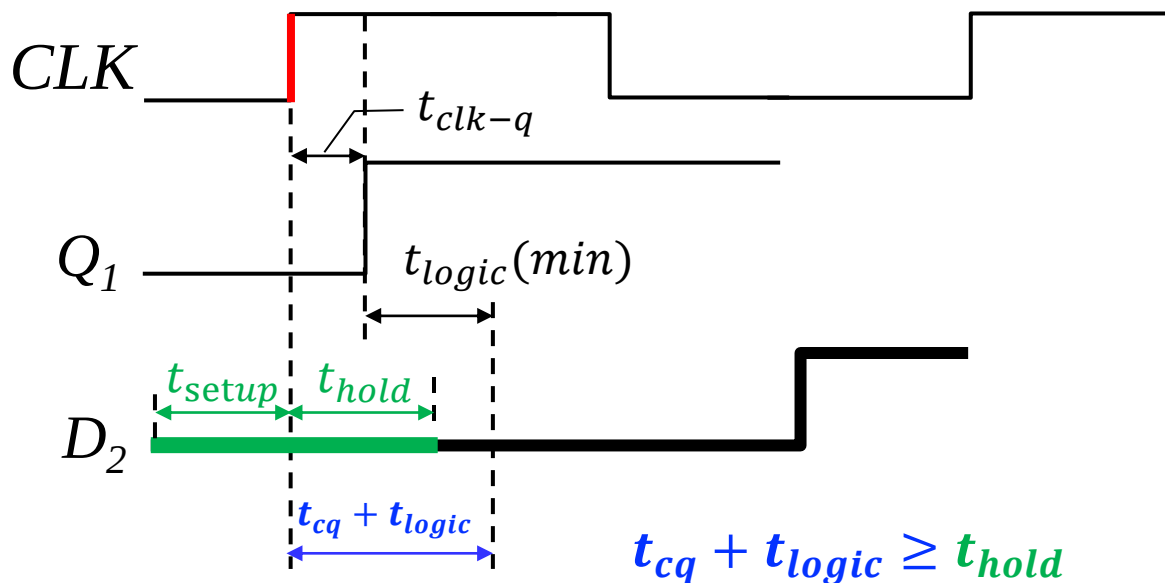
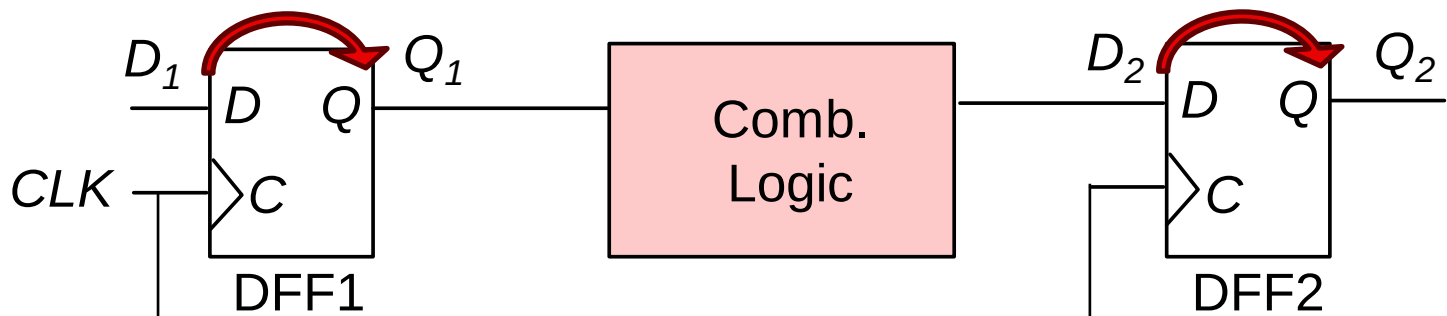
后级DFF的输入信号D必须在触发时钟到来前已经稳定 t_{su} 时间



保持时间约束

- 上升沿触发例子

后级DFF的输入信号D必须在触发时钟到来后继续稳定 t_h 时间



本节课目录

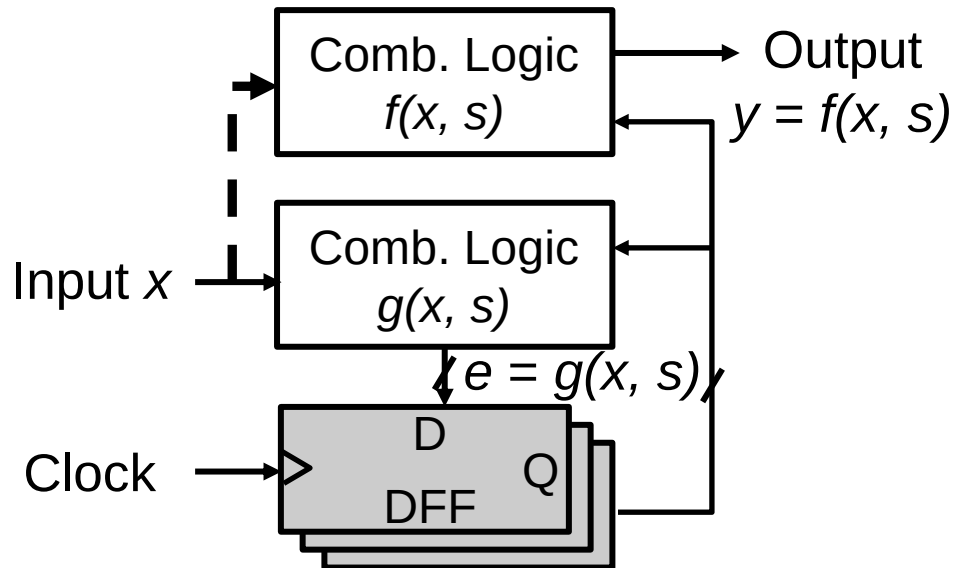
- 概念：时钟，状态和FSM
- 时序逻辑电路：锁存器
- 时序逻辑电路：DFF
- 分析，设计，举例

问题

分析



设计



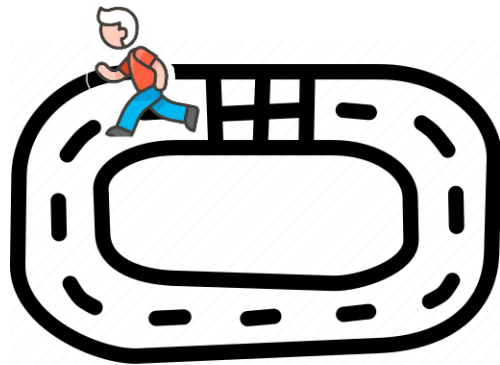
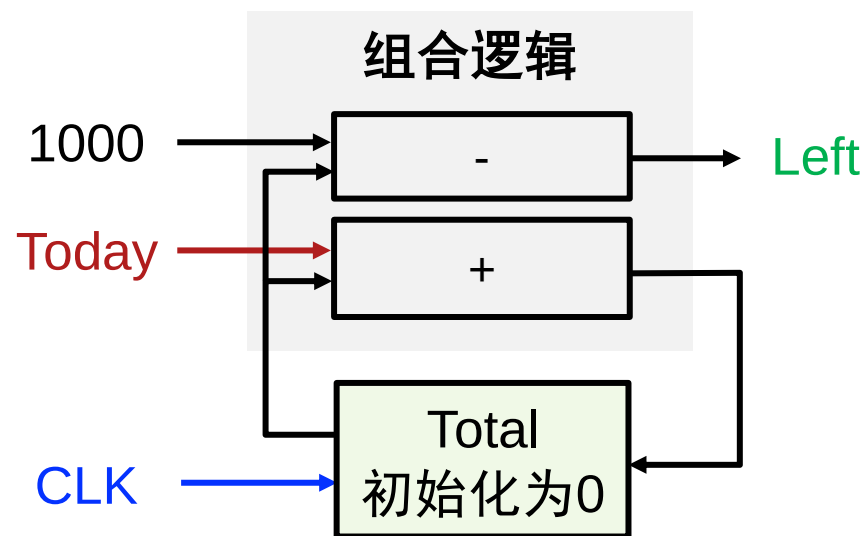
- 状态
 - 什么状态?
 - 状态表示?
 - 状态化简?
- 状态转移
 - 逻辑化简
- 输出函数
 - 优化?

FSM设计方法

- Step 1: 抽象出一个有限状态机
 - 定义 初始状态
 - 得到有限状态机和状态转移关系
- Step 2: 状态化简
- Step 3: 状态分配/表示
- Step 4: 确定激励方程和输出方程
- Step 5: 画出逻辑电路图

例1: 训练累计器

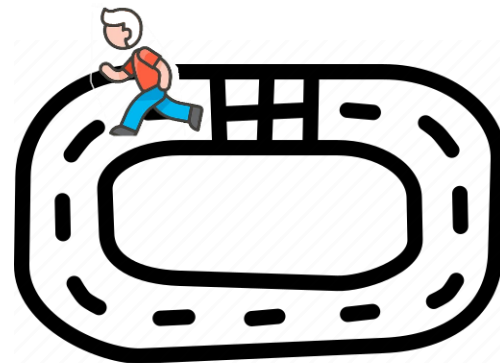
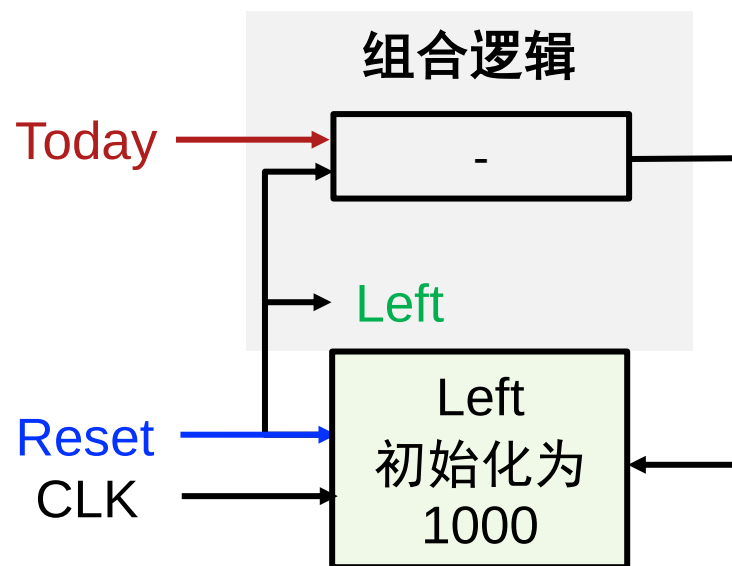
- 输入
 - 跑步距离 **Today**
- 输出
 - 剩余距离 **Left**
 - 输出 = ?
- 状态
 - **Total** 距离
 - 新状态 = ?



逻辑过于复杂?

例1: 训练累计器

- 另一种设计
 - 简化状态和逻辑
 - 状态: **Left**
 - 初始状态: 1000
 - **Left** 也作为输出
 - 状态转移
 - $Left^+ = Left - Today$
 - $Left^+ = \text{Reset?} 1000 : (Left - Today)$
- 初始/最终 状态?
 - ✓ 复位! 检查!



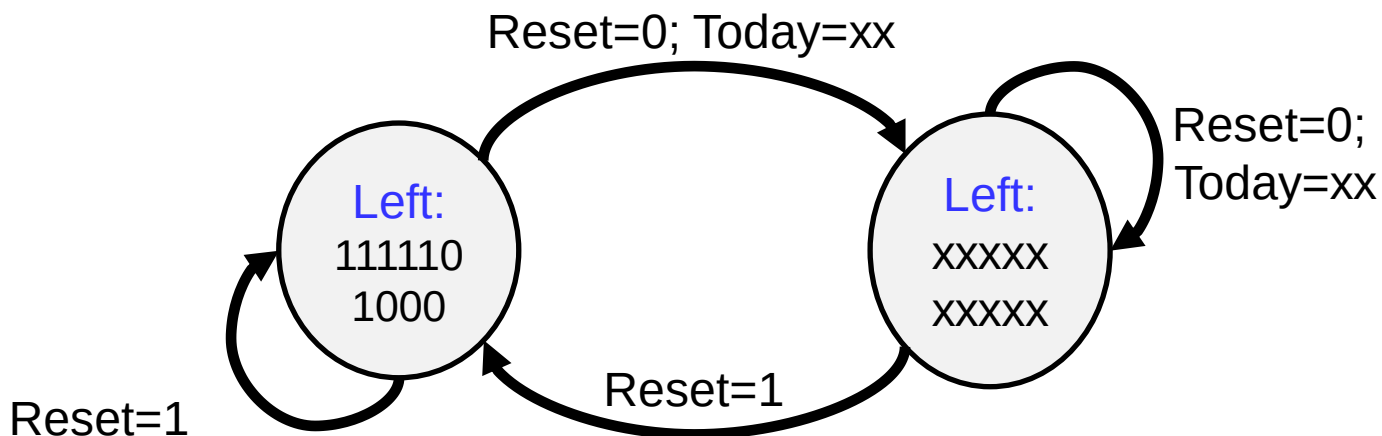
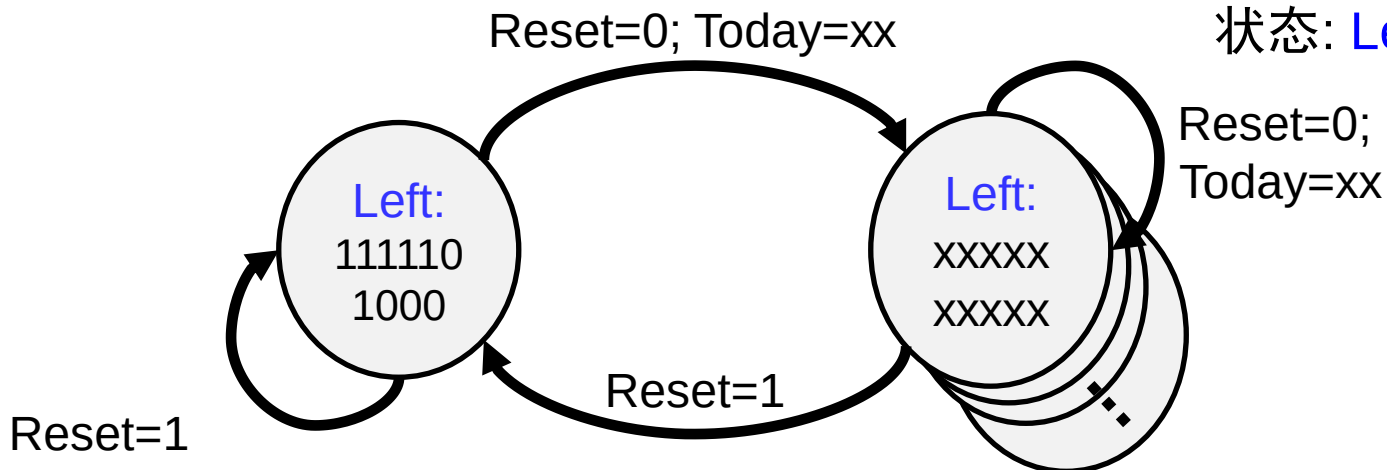
例1: 训练累计器

- Moore机

输入: Today, Reset

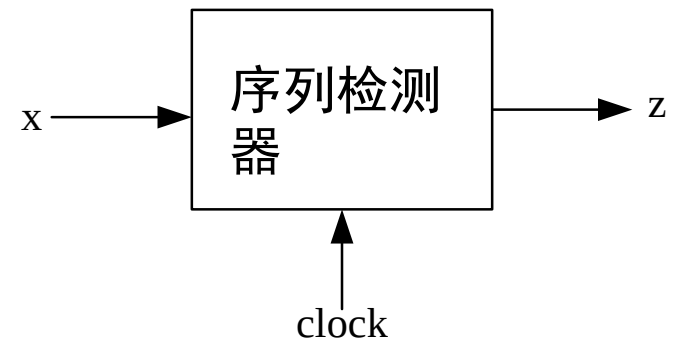
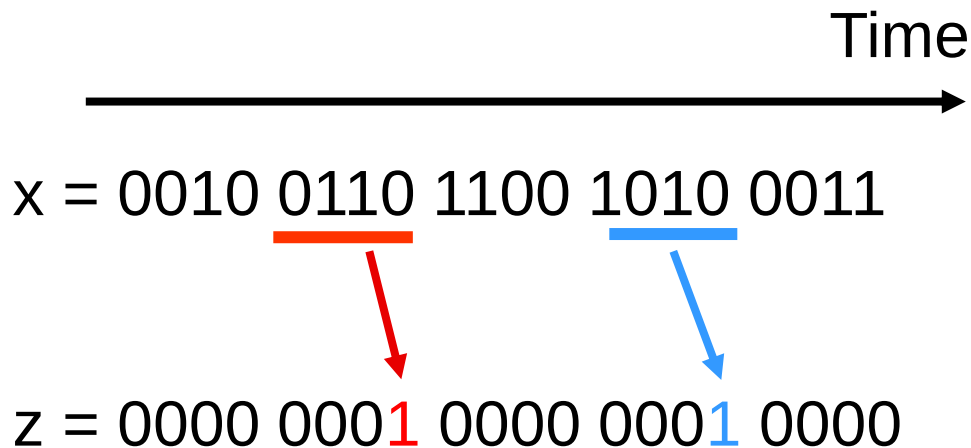
输出: Left

状态: Left



例2: 序列检测器

- 当且仅当接收到0110或者1010时，其输出才为1；其他情况下，输出为0。
- 每接收到4比特输入序列后，状态机回到复位状态



例2: 序列检测器

根据接收到的
序列定义状态

A*: 复位

B: 接收到0

C: 接收到1

D: 接收到00

E: 接收到01

F: 接收到10

G: 接收到11

H: 接收到000

I: 接收到001

J: 接收到010

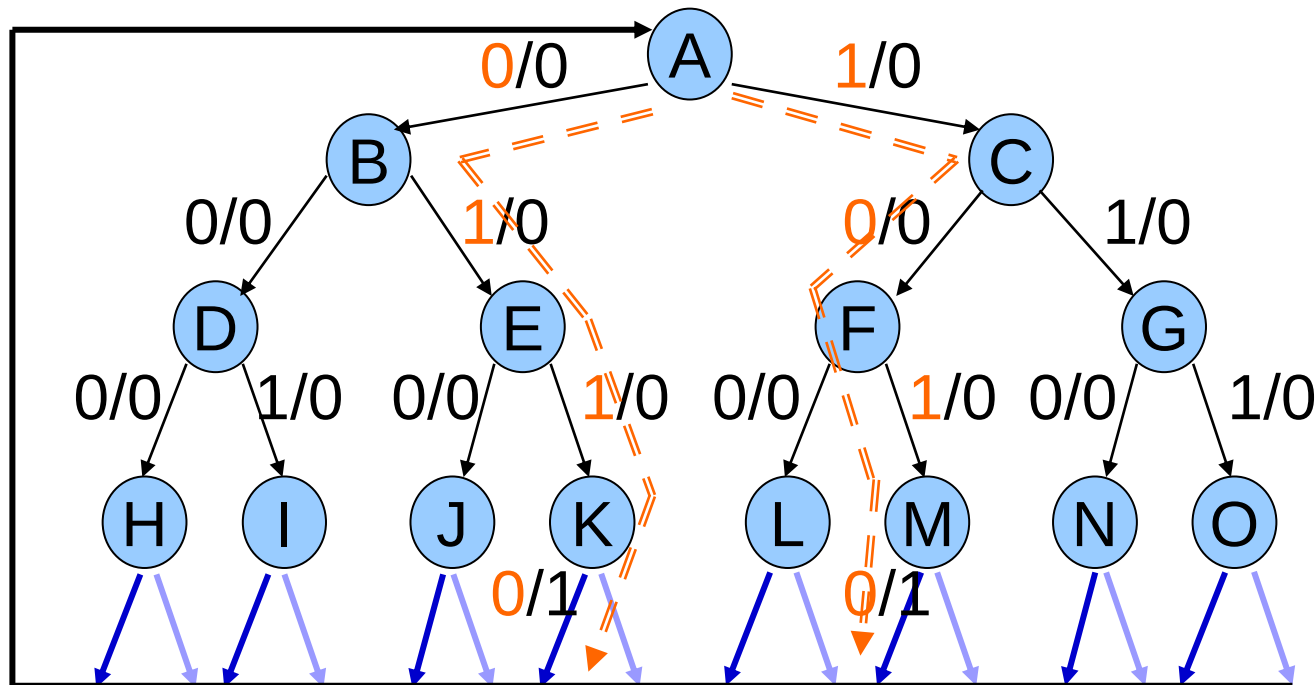
K: 接收到011

L: 接收到100

M: 接收到101

N: 接收到110

O: 接收到111

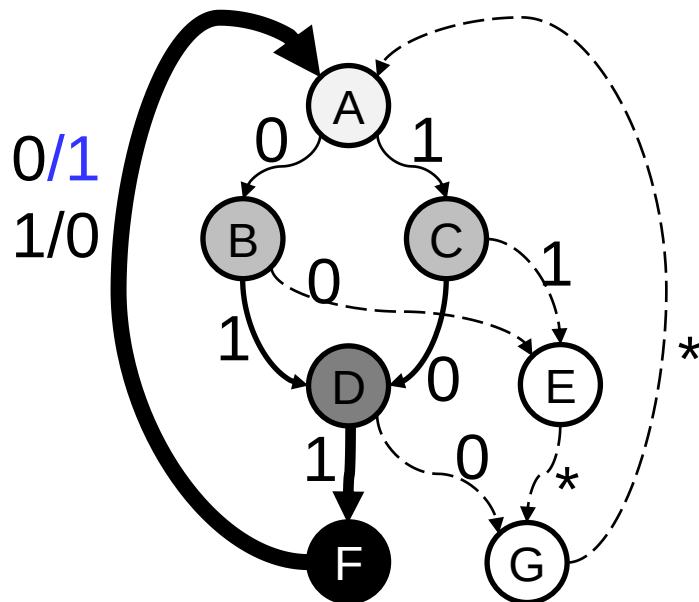


一共15
个状态!

例2: 序列检测器

目标匹配: 0110 or 1010

现在, 状态通过不匹配的模式
和 可能匹配的模式 定义



输出没有全部显示

A: 复位

B: 0xxx (第一个比特为0)

C: 1xxx (第一个比特为1)

D: 01xx or 10xx (前2位匹配)

E: 00xx or 11xx (前2位不匹配)

F: 101x 011x (前三位匹配)

G: 前3位不匹配

关键:
合并冗余!
找到等价状态!

FSM设计方法

思考：FSM抽象可有一个统一的方法？

设计方法

- Step1: 确定输入输出并抽象出有限状态机
- Step2: 状态化简!
- Step3: 状态分配
- Step4: 确定激励方程和输出方程
- Step5: 画出逻辑电路图

Step2: 状态化简

问题：数学上是否有一般性的方法？

- 状态化简方法
 - 行匹配
 - 蕴含表
 - 两个一起使用

等价状态

- 两个状态P, Q等价, 记作 $P \equiv Q$
 - 当且仅当对于各输入情况, 输出相同, 次态是等价
- 两个状态不等价是**可区分的**
 - 至少有一种输入序列, 会使得它们产生不同的输出



行匹配技术

- 找出具有相同次态和输出的行，合并为新的一行
- 重复行匹配，直到没有行可以合并

输入序列	现态	次态		输出	
		x=0	x=1	x=0	x=1
Reset	A	B	C	0	0
0	B	D	E	0	0
1	C	F	G	0	0
00	D	H	I	0	0
01	E	J	K	0	0
10	F	L	M	0	0
11	G	N	O	0	0
000	H	A	A	0	0
001	I	A	A	0	0
010	J	A	A	0	0
011	K	A	A	1	0
100	L	A	A	0	0
101	M	A	A	1	0
110	N	A	A	0	0
111	O	A	A	0	0

等价状态!

例2: 行匹配迭代

输入序列	现态	次态		输出	
		x=0	x=1	x=0	x=1
reset	A	B	C	0	0
0	B	D	E	0	0
1	C	F	G	0	0
00	D	H	I	0	0
01	E	J	K	0	0
10	F	L	M	0	0
11	G	N	O	0	0
000	H	A	A	0	0
001	I	A	A	0	0
010	J	A	A	0	0
011 or 101	K'	A	A	1	0
100	L	A	A	0	0
110	N	A	A	0	0
111	O	A	A	0	0

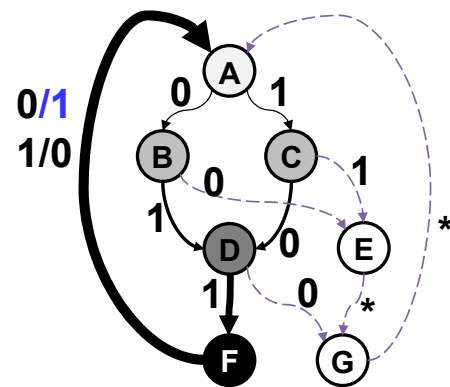
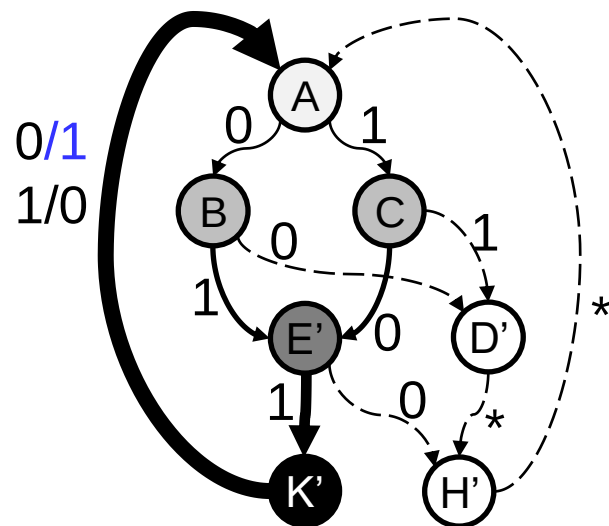
相同次态、
相同输出

1. 合并等价状态
2. 继续查找

例2: 行匹配迭代

输入序列	现态	次态		输出	
		x=0	x=1	x=0	x=1
Reset	A	B	C	0	0
0	B	D	E	0	0
1	C	F	G	0	0
00	D	H'	H'	0	0
01	E	H'	K'	0	0
10	F	H'	K'	0	0
11	G	H'	H'	0	0
Not (011 or 101)	H'	A	A	0	0
011 or 101	K'	A	A	1	0

输入序列	现态	次态		输出	
		x=0	x=1	x=0	x=1
Reset	A	B	C	0	0
0	B	D'	E'	0	0
1	C	E'	D'	0	0
00 or 11	D'	H'	H'	0	0
01 or 10	E'	H'	K'	0	0
Not (011 or 101)	H'	A	A	0	0
011 or 101	K'	A	A	1	0



条条大路通罗马

行匹配的限制

- 有时难以找到等价状态
- 举例: A与C两个状态是否可以合并?

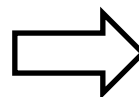
现态	次态		输出
	x=0	x=1	
A	A	B	0
B	B	C	1
C	C	B	0

$A \stackrel{?}{\equiv} C$

蕴含表化简技术

- 根据状态表，构造蕴含表
 - 每个交叉单元表示 状态 j 和 状态 k
 - CAD工具使用

现态	次态		输出	
	x=0	x=1	x=0	x=1
A	D	C	0	1
B	A	C	0	0
C	A	C	0	0
D	D	B	0	0



B			
C			
D			
	A	B	C

蕴含表化简技术

- 根据状态表，构造蕴含表
 - 每个交叉单元 表示 状态 j 和 状态 k
 - 情景- 1: 不同输出
 - 在交叉单元内标记 “x”

现态	次态		输出	
	x=0	x=1	x=0	x=1
A	D	C	0	1
B	A	C	0	0
C	A	C	0	0
D	D	B	0	0

A和其他3个状态
不等价

B	x		
C	x		
D	x		
	A	B	C

蕴含表化简技术

- 根据状态表，构造蕴含表
 - 情景- 1: 不同输出，标记 “x”
 - 情景- 2: 等价状态 (输出/次态)
 - 标记 “√”

现态	次态		输出	
	x=0	x=1	x=0	x=1
A	D	C	0	1
B	A	C	0	0
C	A	C	0	0
D	D	B	0	0

$B \equiv C$

B	x		
C	x	√	
D	x		
	A	B	C

蕴含表化简技术

- 从状态转移表到蕴含表
 - 情景- 1: 不同输出, 标记 “x”
 - 情景- 2: 等价状态 标记 “√”
 - 其余填入等价条件

现态	次态		输出	
	x=0	x=1	x=0	x=1
A	D	C	0	1
B	A	C	0	0
C	A	C	0	0
D	D	B	0	0

只有满足如下条件
两个状态才等价

B	x		
C	x	√	
D	x	A=D B=C	A=D B=C
	A	B	C

蕴含表化简技术

- 从状态转移表到蕴含表
 - 情景- 1: 不同输出, 标记 “x”
 - 情景- 2: 等价状态 标记 “√”
 - 其余填入等价条件
 - 检查条件: 不满足时标记“x”

现态	次态		输出	
	x=0	x=1	x=0	x=1
A	D	C	0	1
B	A	C	0	0
C	A	C	0	0
D	D	B	0	0

$A \neq D \rightarrow B \neq D, C \neq D$

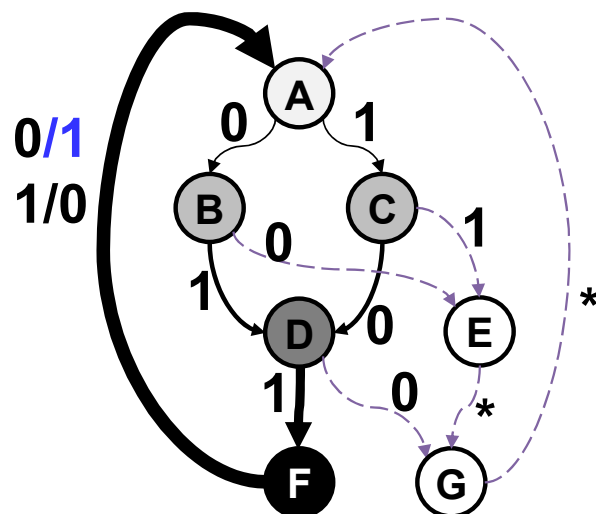
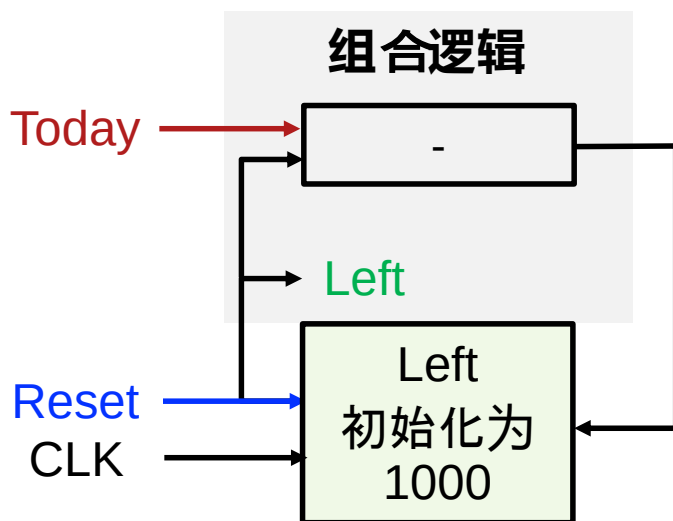
B	x		
C	x	√	
D	x	x	x
	A	B	C

设计方法

- Step1: 确定输入输出并抽象出有限状态机
- Step2: 状态化简
- **Step3: 状态分配**
- Step4: 确定激励方程和输出方程
- Step5: 画出逻辑电路图

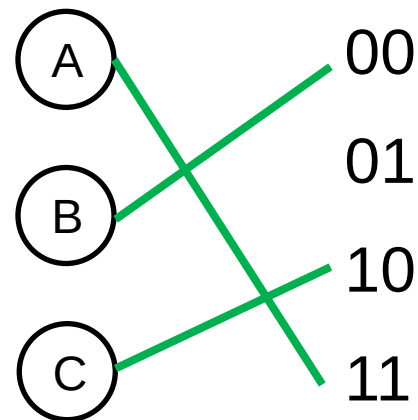
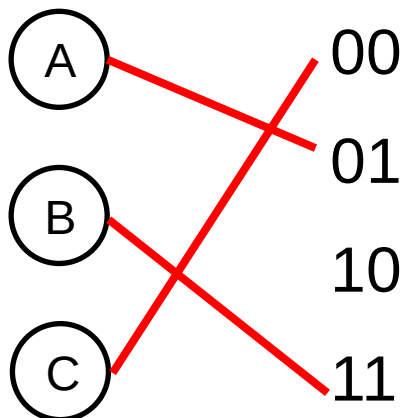
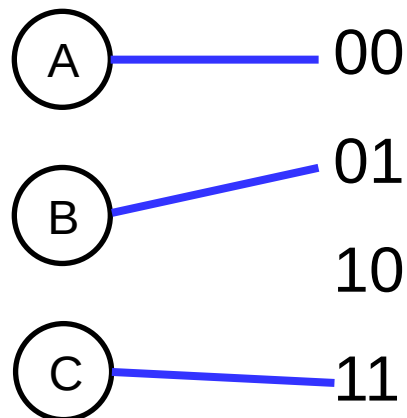
Step3: 状态分配

- 状态编码/表示
 - 如何编码状态？

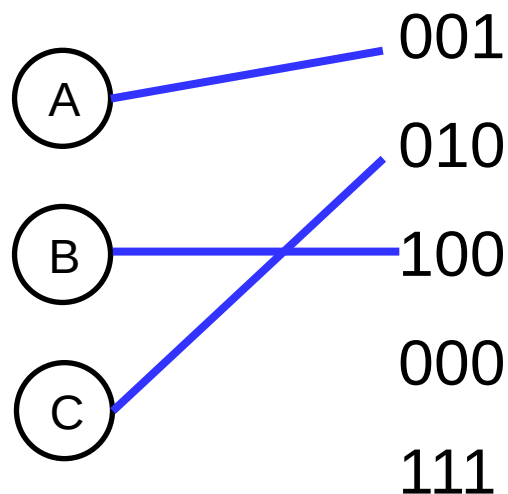
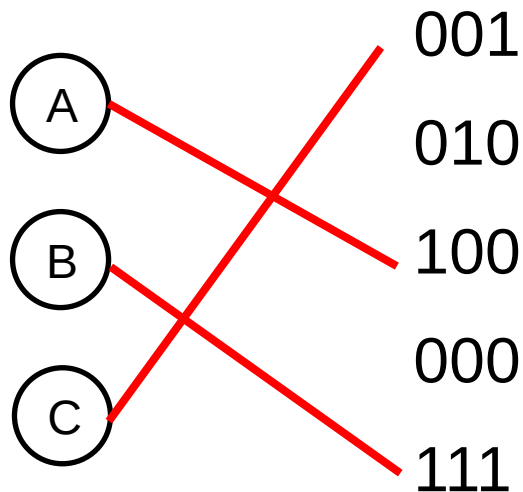


Step3: 状态分配（三个状态为例）

- 使用2比特编码3个状态



- 使用3比特进行编码（下图只列出了部分编码）



Step3: 状态分配

- 独热编码(One-hot Encoding)
 - 引入更多触发器，以便简化次态和输出逻辑
 - n 个状态用 n 个触发器进行编码
 - 每个状态都用一个 n 位二进制码表示，其中只有1位被置1

(A)	001
(B)	010
(C)	100
	000
	111

Step3: 状态分配

- 紧凑型状态编码/表示

- 如何用最少的二进制0/1编码状态（节省DFF数量）？
- P 个状态需要K比特二进制数表示
 - 状态分配方法不唯一

$$2^{K-1} < P \leq 2^K$$

状态编码方法

- 独热编码
- 随机编码
- 顺序编码
 - 二进制加法计数顺序
 - 格雷顺序
- 面向输出的编码
- 启发式方法
 - 最少位变化
 - 基于次态和输入/输出的准则

顺序编码

现态	次态		输出 z	
	x=0	x=1	x=0	x=1
A*	A	B	0	0
B	B	C	0	0
C	D	E	0	0
D	F	G	1	0
E	C	B	0	1
F	D	H	1	0
G	B	C	0	1
H	F	G	0	0

现态 $Q_1Q_2Q_3$	次态 $Q_1^+Q_2^+Q_3^+$		输出 z	
	x=0	x=1	x=0	x=1
000	000	001	0	0
001	001	010	0	0
010	011	100	0	0
011	101	110	1	0
100	010	001	0	1
101	011	111	1	0
110	001	010	0	1
111	101	110	0	0

基于次态和输入/输出的准则

- 根据优先级定义进行编码
 - 最高优先级 → 中等优先级 → 最低优先级
- 为了方便化简状态转移和输出函数！
 - 卡诺图中，相同次态/输出的状态编码相近（k-map相邻）

		Q_2Q_3			
		00	01	11	10
Q_1	0	A	B	C	H
	1	E	G	F	D

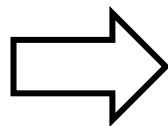
→ A: 000, B:001, C: 011, D:110
E: 100, F:111, G: 101, H:010

基于次态和输入/输出的准则

- 最高优先级:为了简化状态表
 - 输入相同的情况下, 次态相同的状态应该具有相邻的状态编码 (需要考虑不同输入情况)

规则I: (B,G)*2, (C,F), (D,H)*2, (A,E)

现态	次态		输出	
	x=0	x=1	x=0	x=1
A*	A	B	0	0
B	B	C	0	0
C	D	E	0	0
D	F	G	1	0
E	C	B	0	1
F	D	H	1	0
G	B	C	0	1
H	F	G	0	0



现态 $Q_1Q_2Q_3$	次态 $Q_1^+Q_2^+Q_3^+$		输出 z	
	x=0	x=1	x=0	x=1
000	000	001	0	0
001	001	011	0	0
011	110	100	0	0
110	111	101	1	0
100	011	001	0	1
111	110	010	1	0
101	001	011	0	1
010	111	101	0	0

基于次态和输入/输出的准则

- 中等优先级
 - 同一状态的次态应该具有相邻的状态编码

现态	次态		输出	
	x=0	x=1	x=0	x=1
A*	A	B	0	0
B	B	C	0	0
C	D	E	0	0
D	F	G	1	0
E	C	B	0	1
F	D	H	1	0
G	B	C	0	1
H	F	G	0	0

规则2:

(A,B): A&B 一起, 是同一状态 (A) 的次态

(B,C)*3, (D,E), (F,G)*2, (D,H)

基于次态和输入/输出的准则

- 最低优先级
 - 具有相同输出的状态应具有相邻的状态编码.

现态	次态		输出	
	x=0	x=1	x=0	x=1
A*	A	B	0	0
B	B	C	0	0
C	D	E	0	0
D	F	G	1	0
E	C	B	0	1
F	D	H	1	0
G	B	C	0	1
H	F	G	0	0

规则3:

(D,F): 状态D和F有相同输出

(E,G): 状态E和G有相同输出

基于次态和输入/输出的准则

- 最高优先级
 - 输入相同的情况下，次态相同的状态应该具有相邻的状态编码
- 中等优先级
 - 同一状态的次态应该具有相邻的状态编码
- 最低优先级
 - 具有相同输出的状态应具有相邻的状态编码
- 初始状态分配

按照准则得到的结果

按照优先级的顺序
依次满足

现态	次态		输出	
	x=0	x=1	x=0	x=1
A*	A	B	0	0
B	B	C	0	0
C	D	E	0	0
D	F	G	1	0
E	C	B	0	1
F	D	H	1	0
G	B	C	0	1
H	F	G	0	0

Rule I:

$(B,G)*2, (C,F), (D,H)*2, (A,E)$

Rule II:

$(A,B), (B,C)*3, (D,E), (F,G)*2, (D,H)$

Rule III: $(D,F), (E,G)$

		Q ₂ Q ₃			
		00	01	11	10
Q ₁	0	A	B	C	H
	1	E	G	F	D

按照准则得到的结果

现态	次态		输出	
	x=0	x=1	x=0	x=1
A 000	A	B	0	0
B 001	B	C	0	0
C 011	D	E	0	0
D 110	F	G	1	0
E 100	C	B	0	1
F 111	D	H	1	0
G 101	B	C	0	1
H 010	F	G	0	0

现态 $Q_1Q_2Q_3$	次态 $Q_1^+Q_2^+Q_3^+$		输出 z	
	x=0	x=1	x=0	x=1
000	000	001	0	0
001	001	011	0	0
011	110	100	0	0
110	111	101	1	0
100	011	001	0	1
111	110	010	1	0
101	001	011	0	1
010	111	101	0	0

按照准则得到的结果

$$Q_1^+ = Q_2 \bar{Q}_3 + \bar{Q}_1 Q_2 + Q_2 \bar{x}$$

$$Q_2^+ = Q_2 \bar{x} + Q_1 \bar{Q}_3 \bar{x} + \bar{Q}_2 Q_3 x + Q_1 Q_2 Q_3$$

$$Q_3^+ = \bar{Q}_3 x + \bar{Q}_2 Q_3 + Q_1 \bar{Q}_2 + Q_2 \bar{Q}_3$$

$$z = Q_1 Q_2 \bar{x} + Q_1 \bar{Q}_2 x$$

现态 $Q_1 Q_2 Q_3$	次态		输出	
	$Q_1^+ Q_2^+ Q_3^+$ x=0	$Q_1^+ Q_2^+ Q_3^+$ x=1	z x=0	z x=1
000	000	001	0	0
001	001	011	0	0
011	110	100	0	0
110	111	101	1	0
100	011	001	0	1
111	110	010	1	0
101	001	011	0	1
010	111	101	0	0

课后练习：如果以顺序编码的方式分配状态？

结果比较

顺序编码

现态 $Q_1Q_2Q_3$	次态 $Q_1^+Q_2^+Q_3^+$		输出 z	
	$x=0$	$x=1$	$x=0$	$x=1$
000	000	001	0	0
001	001	010	0	0
010	011	100	0	0
011	101	110	1	0
100	010	001	0	1
101	011	111	1	0
110	001	010	0	1
111	101	110	0	0

基于准则的编码

现态 $Q_1Q_2Q_3$	次态 $Q_1^+Q_2^+Q_3^+$		输出 z	
	$x=0$	$x=1$	$x=0$	$x=1$
000	000	001	0	0
001	001	011	0	0
011	110	100	0	0
110	111	101	1	0
100	011	001	0	1
111	110	010	1	0
101	001	011	0	1
010	111	101	0	0

$$z = Q_1Q_3'x + Q_1'Q_2Q_3x' + Q_1Q_2'Q_3x'$$

$$z = Q_1Q_2\bar{x} + Q_1\bar{Q}_2x$$

输出方程大大简化!

结果比较

顺序编码 $Q_2^+ = Q_3x + Q_1Q_2x + Q_1Q_2'x' + Q_1'Q_2Q_3'x'$

$$z = Q_1Q_3'x + Q_1'Q_2Q_3x' + Q_1Q_2'Q_3x'$$

只使用规则I $Q_2^+ = Q_1Q_2' + Q_1Q_3x'$ 次态方程大大简化!

只使用规则II $Q_2^+ = Q_3x' + Q_1Q_3 + Q_1'Q_2'Q_3$

只使用规则III $Q_2^+ = Q_1Q_2' + Q_1'Q_2 + Q_1'Q_3x$

只使用规则I

		Q ₂ Q ₃			
		00	01	11	10
Q ₁	0	B	G	D	H
	1	C	F	A	E

只使用规则II

		Q ₂ Q ₃			
		00	01	11	10
Q ₁	0	B	C	F	G
	1	A	H	D	E

只使用规则III

		Q ₂ Q ₃			
		00	01	11	10
Q ₁	0	D	F	A	B
	1	E	G	C	H

设计方法

- Step1: 确定输入输出并抽象出有限状态机
- Step2: 状态化简
- Step3: 状态分配
- Step4: 确定激励方程和输出方程
- Step5: 画出逻辑电路图

Step4: 确定激励方程和输出方程

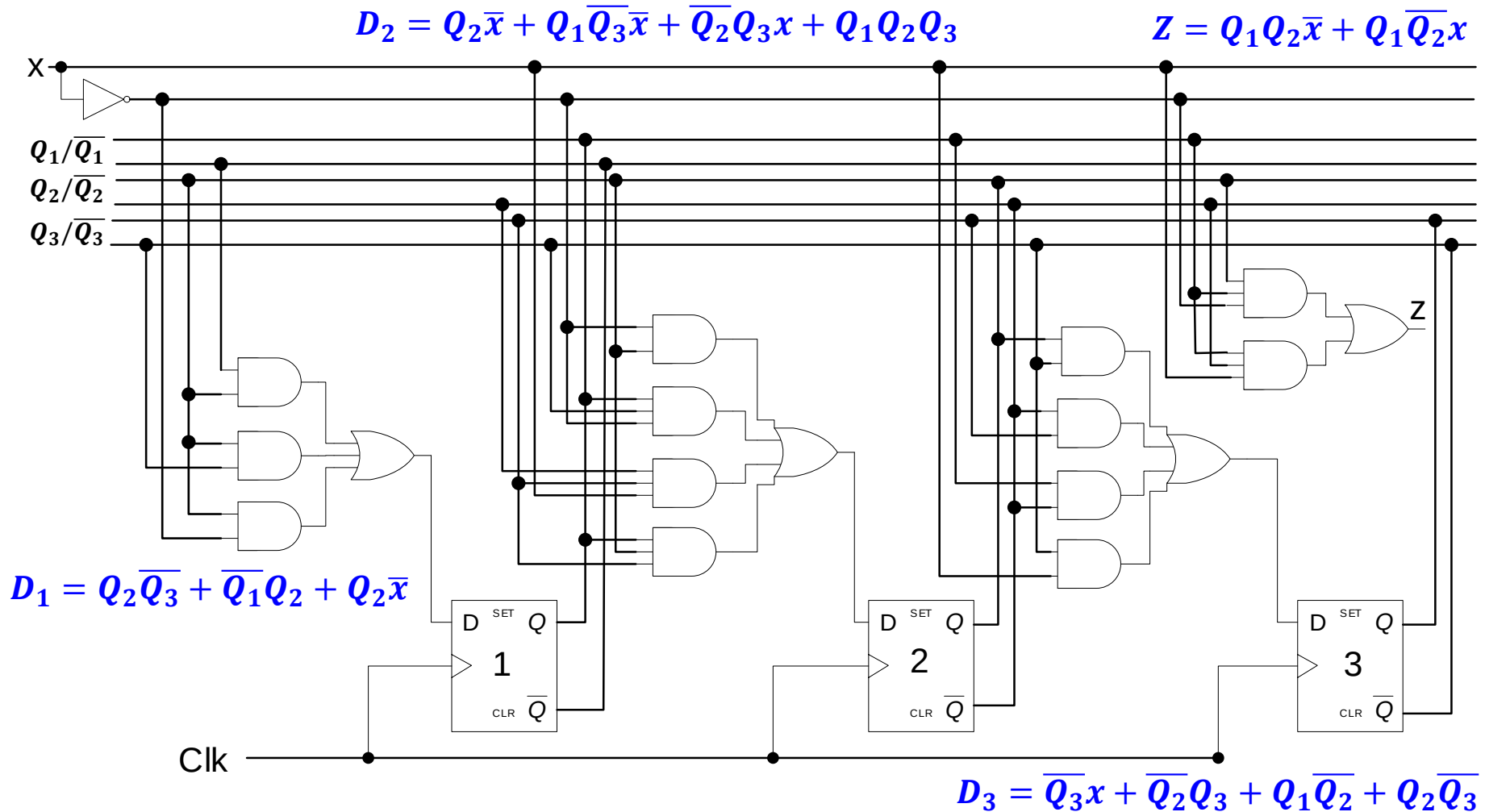
采用基于准则的分配结果

$$\left\{ \begin{array}{l} Q_1^+ = Q_2 \bar{Q}_3 + \bar{Q}_1 Q_2 + Q_2 \bar{x} \\ Q_2^+ = Q_2 \bar{x} + Q_1 \bar{Q}_3 \bar{x} + \bar{Q}_2 Q_3 x + Q_1 Q_2 Q_3 \\ Q_3^+ = \bar{Q}_3 x + \bar{Q}_2 Q_3 + Q_1 \bar{Q}_2 + Q_2 \bar{Q}_3 \\ z = Q_1 Q_2 \bar{x} + Q_1 \bar{Q}_2 x \end{array} \right.$$

使用D触发器实现:

$$Q^+ = D \left\{ \begin{array}{l} D_1 = Q_2 \bar{Q}_3 + \bar{Q}_1 Q_2 + Q_2 \bar{x} \\ D_2 = Q_2 \bar{x} + Q_1 \bar{Q}_3 \bar{x} + \bar{Q}_2 Q_3 x + Q_1 Q_2 Q_3 \\ D_3 = \bar{Q}_3 x + \bar{Q}_2 Q_3 + Q_1 \bar{Q}_2 + Q_2 \bar{Q}_3 \\ z = Q_1 Q_2 \bar{x} + Q_1 \bar{Q}_2 x \end{array} \right.$$

Step5: 画出逻辑电路图



本节课目录

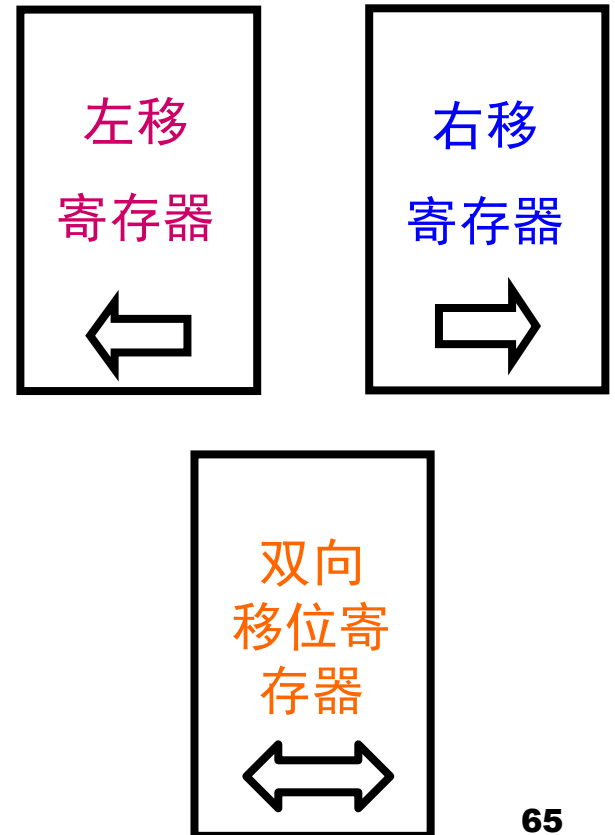
- 概念：时钟，状态和FSM
- 时序逻辑电路：锁存器
- 时序逻辑电路：DFF
- 分析，设计，**举例**

典型的时序逻辑电路

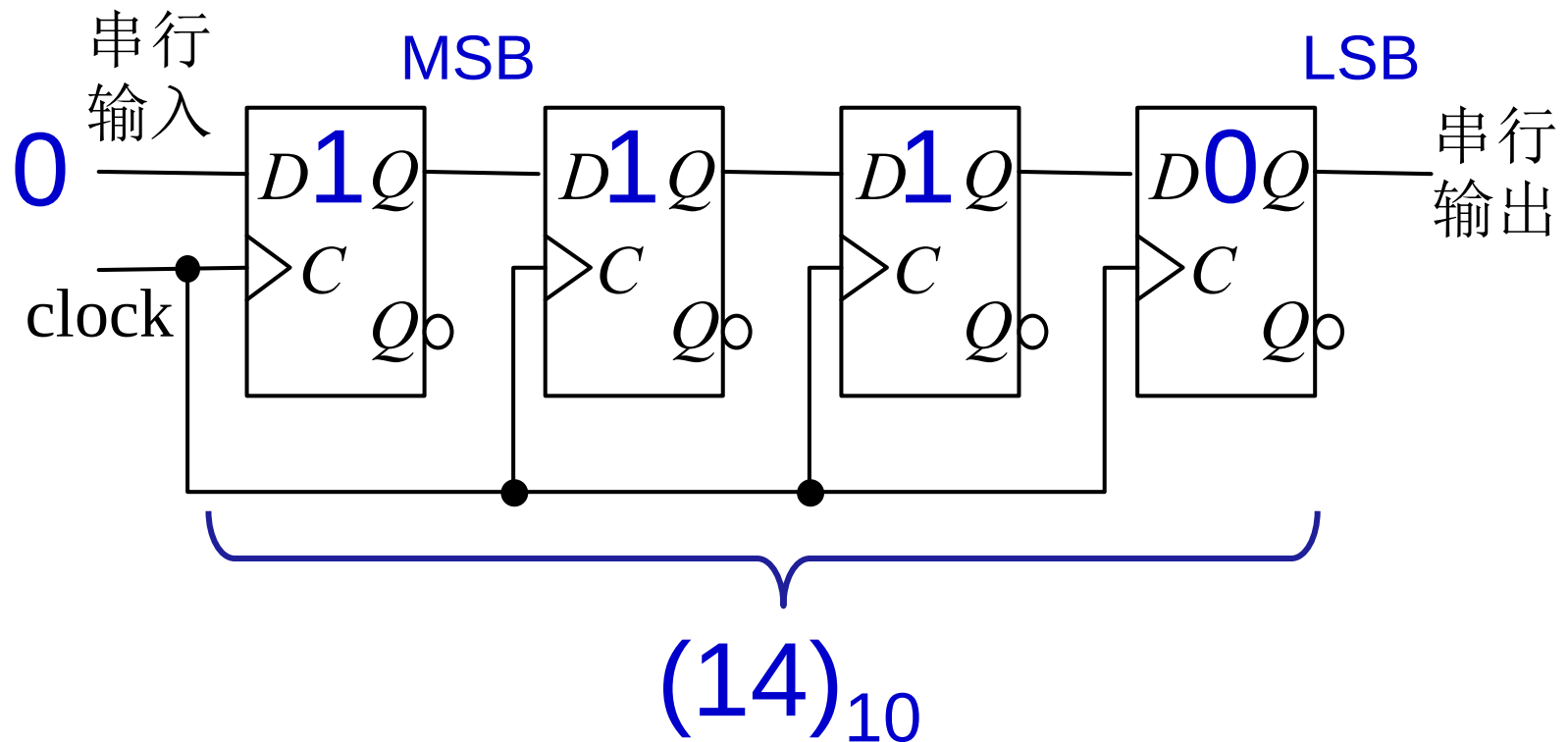
- 寄存器 Register
 - 存储元件
 - 仅存储：存储寄存器
 - 由n个触发器构成的寄存器可存放n位二进制数
 - 移位+存储：移位寄存器
- 计数器 Counter

移位寄存器

- 寄存二进制信息，并进行移位的逻辑元件
- 分类
 - 移位方向
 - 左移、右移、双向移位寄存器
 - 数据输入/输出方式
 - 并行输入，并行输出
 - 串行输入，并行输出
 - 并行输入，串行输出
 - 串行输入，串行输出

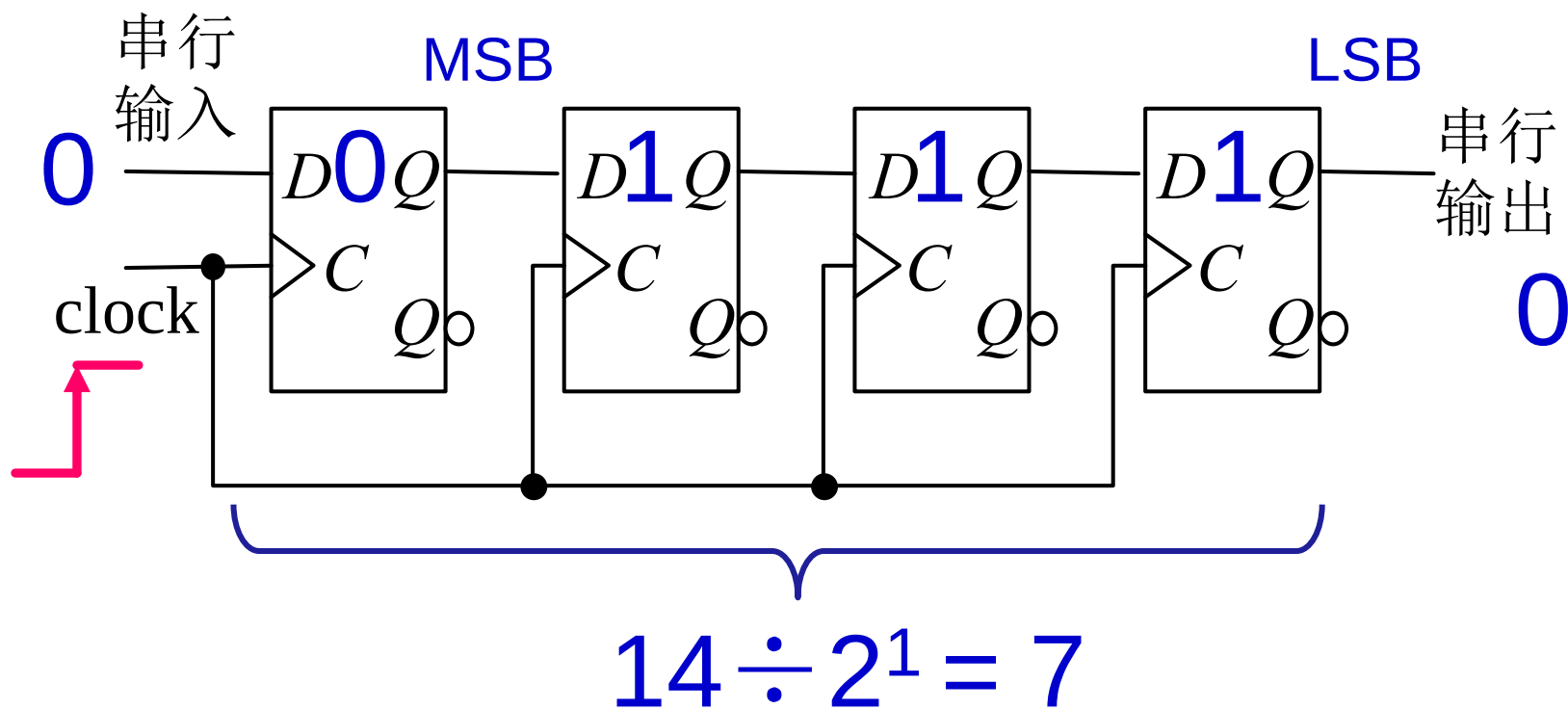


右移移位寄存器



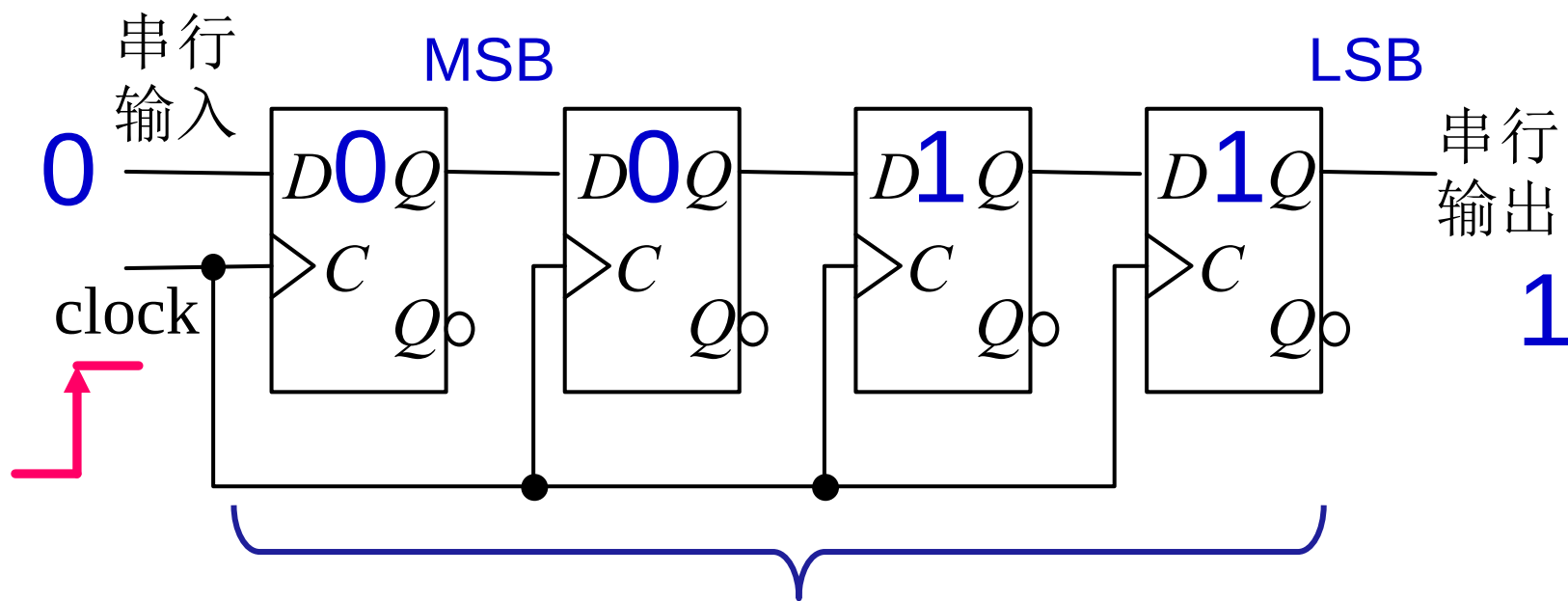
右移移位寄存器

第一个时钟上升沿到来：右移一位



右移移位寄存器

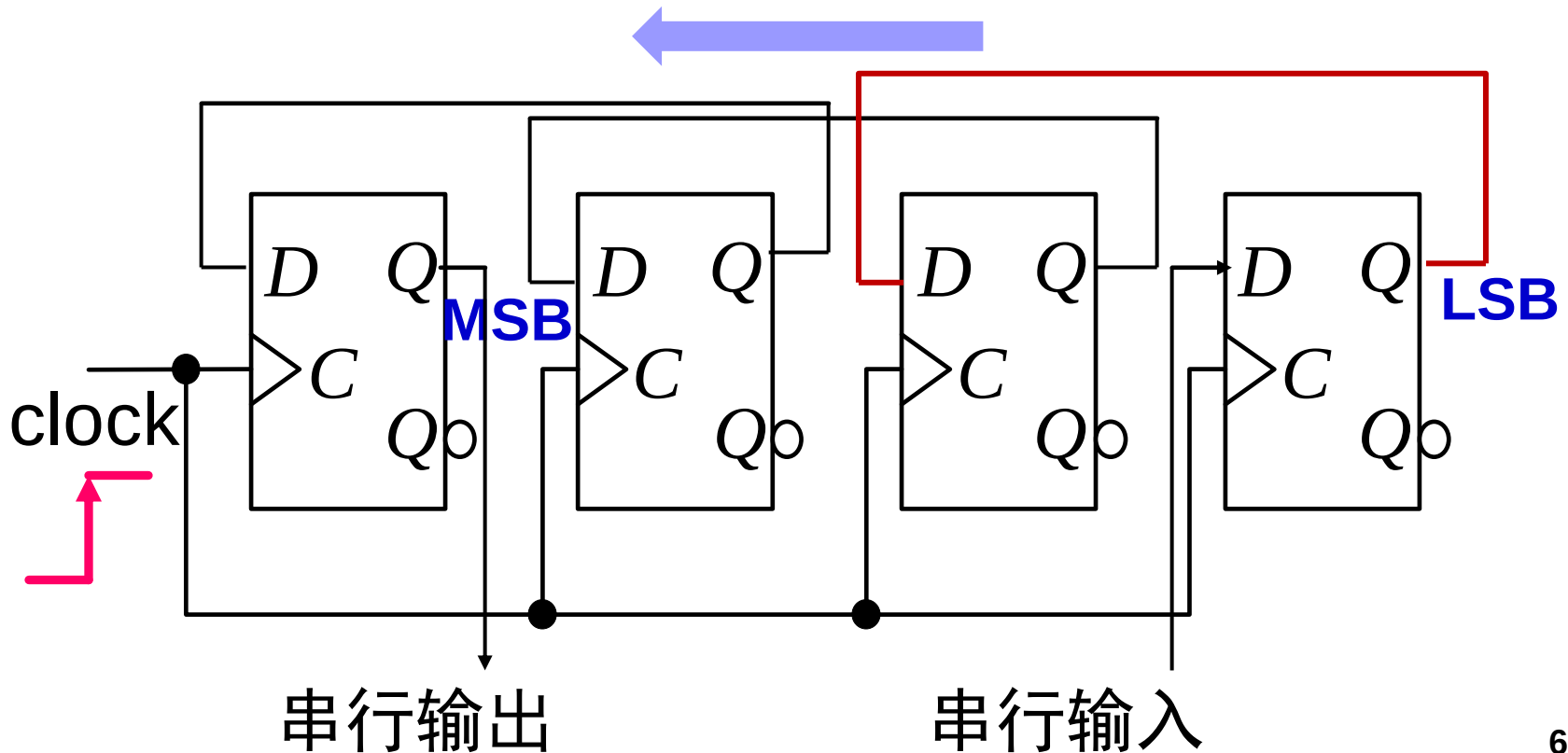
第二个时钟上升沿到来：右移两位



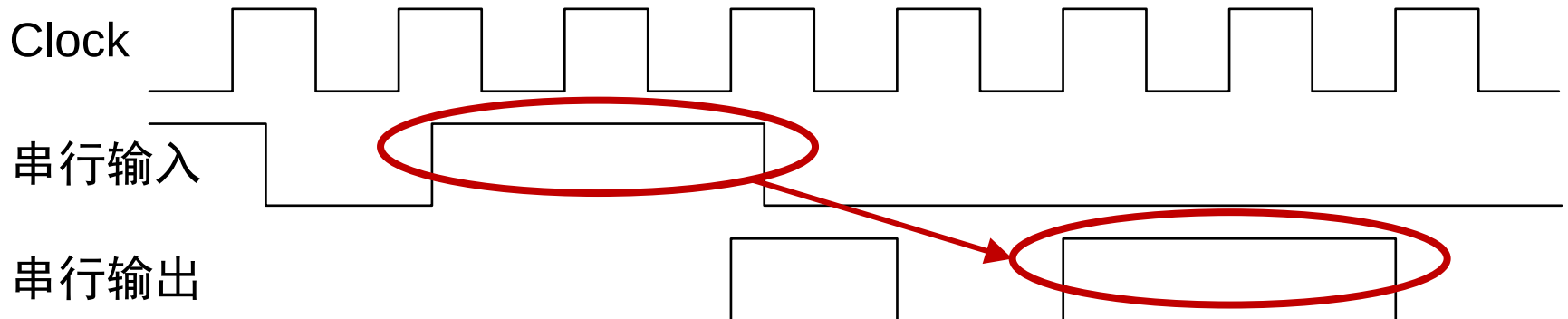
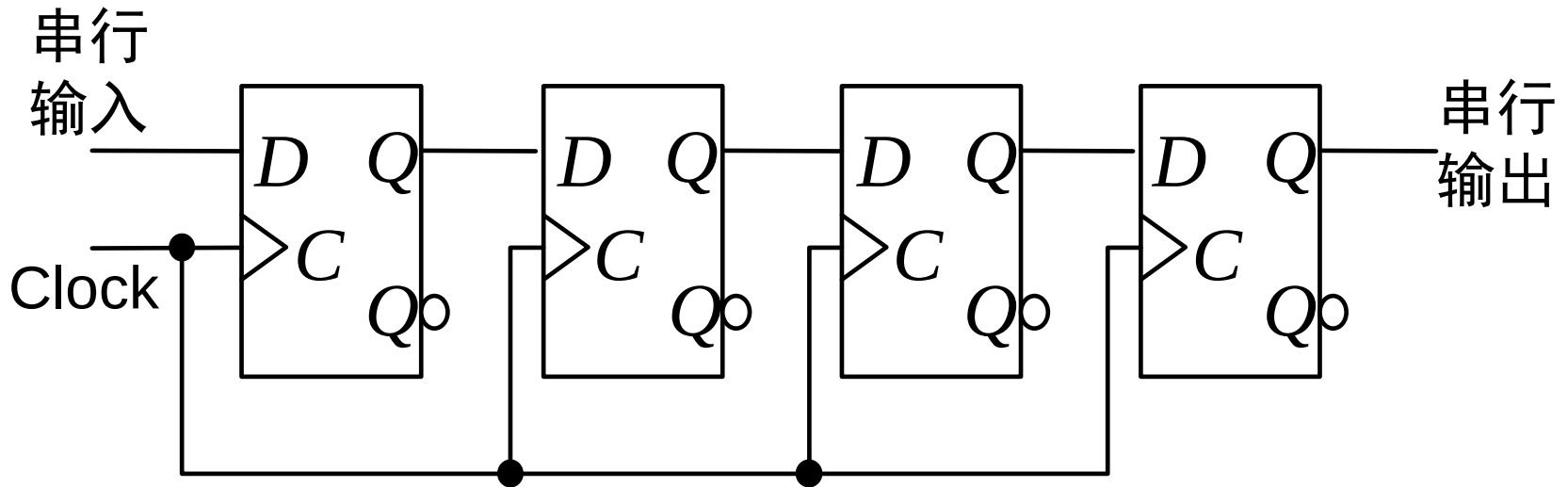
$$14 \div 2^1 \div 2^1 = 3.5 \rightarrow 3 \text{ (取整)}$$

左移移位寄存器

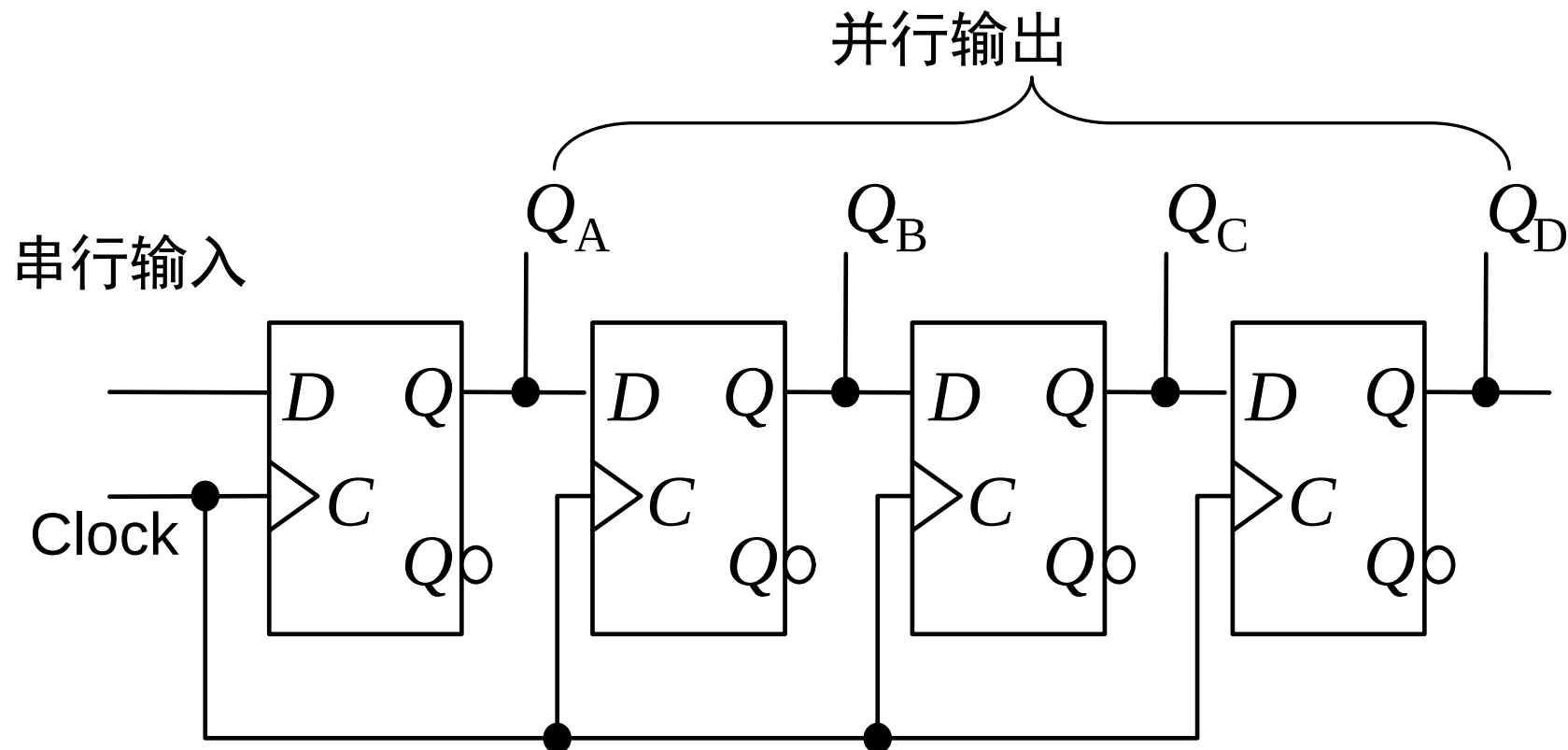
- 分析方法同右移移位寄存器
- 右移→除2，左移→乘2



串行输入，串行输出

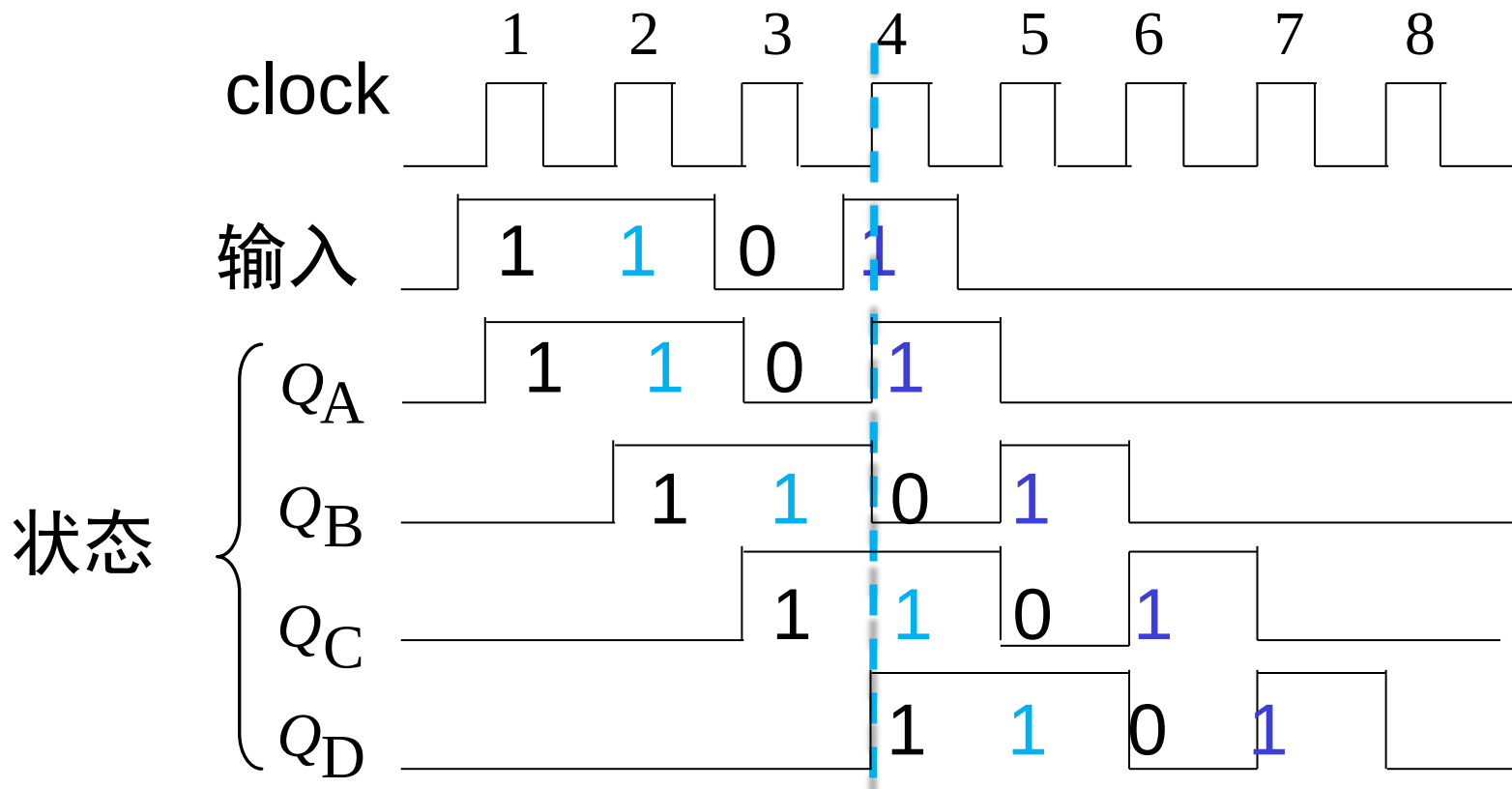


串行输入，并行输出



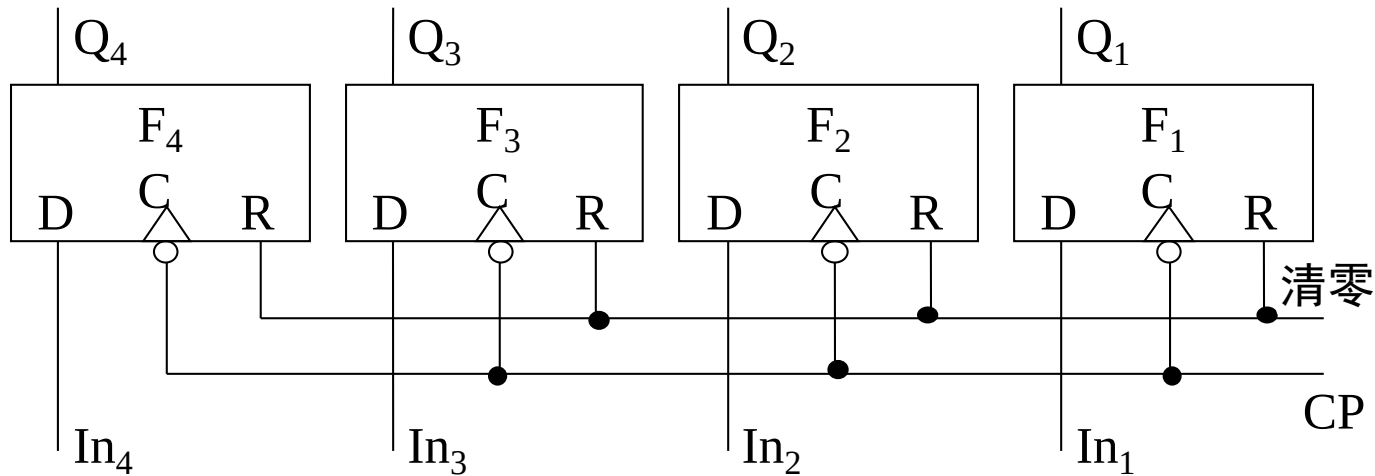
时序图

- 串行输入并行输出的移位寄存器



并行输入，并行输出

- $In_1/In_2/In_3/In_4$ 并行输入至四个触发器中
- 四个触发器并行输出 $Q_1/Q_2/Q_3/Q_4$



串行输出 VS 并行输出

- 串行输出
 - 仅寄存器的最后一个单元中存储的值可直接读出
 - 减少了输出端的数目
- 并行输出
 - 所有触发器的存储值都可同时读出（可见）
 - 输出端数目与触发器个数成正比

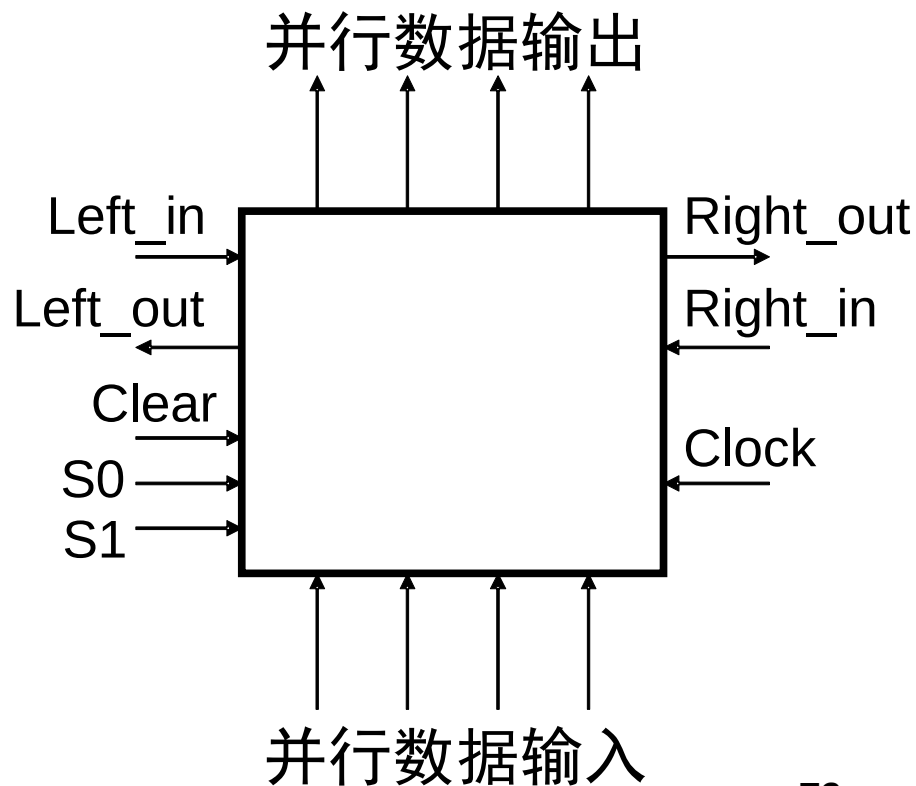
串行输入 VS 并行输入

- 串行输入
 - 只有一个触发器的输入端接收输入数据
 - 将寄存器设置为一组特定的值需要n个周期（n为触发器的数目）
 - 通过3个周期将数据 ‘011’ 写入3-bit寄存器
- 并行输入
 - 寄存器中每个触发器的输入端都可接收数据
 - 允许在一个时钟周期内加载一组新的数值
 - 通过1个周期将数据 ‘011’ 写入3bit寄存器

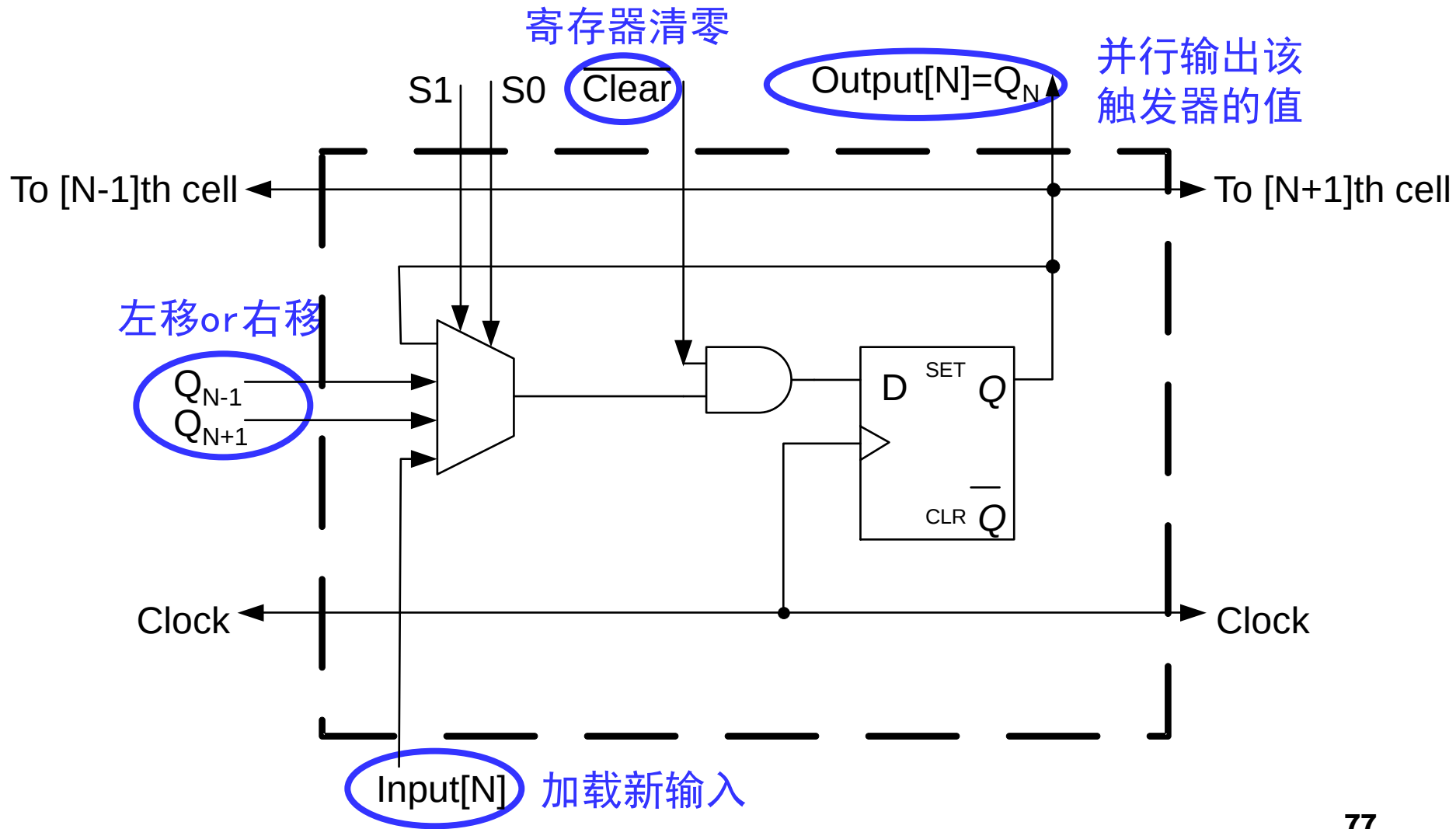
通用移位寄存器

- 双向移位
 - 左移或右移
- 串并输入/串并输出

Clear	S0	S1	Function
0	x	x	清零
1	0	0	保持
1	0	1	右移
1	1	0	左移
1	1	1	并行加载新输入

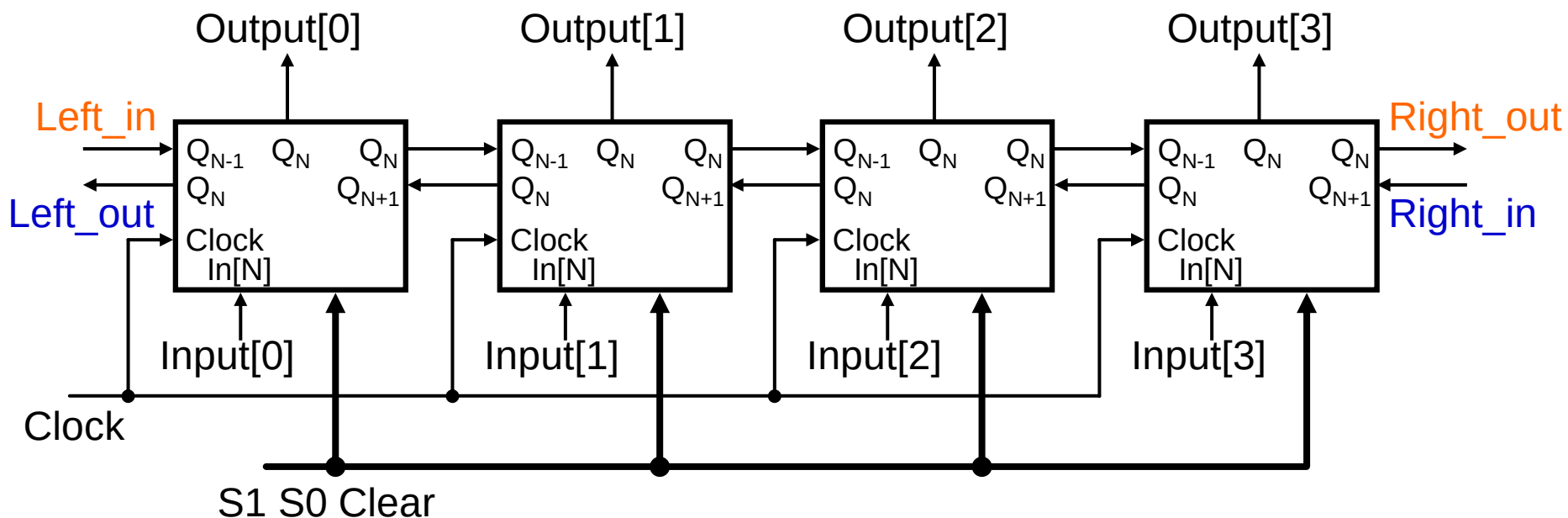


双向移位寄存器的一个单元



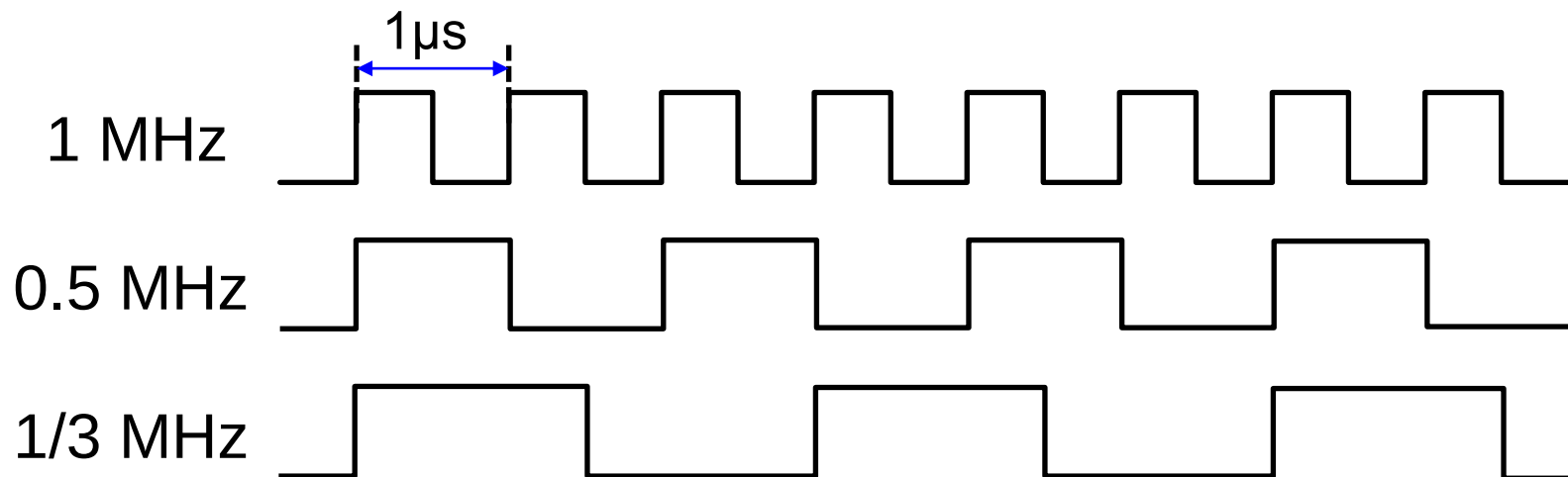
4位双向移位寄存器

- 对前一页的单元进行级联



典型的时序逻辑电路

- 寄存器 Register
- 计数器 Counter
 - 记录输入脉冲的个数，其最大计数个数称作模
 - 用于定时、分频、产生节拍脉冲及运算等



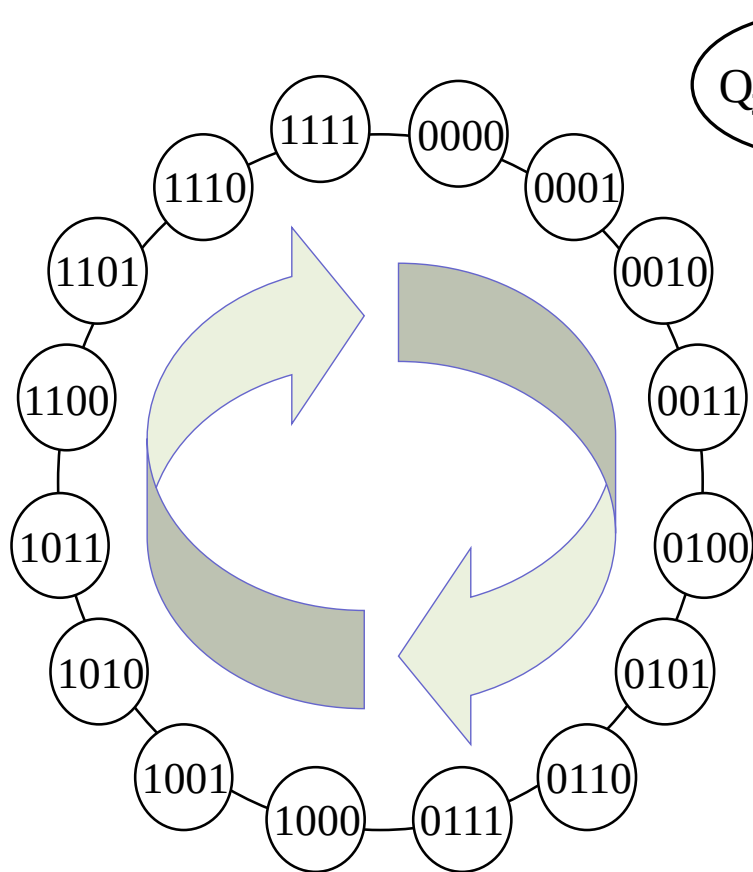
典型的计数器

- 加法/减法计数器
- 特殊进制计数器
- 环形/扭环形计数器
- 异步/同步计数器

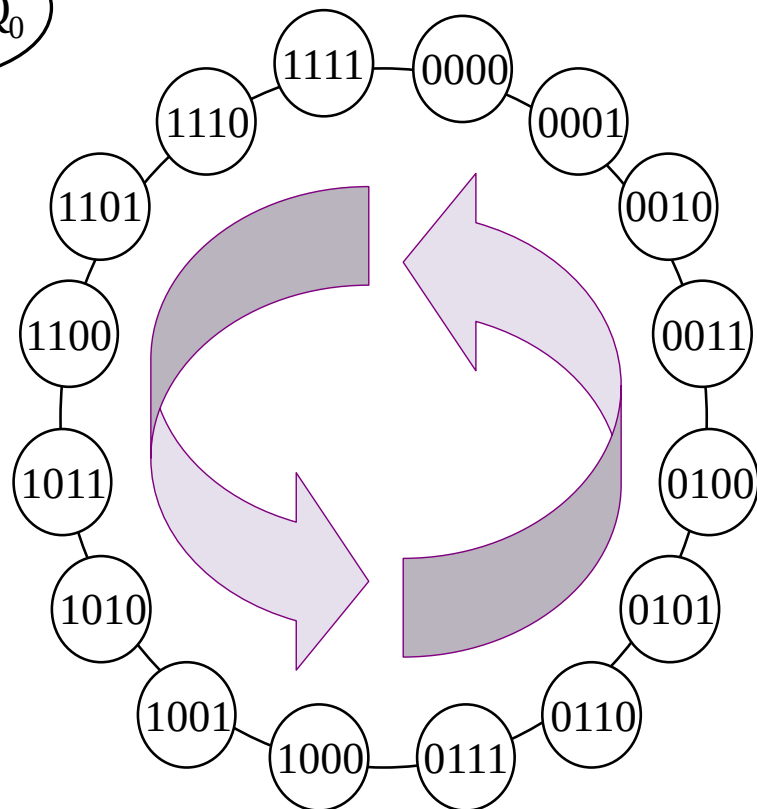
加法up/减法down计数器

- 其输出与状态的值一样
 - 输出计下的数
- 加法计数器
 - 输出状态序列的值增加
 - 00→01→10→11→00
- 减法计数器
 - 输出状态序列的值减少
 - 11→10→01→00→11

加法计数器 vs 减法计数器



加法计数器



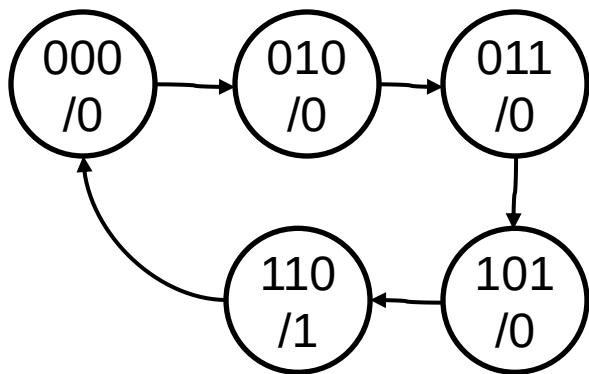
减法计数器

典型的计数器

- 加法/减法计数器
- 特殊进制计数器
- 环形/扭环形计数器
- 异步/同步计数器

例1：设计一个模五的计数器

- Mod=5
- 可用3bit状态变量表示5个状态
- 其他状态：无关项
- 假定状态变化为（不唯一）
 - 000 → 010 → 011 → 101 → 110 → 000
- 仅在最后1个状态110输出1



A	B	C	A ⁺	B ⁺	C ⁺	Z
0	0	0	0	1	0	0
0	0	1	-	-	-	-
0	1	0	0	1	1	0
0	1	1	1	0	1	0
1	0	0	-	-	-	-
1	0	1	1	1	0	0
1	1	0	0	0	0	1
1	1	1	-	-	-	-

例1：设计一个模五的计数器

A	B	C	A^+	B^+	C^+	Z
0	0	0	0	1	0	0
0	0	1	-	-	-	-
0	1	0	0	1	1	0
0	1	1	1	0	1	0
1	0	0	-	-	-	-
1	0	1	1	1	0	0
1	1	0	0	0	0	1
1	1	1	-	-	-	-

- 列出次态方程

ABC

BC	00	01	11	10
A				
0	010	---	101	011
1	---	110	---	000

$$A^+ = C$$

$$B^+ = B' + A'C'$$

$$C^+ = A'B$$

例1：设计一个模五的计数器

A	B	C	A^+	B^+	C^+	Z
0	0	0	0	1	0	0
0	0	1	-	-	-	-
0	1	0	0	1	1	0
0	1	1	1	0	1	0
1	0	0	-	-	-	-
1	0	1	1	1	0	0
1	1	0	0	0	0	1
1	1	1	-	-	-	-

- 列出输出方程

BC	00	01	11	10
A				
0	0	-	0	0
1	-	0	-	1

$$Z = AB \text{ or } Z = AC'$$

Step 5: 画出逻辑电路图

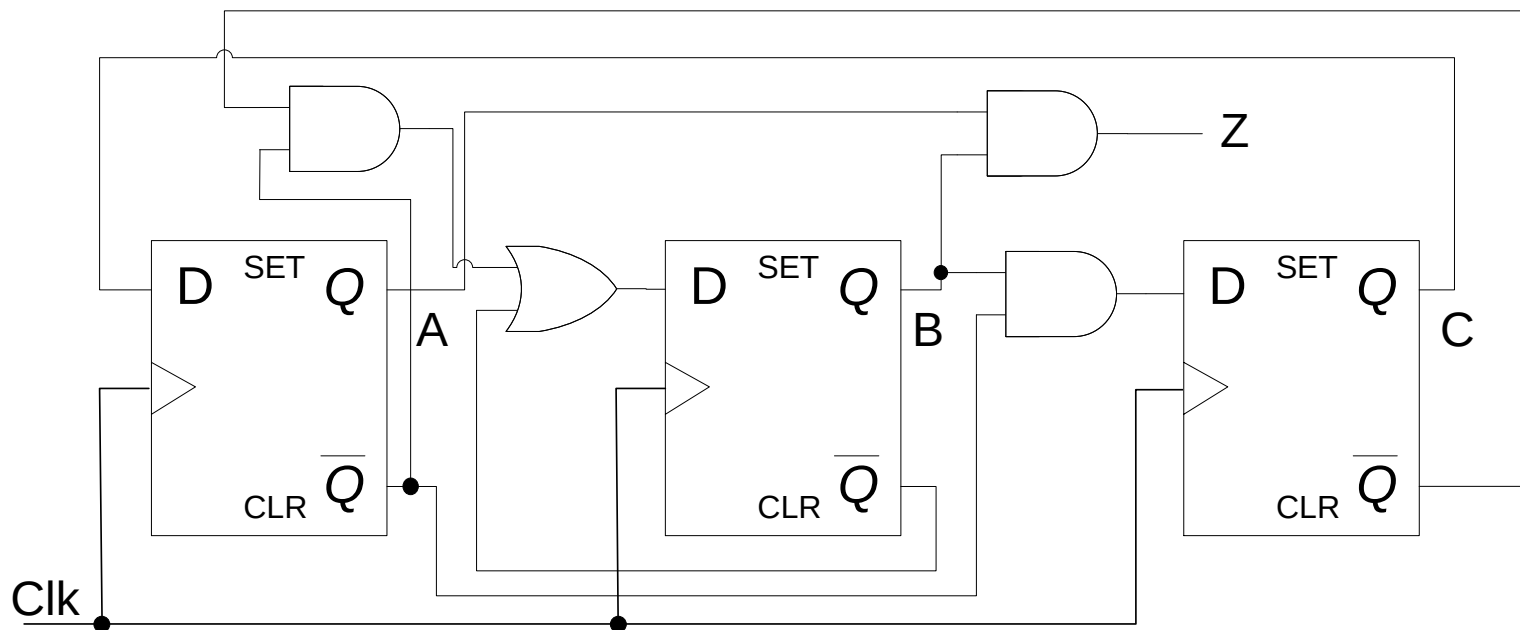
- 利用D触发器

$$A^+ = C$$

$$C^+ = A'B$$

$$B^+ = B' + A'C'$$

$$Z = AB$$



课后练习：课后自行确认该电路

计数器的自启动问题

- 在上例模五计数器中
 - 状态转移: $000 \rightarrow 010 \rightarrow 011 \rightarrow 101 \rightarrow 110 \rightarrow 000$
 - 无关项: 001, 100, 111
- 如果计数器初始化为上述三个状态之一?

计数器的自启动问题

- 自启动
 - 处在非正常工作状态时如果能够经过有限次状态转换进入正常工作状态
- 计数器例子
 - 非正常工作状态: 001, 100, 111
 - 正常工作状态: 其他五个状态
 - 无需其他干预
 - 不需要人为对计数器设置初值

✓ 000→010→011→
101→110→000

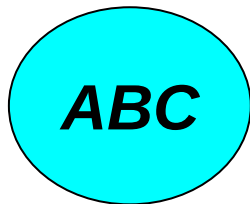
例1能否自启动?

$$A^+ = C$$

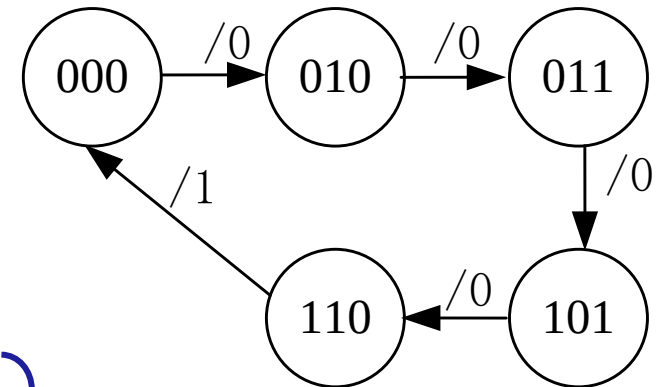
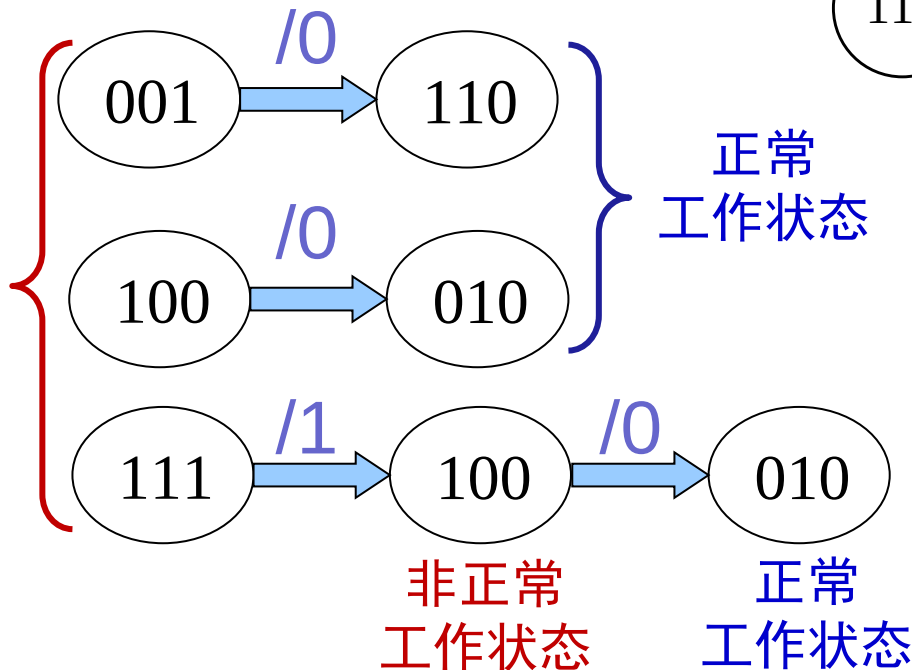
$$B^+ = B' + A'C'$$

$$C^+ = A'B$$

$$Z = AB$$



非正常
工作状态



可以自启动!

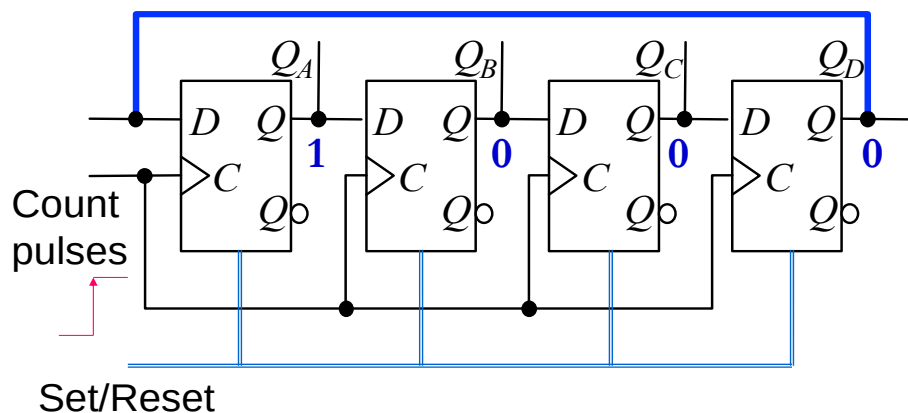
典型的计数器

- 加法/减法计数器
- 特殊进制计数器
- 环形/扭环形计数器
- 异步/同步计数器

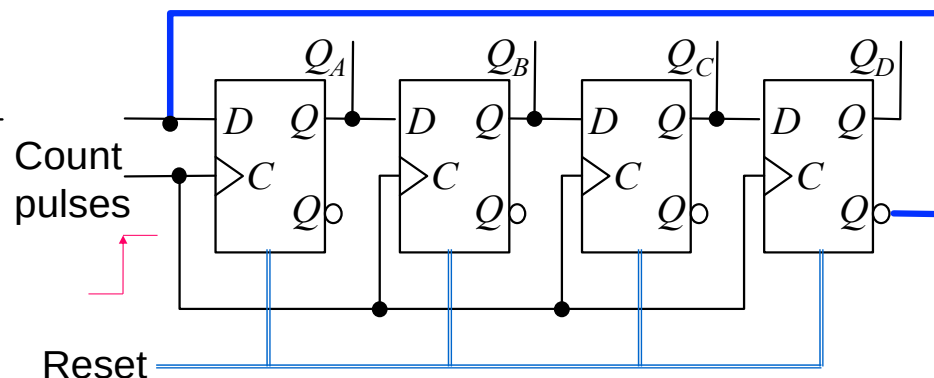
基于移位寄存器的计数器

- 环形计数器
- 扭环形计数器

循环移位寄存器

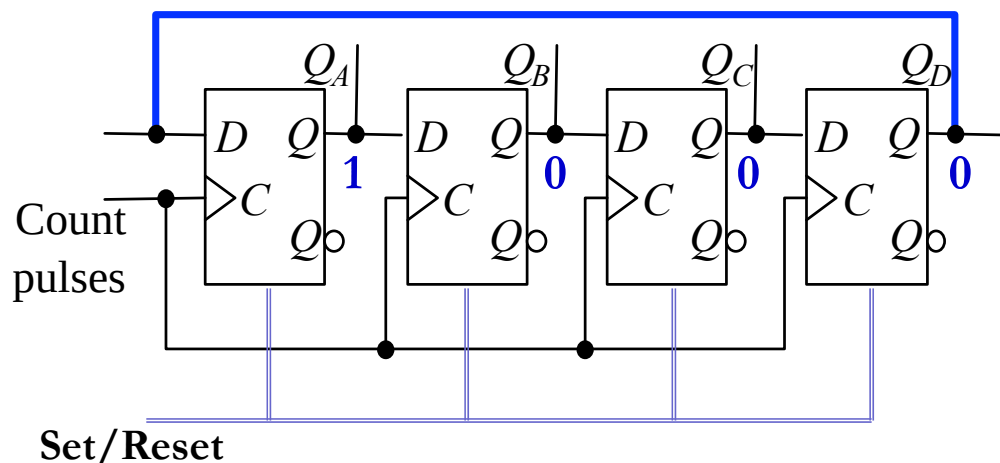


❖ 环形计数器



❖ 扭环形计数器

环形计数器的有限状态机

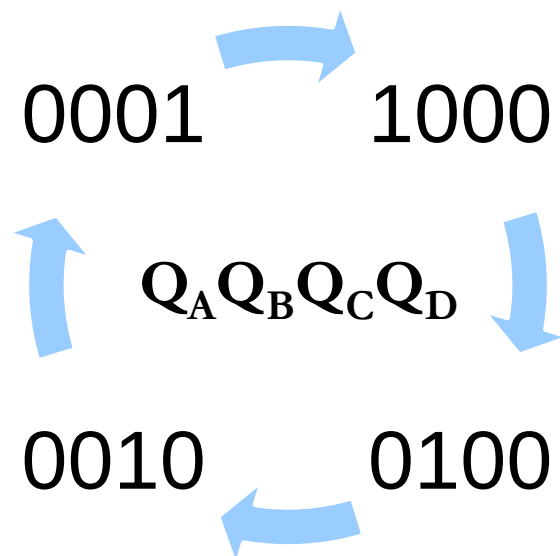


N个触发器表示N个不同的状态
 → 触发器代价！！

Q_A	Q_B	Q_C	Q_D	D
1	0	0	0	8
0	1	0	0	4
0	0	1	0	2
0	0	0	1	1
1	0	0	0	8

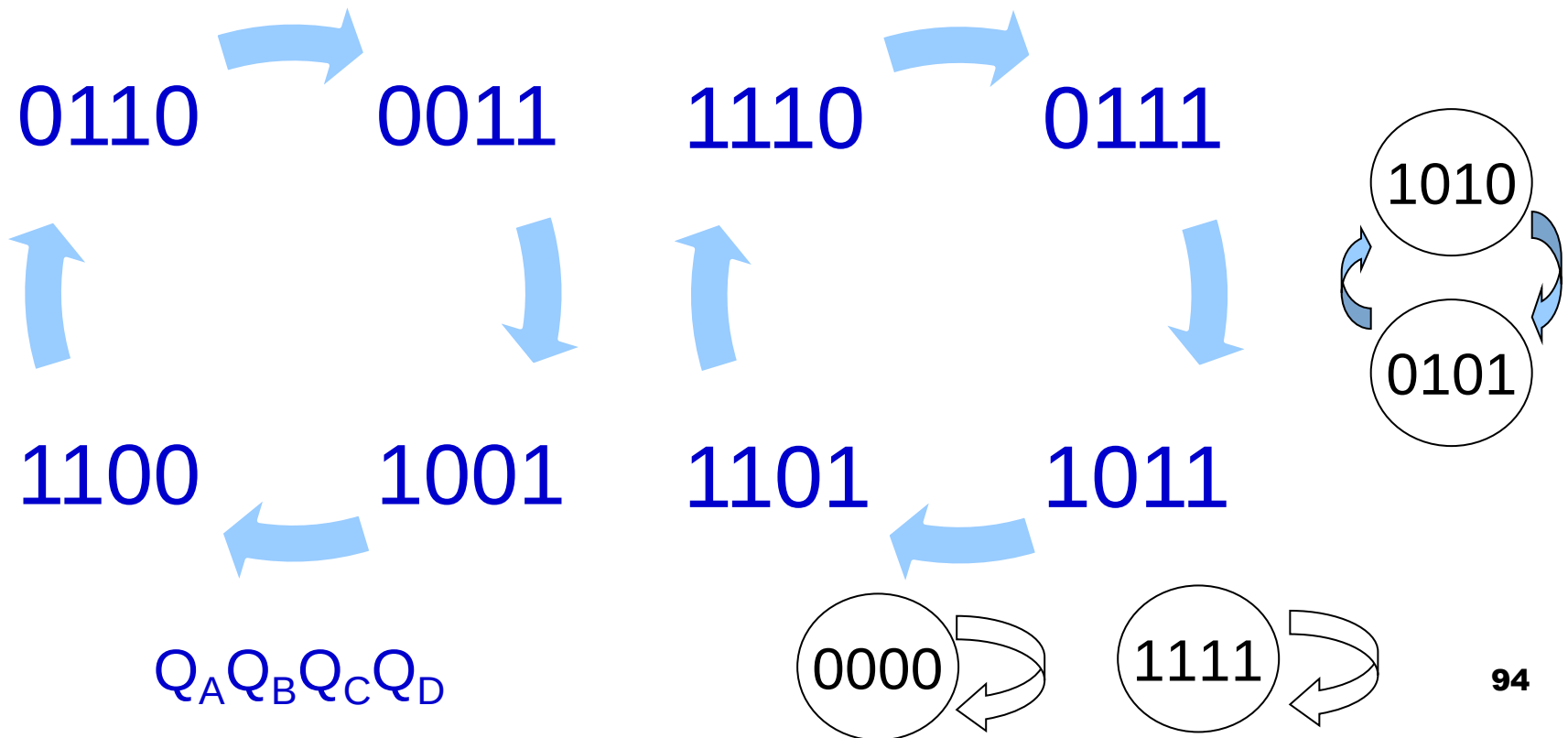
正常计数序列

模4计数器



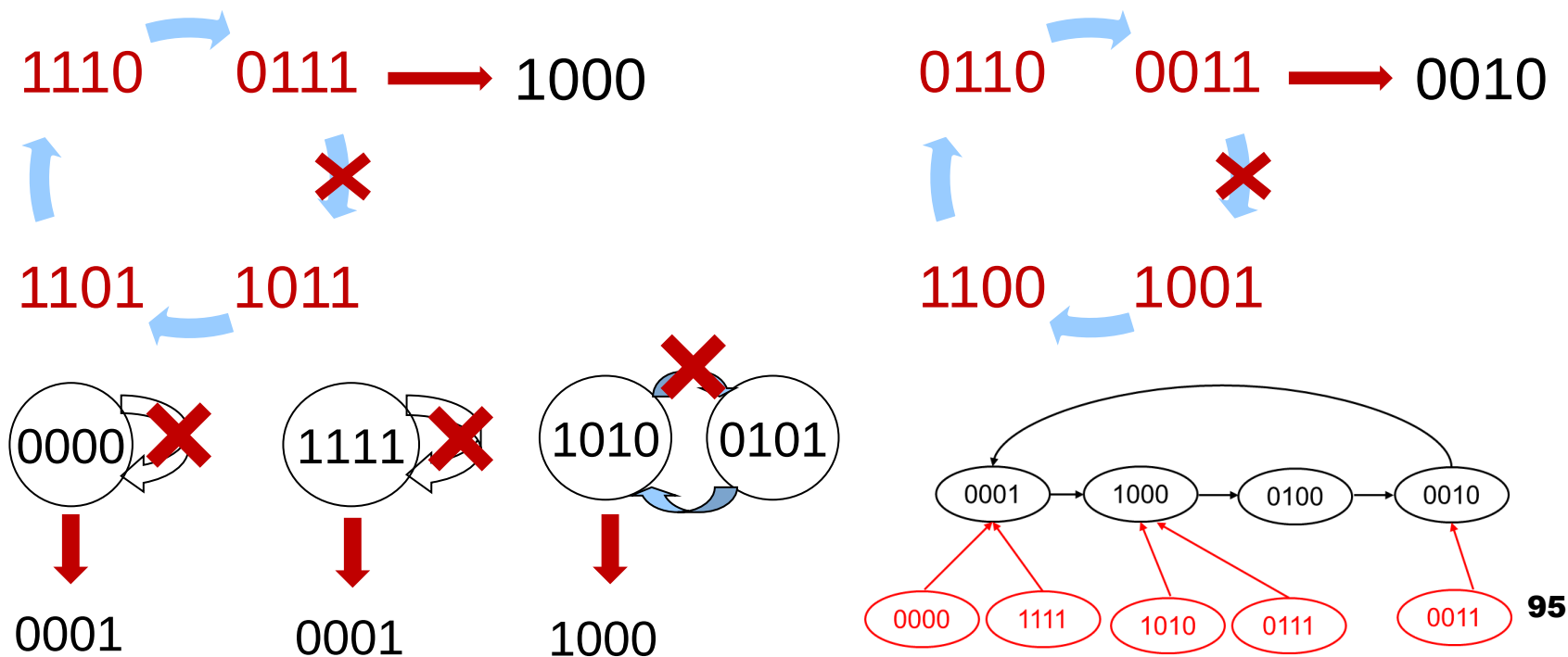
环形计数器: 没有使用的状态

- N个触发器一共有 2^N 个状态
 - $2^N - N$ (12 状态, 当 $N=4$) 没有使用
 - 自启动问题: 如果电路起始于这12个状态?



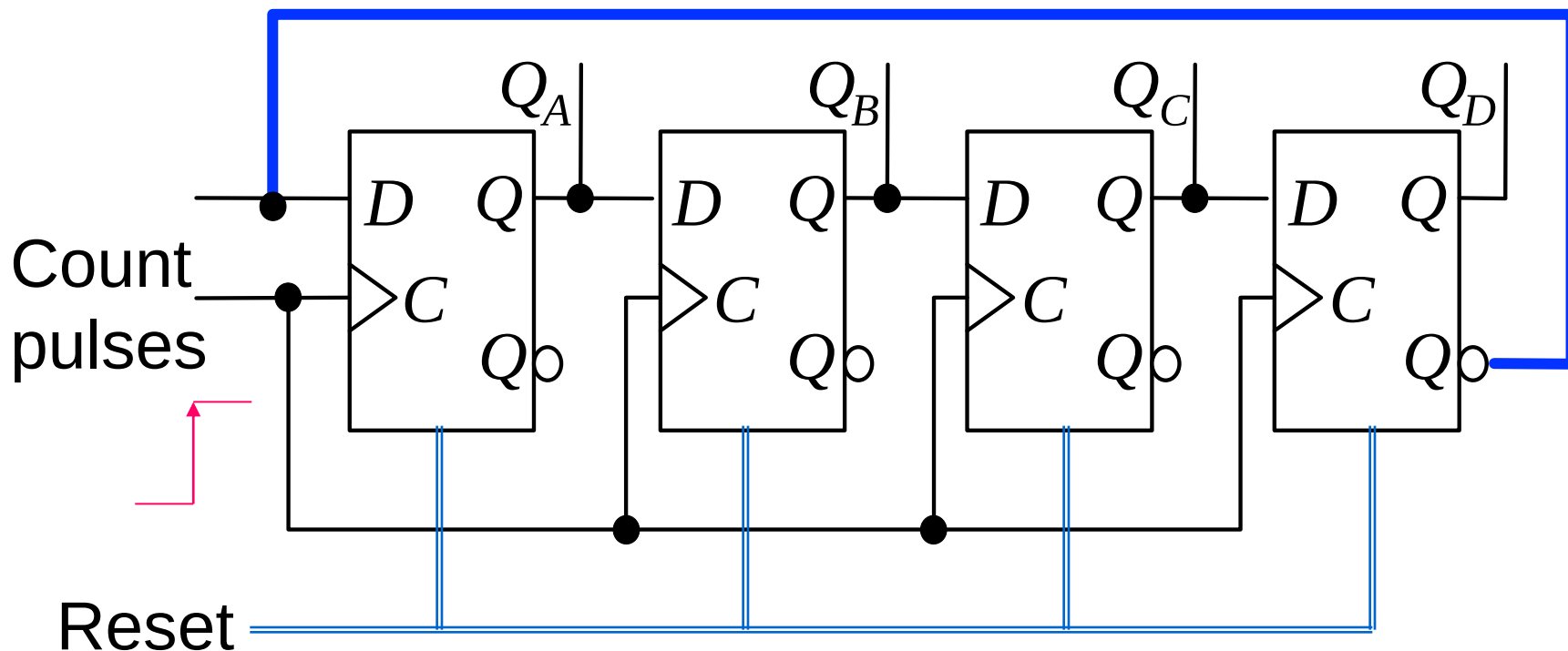
环形计数器:自启动设计

- 方法1: 预置和复位
 - 重新加载触发器的状态
- 方法2: 改变状态转移逻辑
 - 增加从非正常态到正常态的转移

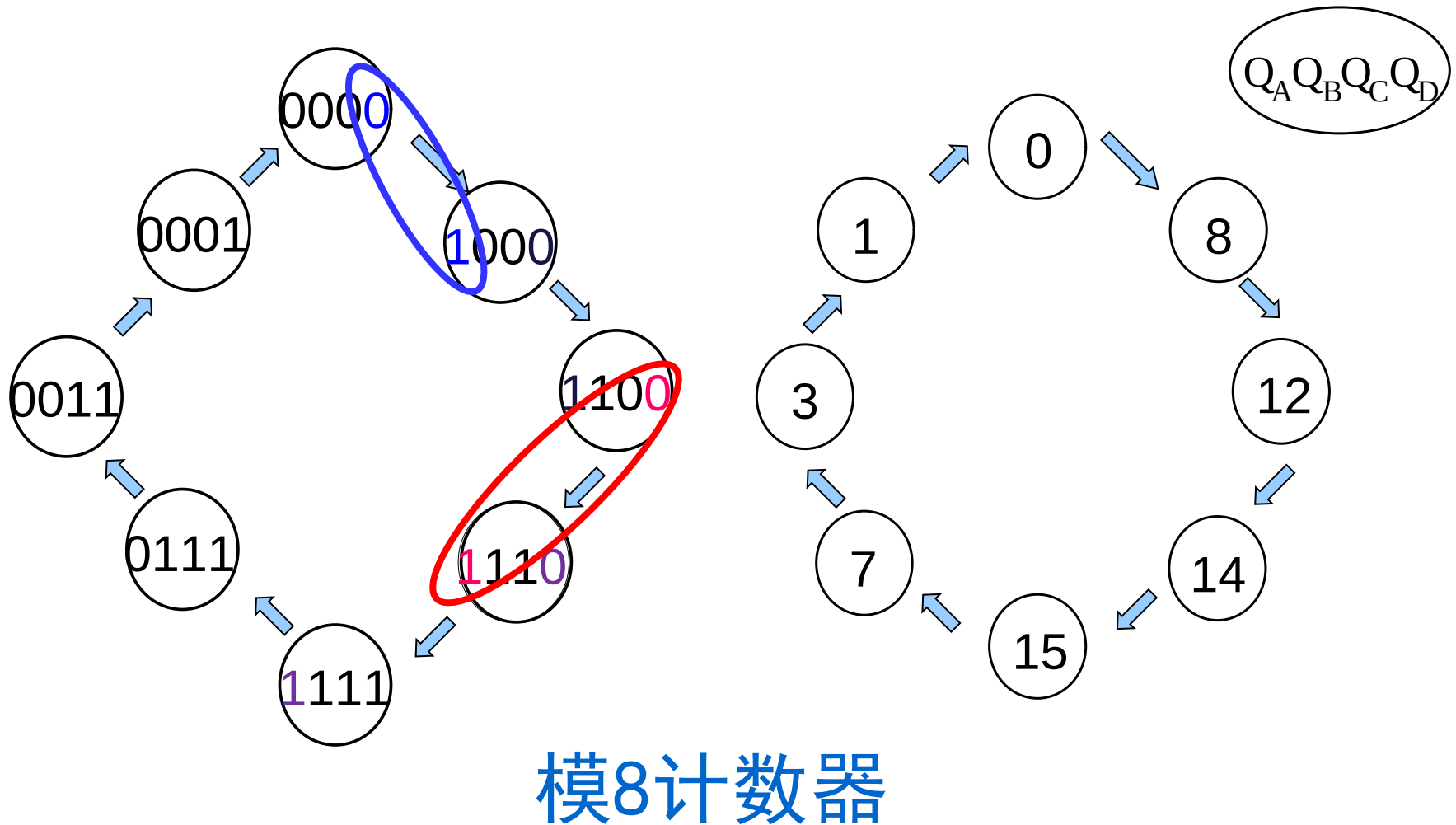


扭环形计数器的逻辑电路

- 将移位寄存器的串行输出的反接回串行输入

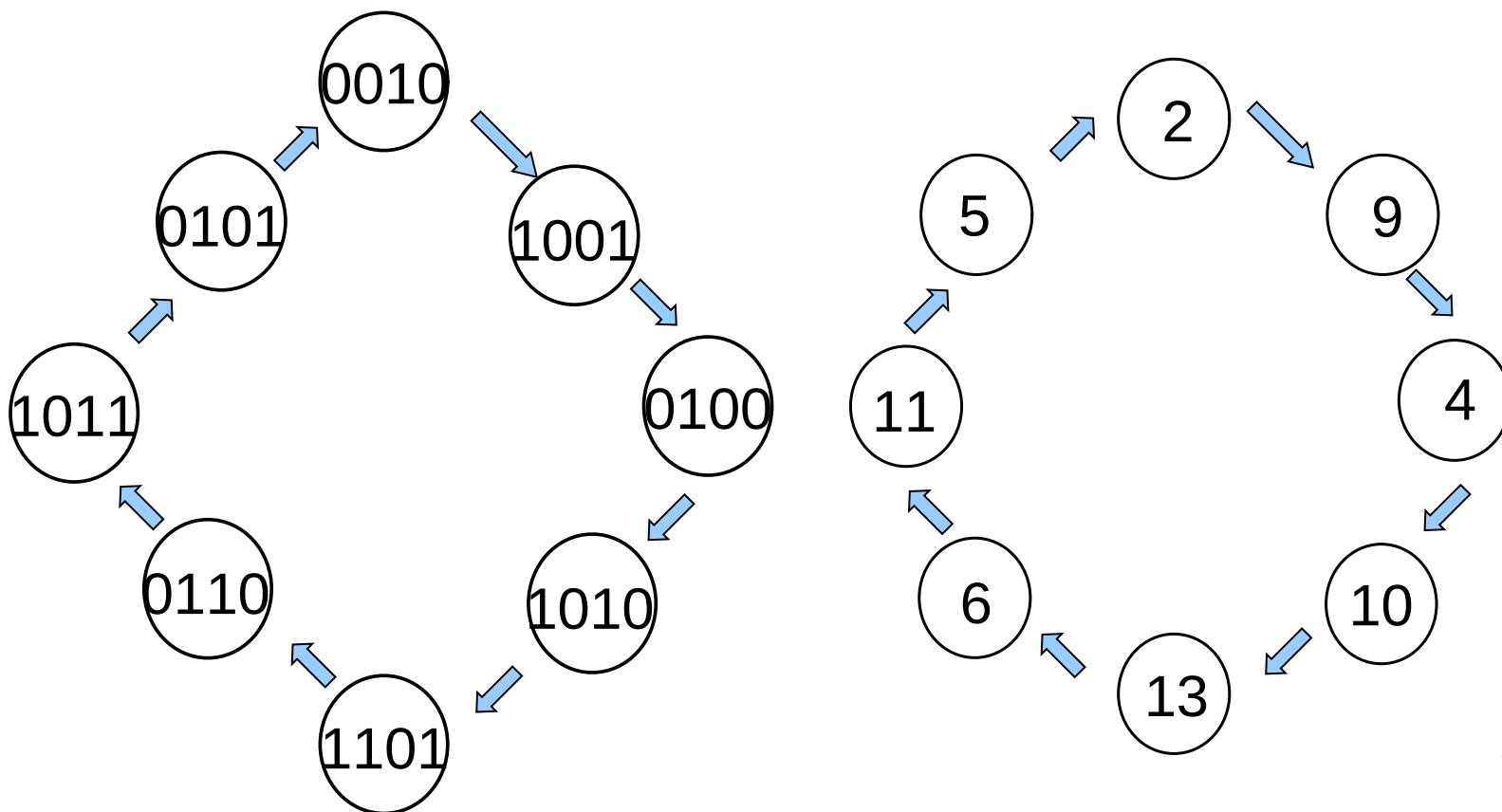


扭环形计数器的状态图



扭环形计数器的自启动?

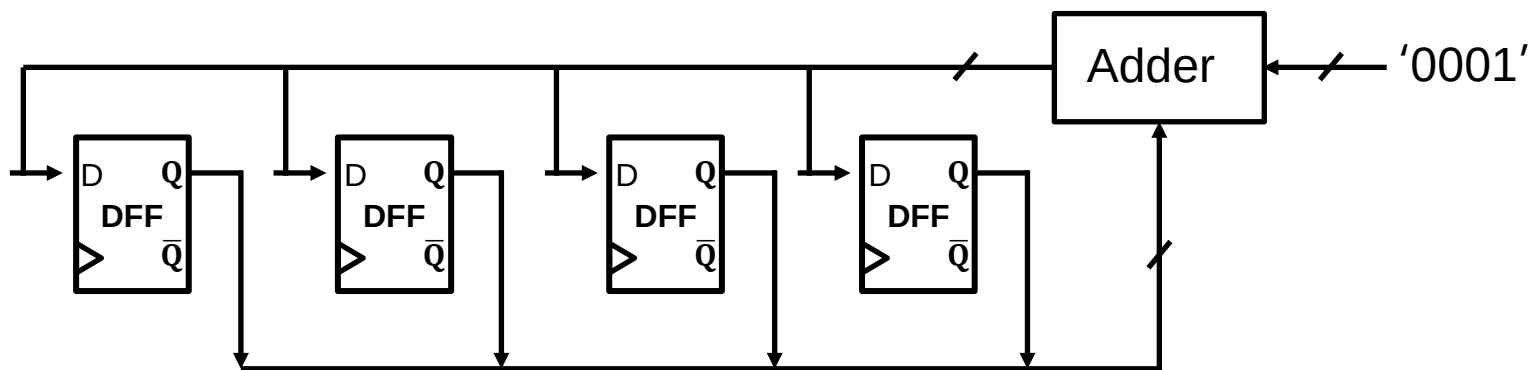
- 还有8个状态呢?
- 自启动设计



扭环形计数器的特点

- 模为 $2N$ 的扭环形计数器
 - N 个触发器，计数序列中 $2N$ 个不同状态
- 比环形计数器的模增加一倍
- 触发器的利用效率仍然不高
 - Q: 如何提升?
 - A: 异步/同步计数器

2^N vs $2N$

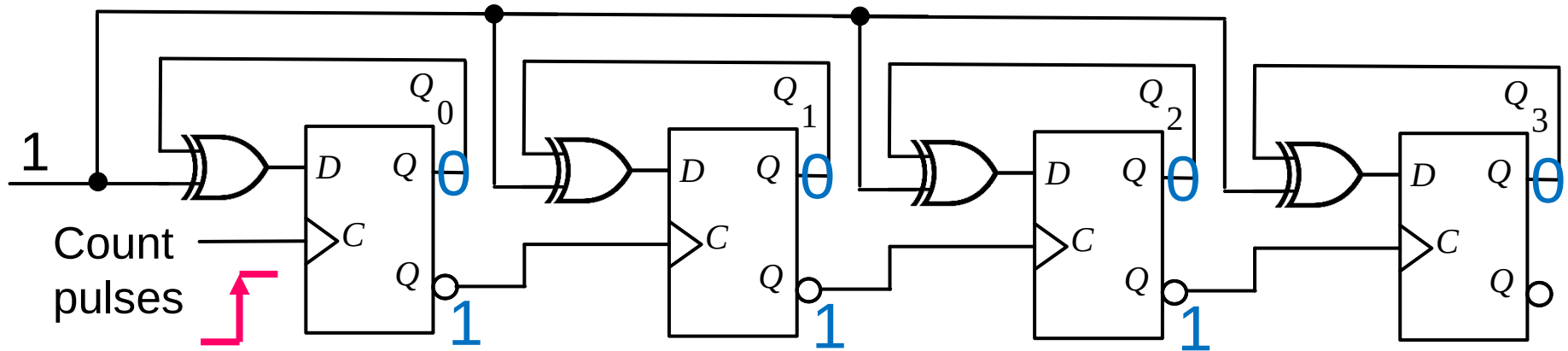


典型的计数器

- 加法/减法计数器
- 特殊进制计数器
- 环形/扭环形计数器
- 异步/同步计数器

异步计数器

- 异步
 - 各个触发器没有使用同一个时钟信号

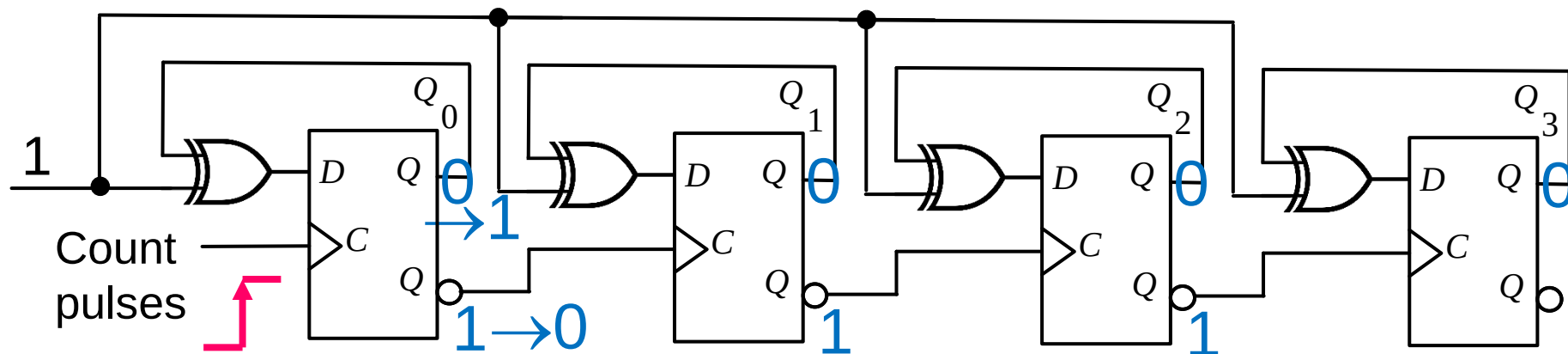


$$\text{次态方程: } Q^+ = \overline{Q} \oplus Q$$

Q: 计数脉冲上升沿到来会使计数器如何变化?

异步计数器

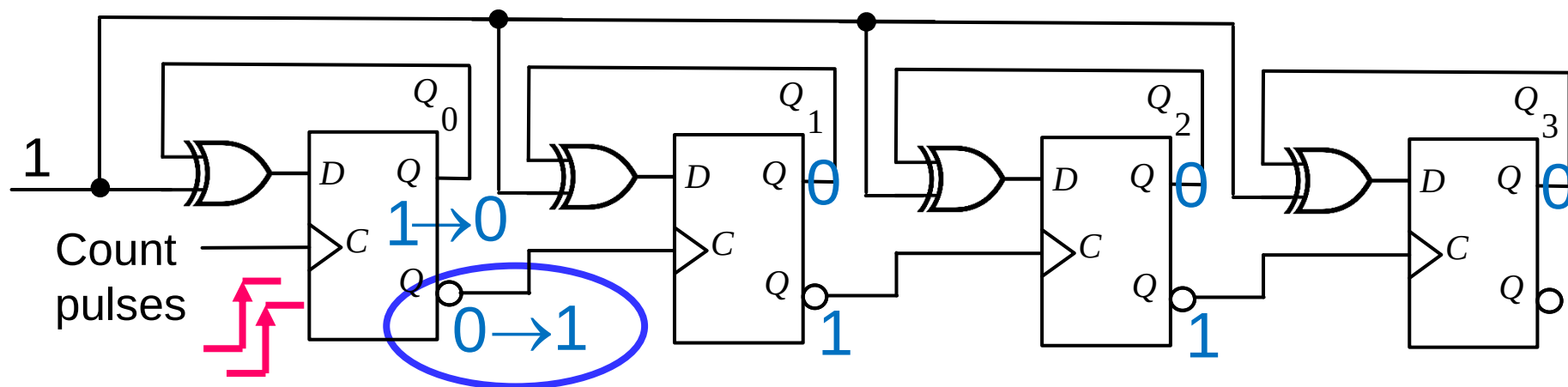
- 异步
 - 各个触发器没有使用同一个时钟信号



异步计数器

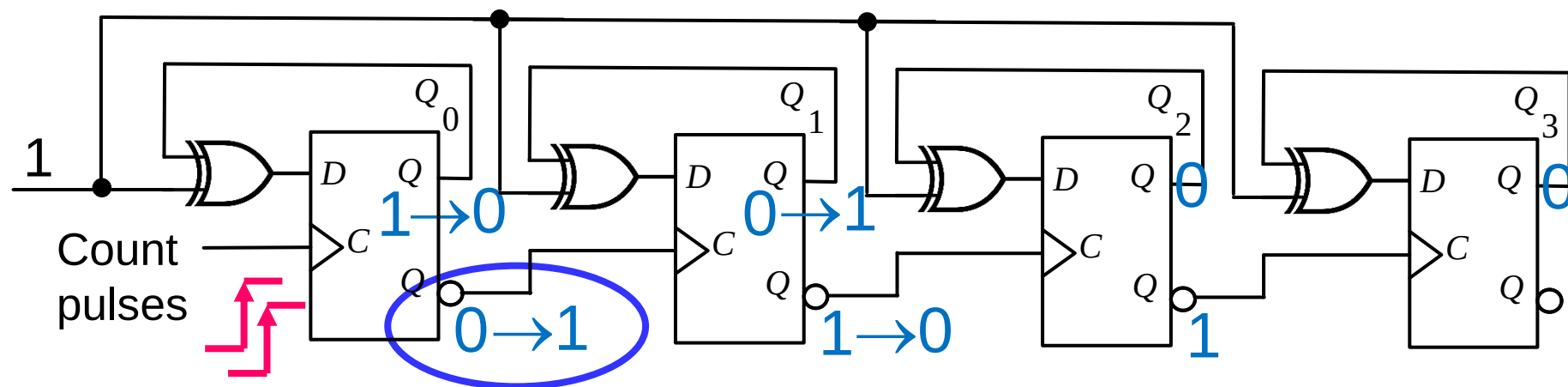
- 异步

- 各个触发器没有使用同一个时钟信号



异步计数器

- 异步
 - 各个触发器没有使用同一个时钟信号

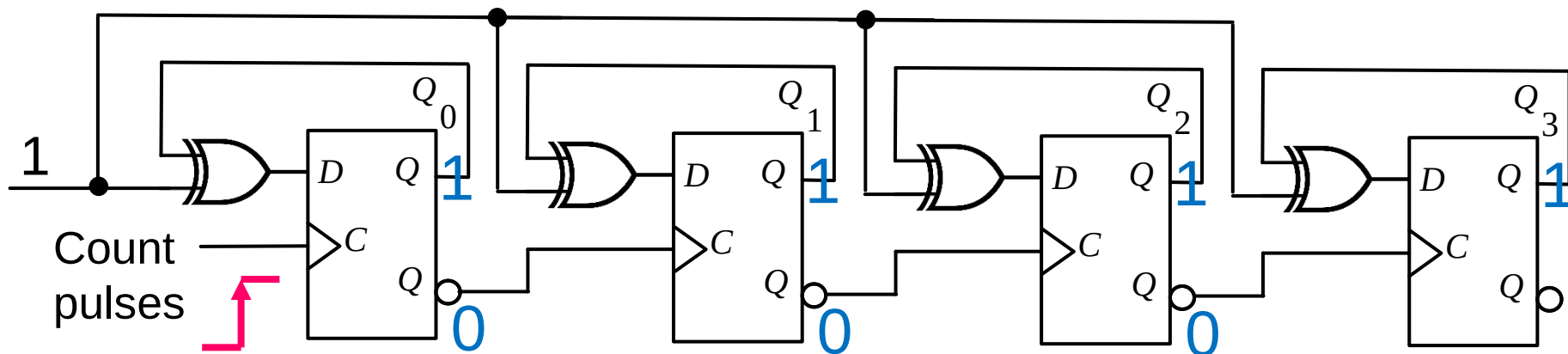


再来一个计数脉冲

这个时钟信号送给下一个触发器

异步计数器

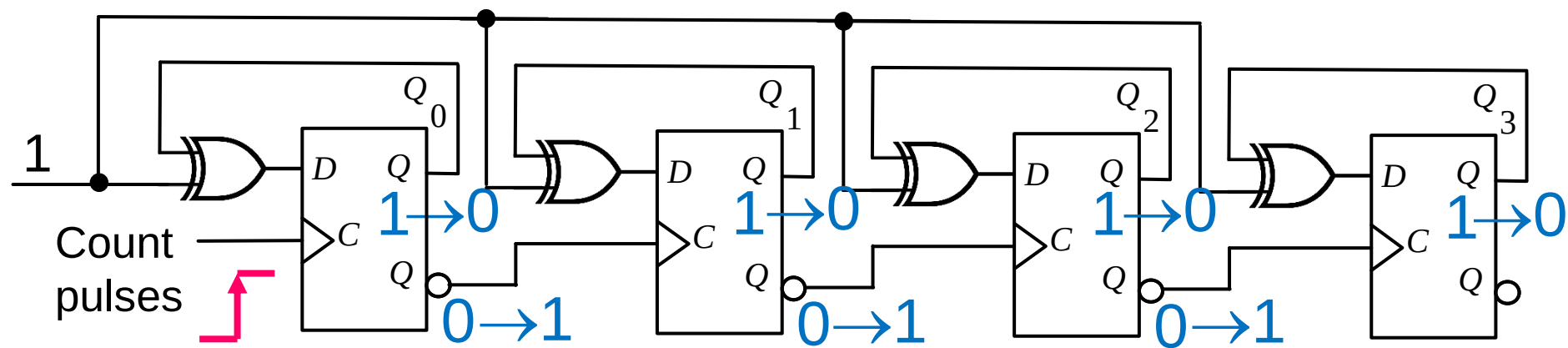
- 异步
 - 各个触发器没有使用同一个时钟信号



当计数器全1时，再来一个计数脉冲会如何？

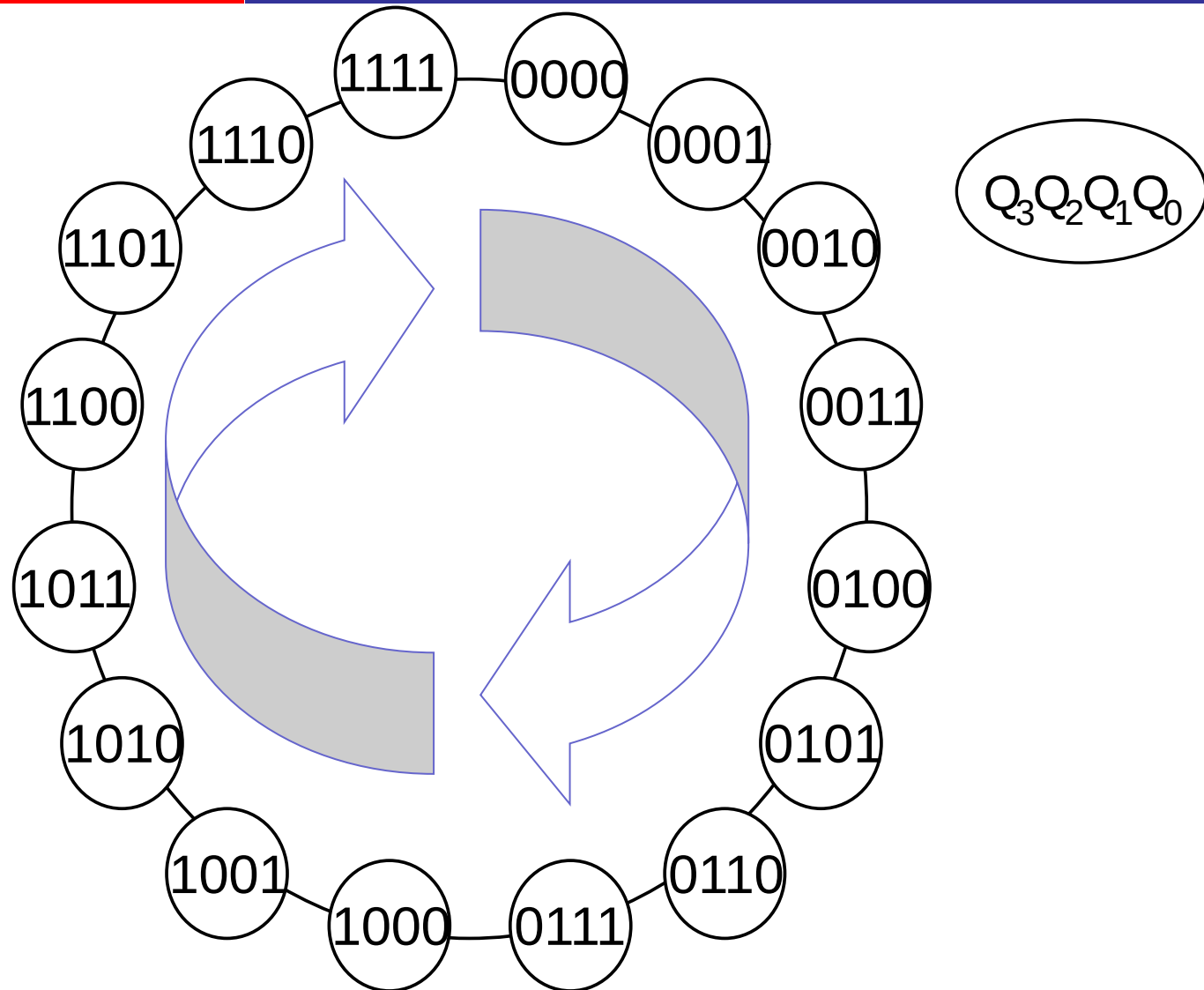
异步计数器

- 异步
 - 各个触发器没有使用同一个时钟信号



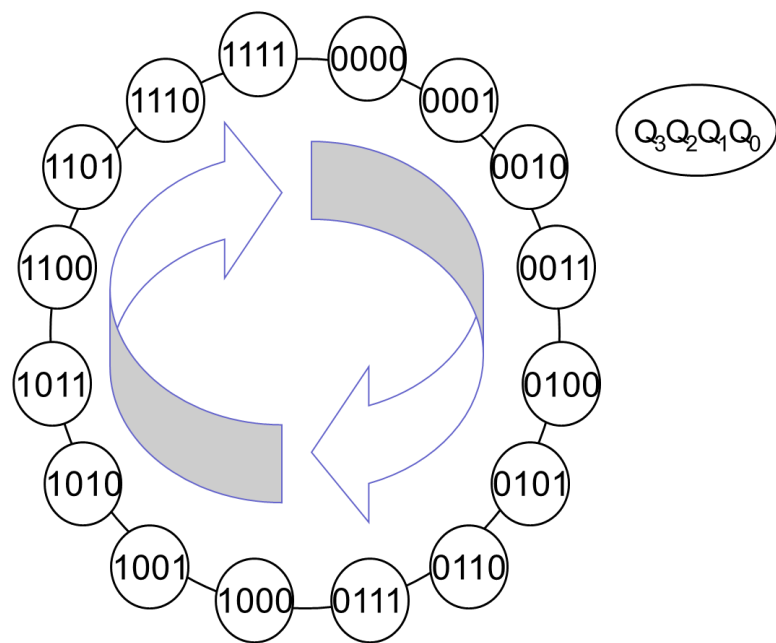
从 1111 到 0000

模16的加法计数器



同步模16加法计数器设计

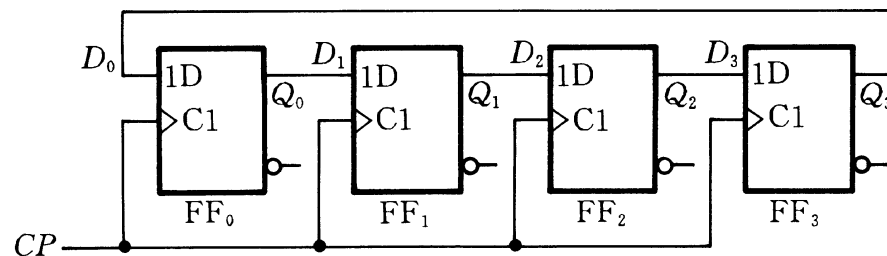
- 异步计数器
 - 不同的触发信号
- 同步计数器
 - 所有触发器相同触发信号
- 现象
 - Q_0 必须翻转
 - Q_i 是否翻转，取决于所有低位的触发器的状态是否都为1



同步 vs 异步

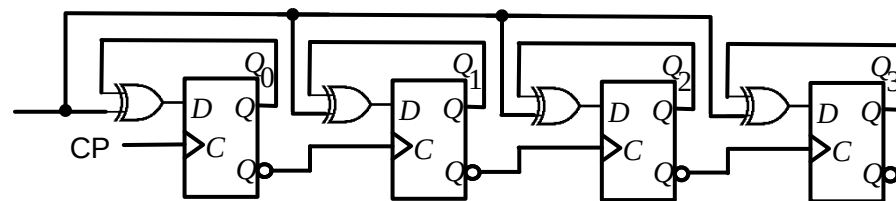
- 同步时序电路

- 所有触发器 “相同” 时钟
- 状态在同一个时钟触发沿变化



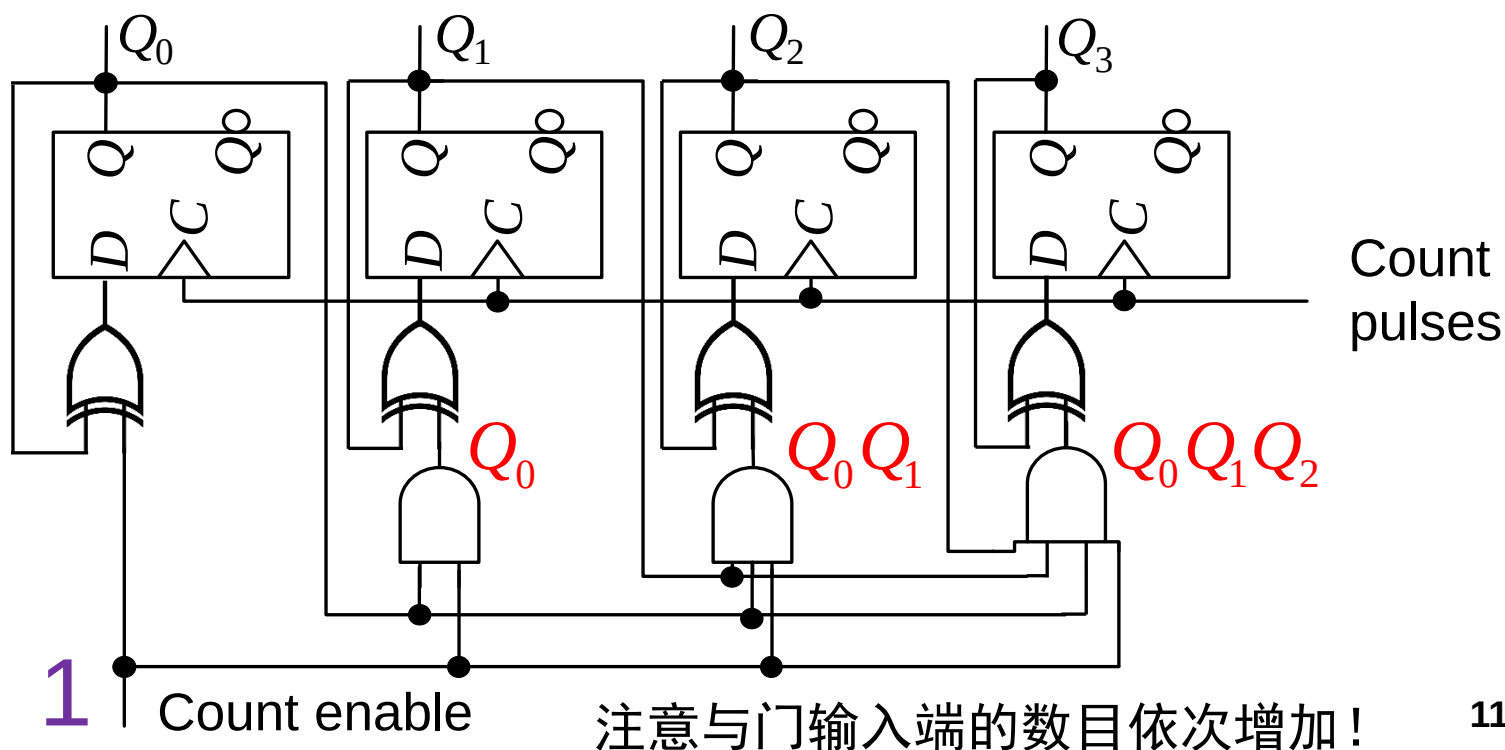
- 异步时序电路

- 没有时钟, or 不同时钟
- 状态在不同时钟的不同触发沿改变

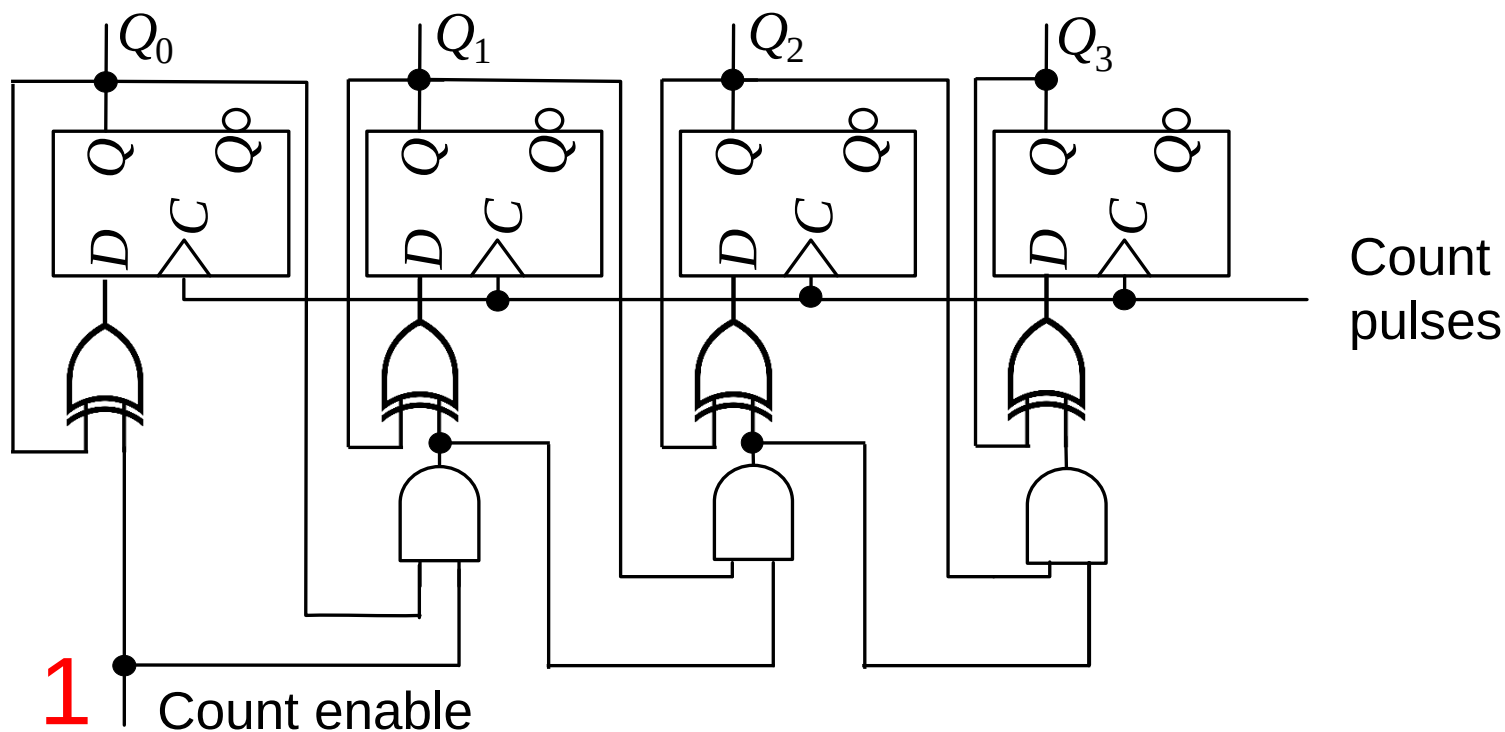


同步模16计数器

- Q_0 必须翻转
 - $Q^+ = In \oplus Q$, set In to 1
- Q_i 是否翻转, 取决于所有低位的触发器的状态是否都为1
 - In_i 为所有低位触发器状态的与

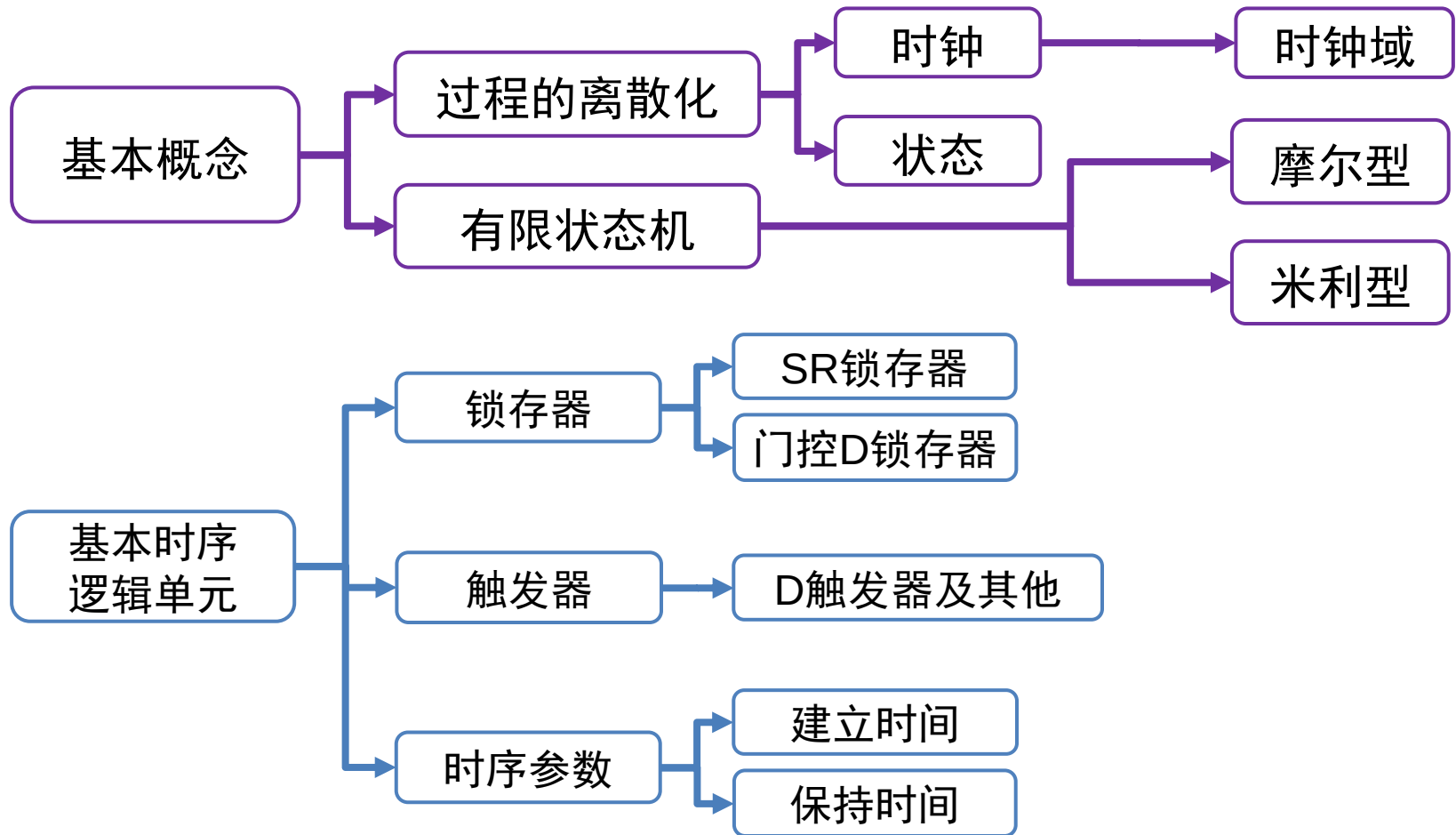


同步模16计数器

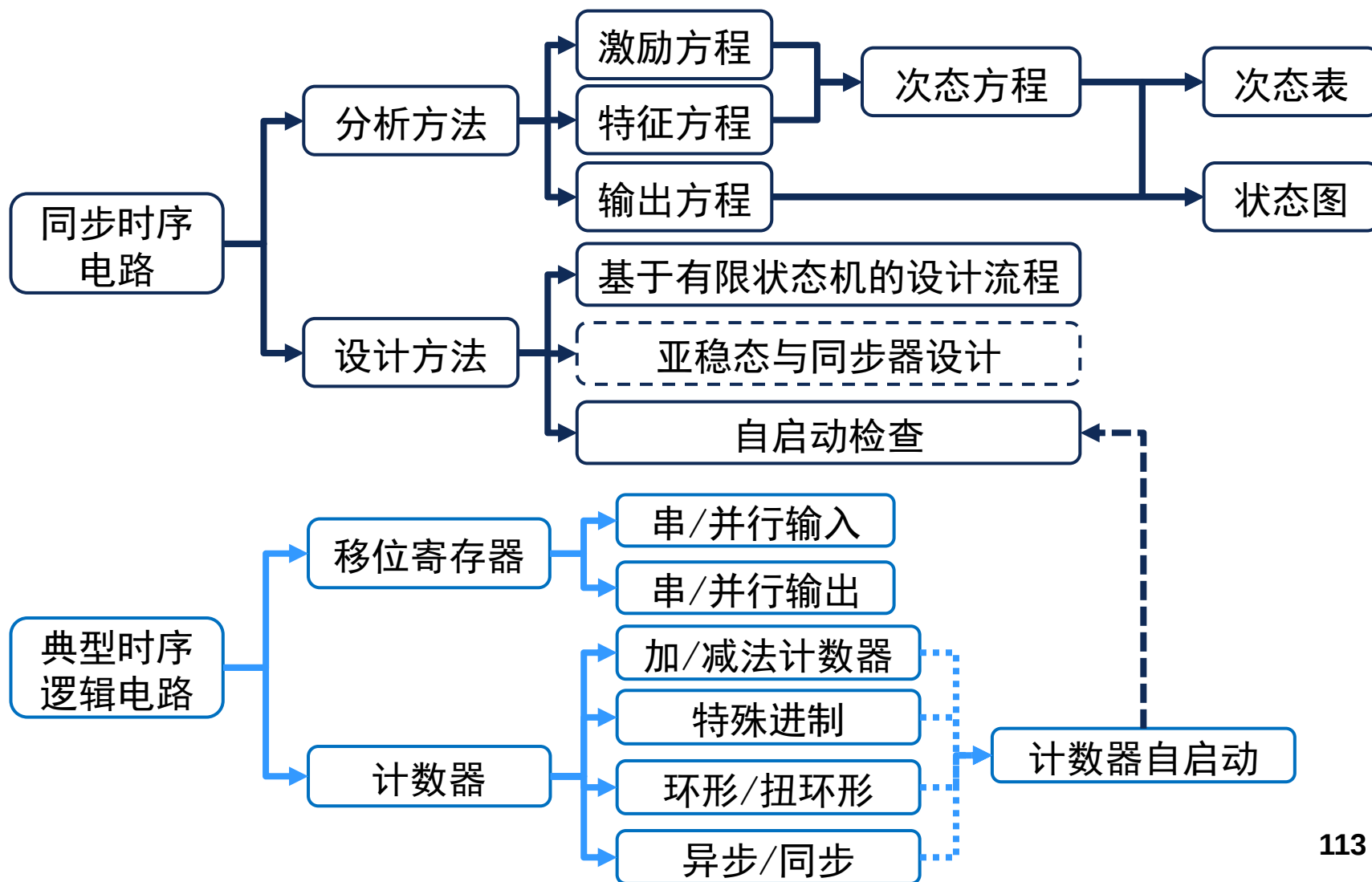


注意速度和面积的平衡/折中

时序逻辑总结(1/2)



时序逻辑总结(1/2)



作业

- 要求
 - 线上交电子版
 - 不要抄袭，迟交一周内扣50%，答案公布后再交不给分
- 本次作业
 - 网络学堂作业区

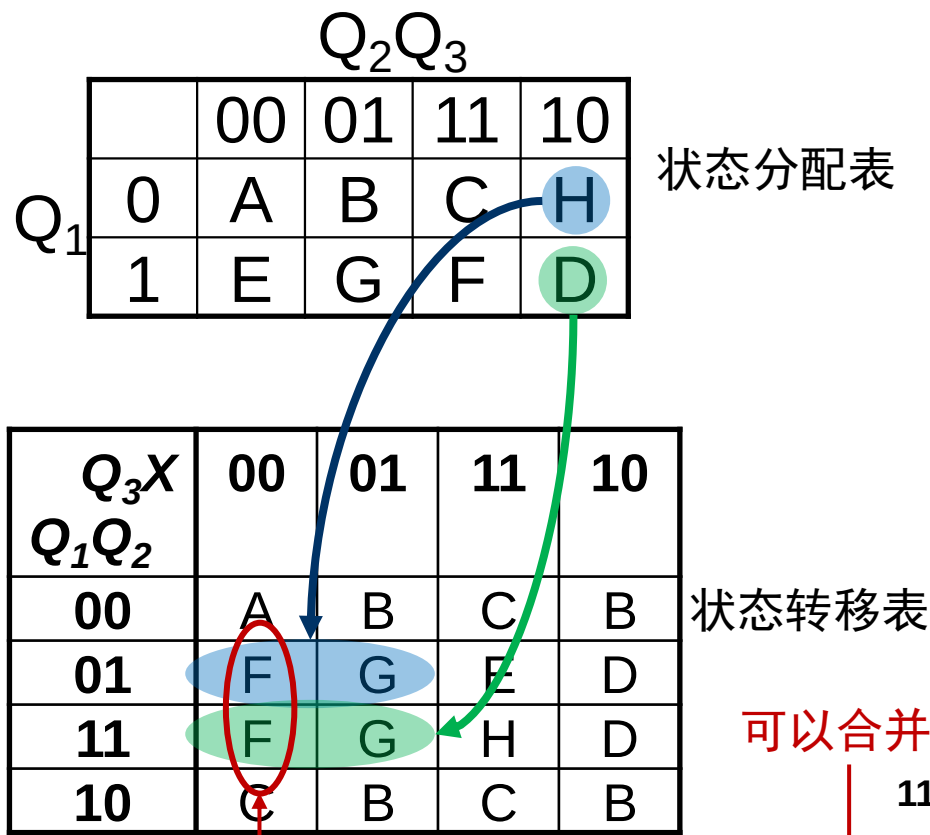
谢谢大家！

- 课后阅读：更多时序逻辑分析化简实例

基于次态和输入/输出的准则

- 最高优先级:为了简化状态表
 - 输入相同的情况下, 次态相同的状态应该具有相邻的状态编码 (需要考虑不同输入情况)

现态	次态		输出	
	x=0	x=1	x=0	x=1
A*	A	B	0	0
B	B	C	0	0
C	D	E	0	0
D	F	G	1	0
E	C	B	0	1
F	D	H	1	0
G	B	C	0	1
H	F	G	0	0

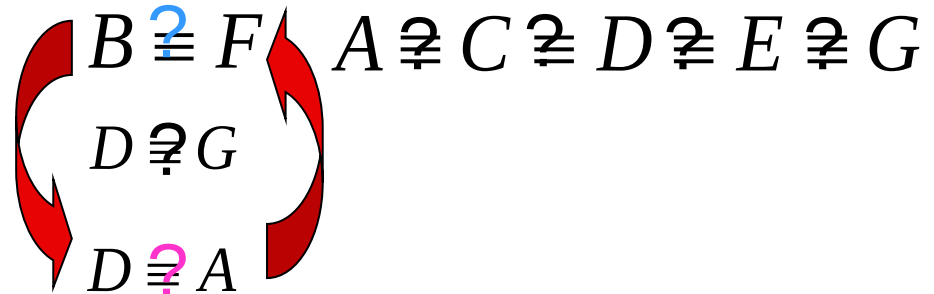


例3：找出最简状态表

现态	次态		输出	
	x=0	x=1	x=0	x=1
A*	A	B	0	0
B	D	C	0	1
C	F	E	0	0
D	D	F	0	0
E	B	G	0	0
F	G	C	0	1
G	A	F	0	0

输出不相同的状态对有:

$A \neq B$ $C \neq B$ $D \neq B$ $E \neq B$ $G \neq B$
 $A \neq F$ $C \neq F$ $D \neq F$ $E \neq F$ $G \neq F$



- 如果两个状态不等价，那么它们一定是可区分的
- 至少有一种输入序列会使得它们作为起始状态时产生不同的输出序列

例3：找出最简状态表

现态	次态		输出	
	x=0	x=1	x=0	x=1
A*	A	B	0	0
B	D	C	0	1
C	F	E	0	0
D	D	F	0	0
E	B	G	0	0
F	G	C	0	1
G	A	F	0	0

B	×					
C	A,F B,E	×				
D	B,F	×	F,D E,F			
E	A,B B,G	×	F,B E,G	D,B E,G		
F	×	D,G	×	×	×	
G	B,F	×	F,A E,F	A,D	B,A G,F	×
	A	B	C	D	E	F

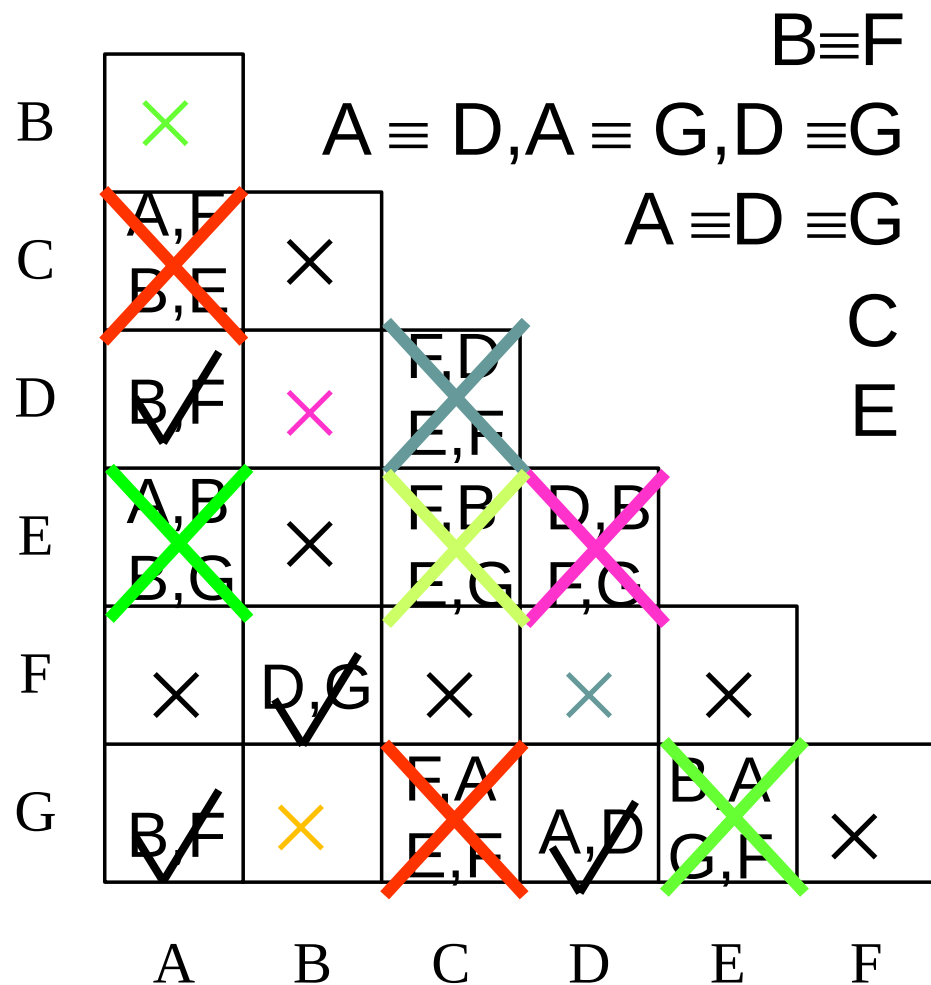
例3：找出最简状态表

现态	次态		输出	
	x=0	x=1	x=0	x=1
A*	A	B	0	0
B	D	C	0	1
C	F	E	0	0
D	D	F	0	0
E	B	G	0	0
F	G	C	0	1
G	A	F	0	0

B	×					
C	A, F B, E	×				
D	B, F	×	E, D E, F			
E	A, B B, G	×	F, B E, G	D, B E, C		
F	×	D, G	×	×	×	
G	B, F	×	F, A E, F	A, D	B, A G, F	×
	A	B	C	D	E	F

例3：找出最简状态表

现态	次态		输出	
	x=0	x=1	x=0	x=1
A*	A	B	0	0
B	D	C	0	1
C	F	E	0	0
D	D	F	0	0
E	B	G	0	0
F	G	C	0	1
G	A	F	0	0



例3：找出最简状态表

$B \equiv F$

$A \equiv D, A \equiv G, D \equiv G$

$A \equiv D \equiv G$

C

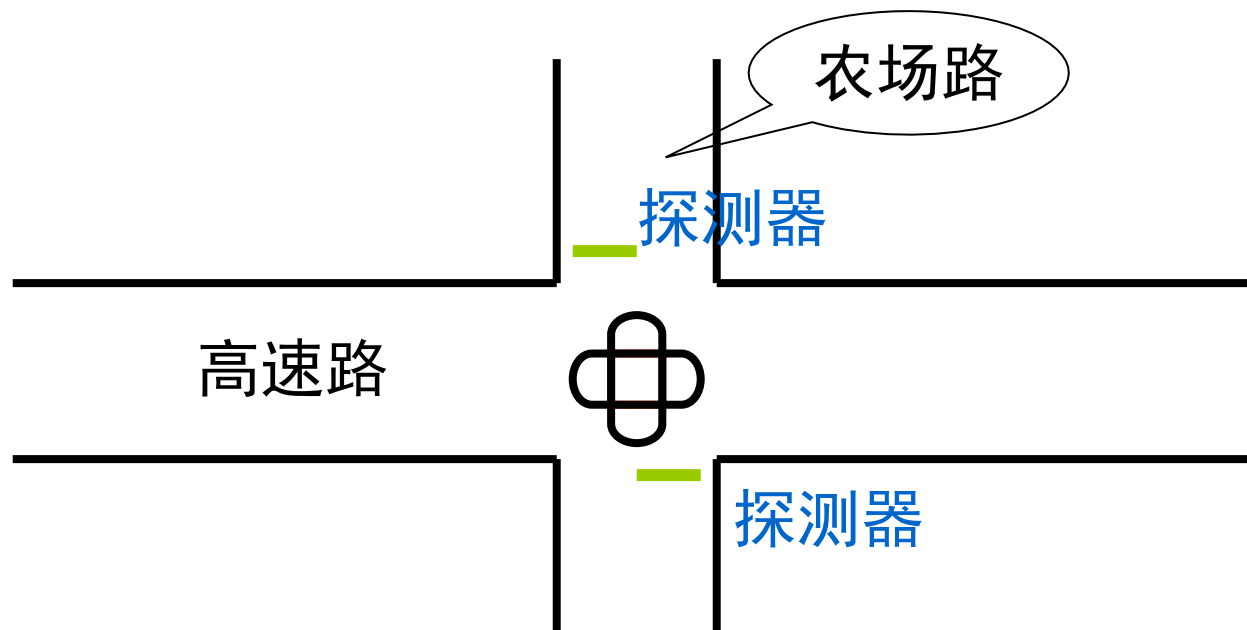
E

现态	次态		输出	
	x=0	x=1	x=0	x=1
A*	A	B	0	0
B	D	C	0	1
C	F	E	0	0
D	D	F	0	0
E	B	G	0	0
F	G	C	0	1
G	A	F	0	0

现态	次态		输出	
	x=0	x=1	x=0	x=1
A*	A	B	0	0
B	A	C	0	1
C	B	E	0	0
E	B	A	0	0

例4：交通灯控制器的设计

- 繁忙的高速路与入烟稀少的农场路的十字路口需要一个交通灯控制电路
- 在农场路上放置探测器，只要该路上有车要过此路口，就发出信号C



例4：交通灯控制器的设计

- 只要探测器在农场路上没有发现任何车辆，高速路方向上的交通灯一直为绿灯
- 如果探测到农场路上有车辆，高速路上的交通灯由绿到黄（需保持一小段时间 TS ）再到红，农场路上的交通灯变绿
- 只要探测器检测到农场路有车，此路上交通灯保持绿灯不变，但是，其时间不能超过一个设定的时间间隔（a long time interval TL ），以保证不造成高速路上的交通拥堵
- 然后，农场路上的交通灯由绿到黄（ TS ）再到红，高速路上的交通灯变绿
- 此时如果农场路上有车等待穿越高速路，高速路上的交通灯仍要保持一个最小时间间隔（ TL ）

Step1: 抽象出FSM

- 输入

- Reset: 控制器是否回到初始状态
- C: 农场路上是否探测到车辆
- TS: 是否到达短时间间隔
- TL: 是否到达长时间间隔

绿灯: 00

黄灯: 01

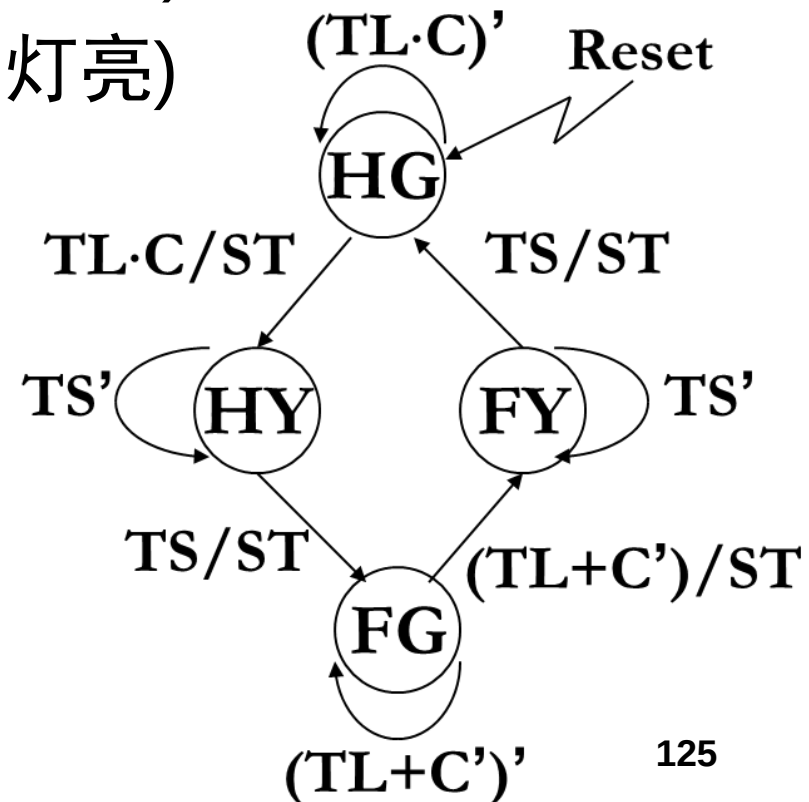
红灯: 10

- 输出

- ST: 复位定时器, 开始长时间间隔和短时间间隔的计时
- H_1H_0 : 高速路上的交通灯显示
- F_1F_0 : 农场路上的交通灯显示

Step1: 抽象出FSM

- HG: 高速路绿灯亮(农场路红灯亮)
- HY: 高速路黄灯亮(农场路红灯亮)
- FG: 农场路绿灯亮(高速路红灯亮)
- FY: 农场路黄灯亮(高速路红灯亮)



Step 2: 状态化简 (略)

输入 C TL TS	现态	次态	输出 ST H ₁ H ₀ F ₁ F ₀
0 × ×	HG	HG	0 00 10
× 0 ×	HG	HG	0 00 10
1 1 ×	HG	HY	1 00 10
× × 0	HY	HY	0 01 10
× × 1	HY	FG	1 01 10
1 0 ×	FG	FG	0 10 00
0 × ×	FG	FY	1 10 00
× 1 ×	FG	FY	1 10 00
× × 0	FY	FY	0 10 01
× × 1	FY	HG	1 10 01

Step3: 状态分配_单点编码

- HG=0001
- HY=0010
- FG=0100
- FY=1000

输入			现态	次态	输出		
C	TL	TS	$Q_3Q_2Q_1Q_0$	$P_3P_2P_1P_0$	ST	H_1H_0	F_1F_0
0	×	×	0001	0001	0	00	10
×	0	×	0001	0001	0	00	10
1	1	×	0001	0010	1	00	10
×	×	0	0010	0010	0	01	10
×	×	1	0010	0100	1	01	10
1	0	×	0100	0100	0	10	00
0	×	×	0100	1000	1	10	00
×	1	×	0100	1000	1	10	00
×	×	0	1000	1000	0	10	01
×	×	1	1000	0001	1	10	01

次态方程和输出方程

$$P_3 = C' \cdot Q_2 + TL \cdot Q_2 + TS' \cdot Q_3$$

$$P_2 = C \cdot TL' \cdot Q_2 + TS \cdot Q_1$$

$$P_1 = C \cdot TL \cdot Q_0 + TS' \cdot Q_1$$

$$P_0 = C' \cdot Q_0 + TL' \cdot Q_0 + TS \cdot Q_3$$

$$ST = C \cdot TL \cdot Q_0 + C' \cdot Q_2 + TS \cdot Q_1 + TL \cdot Q_2 + TS \cdot Q_3$$

$$H_1 = Q_3 + Q_2$$

$$F_1 = Q_1 + Q_0$$

$$H_0 = Q_1$$

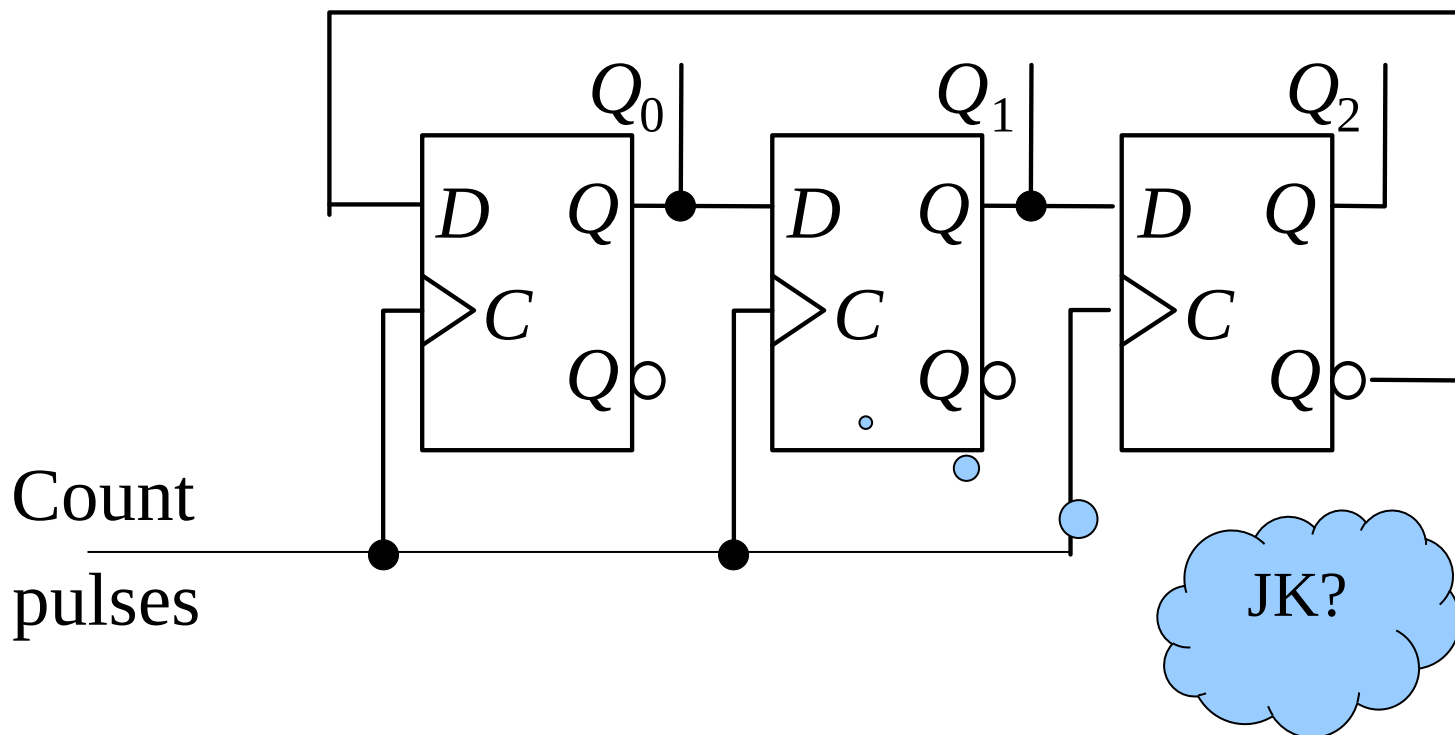
$$F_0 = Q_3$$

实现FSM（略）

- Step4:确定激励方程和输出方程
- Step5:根据激励方程和输出方程，画出两级或者多级逻辑电路图

例5:模六扭环形计数器

- 要求用JK触发器和或非门实现能自启动的带进位输出Z的模六扭环形计数器
- $2N=6$, $N=3$



用JK触发器实现

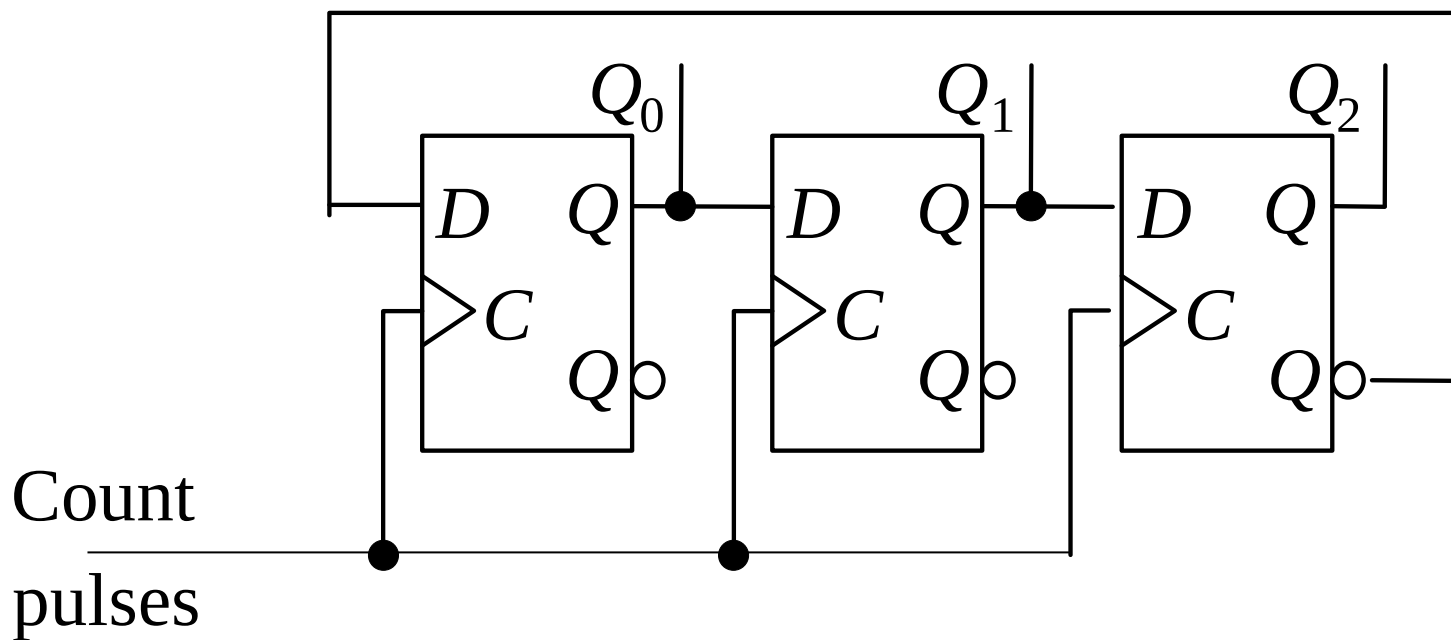
$$Q^+ = D = D(\bar{Q} + Q) = D\bar{Q} + DQ \quad J_0 = D_0 = \bar{Q}_2, K_0 = \bar{D}_0 = Q_2$$

$$Q^+ = J\bar{Q} + \bar{K}Q$$

$$J_1 = D_1 = Q_0, K_1 = \bar{D}_1 = \bar{Q}_0$$

$$J = D, K = \bar{D}$$

$$J_2 = D_2 = Q_1, K_2 = \bar{D}_2 = \bar{Q}_1$$



用JK触发器实现

$$Q^+ = D = D(\bar{Q} + Q) = D\bar{Q} + DQ \quad J_0 = D_0 = \bar{Q}_2, K_0 = \bar{D}_0 = Q_2$$

$$Q^+ = J\bar{Q} + \bar{K}Q$$

$$J_1 = D_1 = Q_0, K_1 = \bar{D}_1 = \bar{Q}_0$$

$$J = D, K = \bar{D}$$

$$J_2 = D_2 = Q_1, K_2 = \bar{D}_2 = \bar{Q}_1$$

